



Trackvance Core

Especificación técnica v1.1

IMPLEMENTACIÓN 0.7.0 · EVOLUCIÓN FUNCIONAL LOCAL

3 octubre 2026 · Estado funcional y evidencia de validación

Arquitectura local y evolución a producto

Monolito modular · Persistencia local · Reglas portables · Evidencia verificable

Documento oficial de referencia. Describe el estado real de Trackvance Core y distingue las capacidades implementadas de los contratos preparados y la infraestructura objetivo.

Trackvance Colombia SAS

Contenido

1. Alcance, estados e invariantes	9
Baseline y evolución aprobada	9
Invariantes que continúan vigentes	9
Estados documentales y límites del alcance	9
2. Arquitectura lógica estable	10
3. Despliegue local de referencia	11
Dimensionamiento observado y aplicado	11
Salud y separación operativa	12
4. Código, stack y dependencias	12
Responsabilidades y archivos de la evolución	12
5. Fuentes y lectores de datasets	13
Recorrido asíncrono vigente	13
Recepción, identidad y límites	13
Snapshots SQL reales y coherencia	13
Perfil completo y publicación	14
Estado, reintentos y diagnóstico	14
Decisión de adquisición: ADR 0020: adquisición asíncrona y snapshots multipart	14
Decisión y diagnóstico	14
Lectura y perfil	15
Almacenamiento, publicación y numeración	15
Interfaz y certificación	15
5A. Módulo Conexiones	16
Límites de responsabilidad	16
Representación y límites	16
Versiones y evidencia	16
SecretStore, permisos y auditoría	16
Fuentes futuras y frontera de salida	17
5B. Data Delivery	17
Adaptación vigente 0.7.0 a volumen	17
Preflight persistido y reutilización segura	18
PreparedDelivery en disco y frontera STARTED	18
Lotes y una transacción remota	19
A. Propósito y flujo funcional	19
B. Identidad y versiones del destino	19
C. Contrato DataSink y límites de responsabilidad	20

D. Mapping, selección y orden	20
E. Texto y temporales: ejemplos exactos	20
F. Target nuevo o existente	21
G. CREATE_AND_LOAD	21
H. APPEND	21
I. OVERWRITE	22
J. UPSERT y claves compuestas	22
K. PostgreSQL: transacción y conteos fiables	22
L. SQL Server: concurrencia y conteos	23
M. Familias de preflight y fallos	23
N. Preview, configuración y publicación	23
O. Workers, lanes y frontera de commit	24
P. Estados del intento y del Run	24
Q. UNKNOWN: revisión operacional append-only	25
R. COMMITTED + PENDING_REPAIR: sólo evidencia local	25
S. Idempotencia y concurrencia	26
T. Métricas: significado y límites	26
U. Receipt y manifest: ejemplo abreviado	26
V. Linaje persistido y auditoría	27
W. Permisos, secretos y aislamiento	27
X. Backup, restauración y compatibilidad	28
Y. Benchmark dedicado y operación en UI	28
Z. Límites y criterio de aceptación	29
6. Carga, esquema y versionado	29
7. Profiling, identificadores y transforms	30
Inferencia inequívoca	30
Transformaciones funcionales	30
8. Reglas portables y Data Intake	31
Condiciones y referencias seguras	31
Editor y resultados	32
Caso operativo: aceptación y evidencia reutilizable	32
9. ReconOps y normalización declarada	32
Políticas y orden de ejecución	33
Clasificaciones estables	33
Caso operativo: claves, tolerancia y diferencias	33
10. Sentinel y métricas históricas	34
Compatibilidad de series	34
Programación local e historia visible	34

Caso operativo: monitoreo y comparación histórica 35

11. Excepciones con validación técnica 35

Criterios de elegibilidad 36

Gestión operativa 36

Caso operativo: cierre respaldado por nueva ejecución 36

12. Centro de Control y navegación 37

Rutas SPA 37

Carga diferida y continuidad de la navegación 37

13. StorageProvider, artefactos y linaje 38

Archivo único y conjunto lógico 38

Resultados, evidencia y consultas 38

Linaje y preservación 38

14. Manifest schema 2 y auditoría 38

Identidad y eventos 39

15. Reportes Excel de negocio y resultado completo 39

16. Ejecución, planificación, PySpark y recuperación 40

Selección y congelación del plan 40

ProcessingEngine y ejecución real 40

Spark local 41

Standalone opcional y ubicación del driver 41

Lease, cancelación, timeout y publicación 41

Paridad y límites comprobados 41

17. Modelo persistente, restricciones y migraciones 42

Migraciones aditivas y baseline conservada 42

Campos y vínculos del inventario completo 42

acquisition_runs 42

acquisition_uploads 44

artifact_links 44

artifacts 45

audit_events 45

configurations 46

dataset_source_bindings 46

dataset_versions 47

datasets 48

delivery_attempts 48

delivery_automation_versions 49

delivery_automations 50

delivery_destination_versions	50
delivery_destinations	51
delivery_input_claims	51
delivery_occurrences	52
delivery_reviews	52
delivery_target_decisions	53
delivery_target_guards	54
delivery_target_policies	54
event_consumptions	55
exception_attachments	55
exceptions	56
external_connection_versions	57
external_connections	58
external_identities	58
findings	59
idempotency_keys	59
internal_notifications	60
jobs	60
metric_history	61
monitor_occurrences	62
monitor_schedule_versions	62
monitor_schedules	63
notification_deliveries	63
oidc_login_attempts	64
outbox_events	64
role_permissions	65
roles	65
runs	66
sessions	67
users	67

18. API, DTOs y errores

68

Adquisición y transferencia	68
Ejecución y preflight	68
Automatizaciones y bandeja	68
Ejemplo sanitizado	68
Inventario ejecutable completo	69

19. Contratos HTTP, identidad y permisos

73

20. Parámetros, operación, backup y upgrade

73

Parámetros efectivos y unidades	73
Cotas fijas y controles complementarios	76

Backup nativo state 6	76
Compatibilidad estricta 0.6.1→0.7.0	77
Actualización controlada de la instalación principal	77
20A. Benchmark reproducible y límites comprobados	78
Objetivos y representación	78
Separación entre pruebas de engine e integración	78
Método de medición por etapa y recorrido integral	78
Evidencia conocida y progresión	80
21. Automatización Delivery, ocurrencias y outbox	82
Configuración versionada y entrada	82
Tres identidades de idempotencia	83
Ticks, atraso, DST y solapamiento	83
Target reconocido y UNKNOWN	83
Outbox atómica y consumidores independientes	83
22. Calidad, certificación y matriz de requisitos	84
Resultados ejecutados 0.7.0 y gates pendientes	85
Matriz de requisitos, implementación y evidencia	87
Reglas de aislamiento y evidencia	88
23. Decisiones vigentes y control documental	88
ADRs y responsabilidades	88
Fuente, candidato, revisión y publicación	88
Registro de revisión	89
24. Antecedente: cambios funcionales y técnicos 0.5.1	89
1. Reparación local de evidencia COMMITTED	89
2. Revisión operacional de UNKNOWN	89
3. Semántica uniforme de métricas	89
4. Desglose UPSERT PostgreSQL por capacidades	90
5. Regresión temporal exhaustiva	90
6. Linaje canónico sin migración histórica	90
7. Benchmark exclusivo de Delivery	90
8. División del frontend en módulos de carga diferida	90
25. RBAC dinámico y administración real de roles	91
26. Usuarios, credenciales y primer acceso	93
Alta e identidad estable	93
Emisión efímera y entrega externa	93
Modal obligatorio y ciclo de vida del plaintext	94
Regeneración y concurrencia	95

Login, expiración y primer acceso	95
Estado, permisos y auditoría	96
27. SSO Microsoft y Google	96
28. Notificaciones internas y conservación de historia	97
Bandeja vigente por usuario	97
Destinatarios y autorización	97
Historia y credenciales	97
29. Persistencia y recuperación de identidad/automatización	98
30. Auditoría técnica de Data Delivery	98
Cambio funcional respecto a 0.5.1	98
Uso del builder	98
Qué significan las columnas	99
Esquemas y compatibilidad de tipos	99
Modelo persistente y migración	99
Fronteras transaccionales y permisos	100
Drift, fallos y estado UNKNOWN	101
Contratos y evidencia	101
Backup/restore, pruebas y límites	102
31. Antecedente: cambios funcionales y técnicos 0.6.0	102
A. Roles y catálogo de permisos	103
B. Usuarios, credenciales y SSO	103
C. Notificaciones	104
D. Auditoría de publicación	104
Resultados del antecedente	105
32. Configuración y prueba real SSO	105
Preparación común	105
A. Cuenta personal Microsoft: Outlook, Hotmail, Live	105
B. Microsoft Entra work/school	106
C. Gmail personal	106
D. Google Workspace	106
Verificación y desvinculación	106
Errores frecuentes	107
33. Operación de credenciales y retiro de SMTP	107
34. Matriz completa de autorización	108
35. Antecedente: cambios funcionales y técnicos 0.6.1	113

1. Situación 0.6.0	113
2. Problema funcional	113
3. Decisión de producto	113
4. Solución funcional	113
5. Solución técnica	113
6. Cambios API	113
7. Cambios UI	113
8. Persistencia	113
9. Seguridad y límites de memoria	113
10. Compatibilidad	114
11. E2E y privacidad de evidencia	114
12. Upgrade Docker real	114
13. CI y trazabilidad documental	114
14. Limitaciones	114
15. Pendientes posteriores	114
Resultados ejecutados 0.6.1	114

1. Alcance, estados e invariantes

Baseline y evolución aprobada

IMPLEMENTACIÓN 0.7.0 conserva el nombre documental v1.1 y evoluciona la baseline 0.6.1 exacta 6fac26b3648cb4a4b50c094ef12c1e103bc97ddd. Aquella revisión tenía 31 tablas, Alembic 0012_delivery_target_audit, cinco servicios Compose, Polars operativo y cargas síncronas de 10 MiB/100.000 filas. PySpark, adquisición durable, automatización Delivery y bandeja funcional no estaban operativos. Las pruebas iniciales de esta evolución verificaron 1.197 tests backend y 174 frontend. Son diagnóstico de partida; la certificación 0.7.0 tiene evidencia independiente.

La revisión implementa adquisición en background, lectura y publicación por lotes, datasets multipartes, PySpark local y Standalone opcional, preflight persistido, preparación Delivery sellada en disco, scheduler independiente, Intake→Delivery, outbox PostgreSQL y bandeja personal. Las interfaces normales registran y siguen estos trabajos. El modelo final tiene 42 tablas y tres migraciones nuevas 0013..0015; los bytes de 0001..0012 se conservan. La revisión de implementación y las ejecuciones conocidas se registran en el capítulo 22. El HEAD documental final y su CI se registran externamente para evitar la referencia circular del PDF.

Invariantes que continúan vigentes

- DatasetVersion, revisiones de configuración, resultados publicados y manifiestos

conservan identidad, hashes y linaje. Los contratos nuevos no reinterpretan archivos únicos, perfiles ni evidencia de releases anteriores.

- EXACT_OBSERVED preserva espacios, ceros iniciales, Unicode, null y precisión.

Una muestra sirve para preview; el perfil, unicidad, referencias, decisiones y resultados completos se calculan sobre toda la población.

- El worker publica metadata únicamente mientras mantiene la propiedad del Job

y su lease vigente. Perder un lease invalida la autoridad de un intento antiguo.

- La solicitud HTTP termina al registrar el trabajo durable. Cerrar una página

después del registro no cancela el job. Una transferencia incompleta no es un archivo recibido y no puede prometerse continuidad al cerrar el navegador.

- SUCCESS expresa terminación técnica. APPROVED, APPROVED_WITH_WARNINGS,

REJECTED y decisiones Sentinel/Delivery expresan resultados de negocio distintos.

- DataSink conserva una única transacción remota por entrega; un lote no implica

commit. UNKNOWN nunca dispara un replay automático. Reparar evidencia no ejecuta DDL/DML. Una cancelación tardía no revoca un COMMITTED confirmado.

- Identidad y permisos se resuelven desde las cuentas y roles vigentes. Un actor

SYSTEM describe el disparador; un usuario real autoriza cada automatización.

- Los secretos fuente y destino se separan. Los executors Spark, scheduler y

consumidores de eventos no reciben credenciales SQL de negocio.

- notification_deliveries conserva historia de 0.6.0. La nueva bandeja usa otras

tablas y no envía correo, credenciales, valores de negocio ni errores del driver.

Estados documentales y límites del alcance

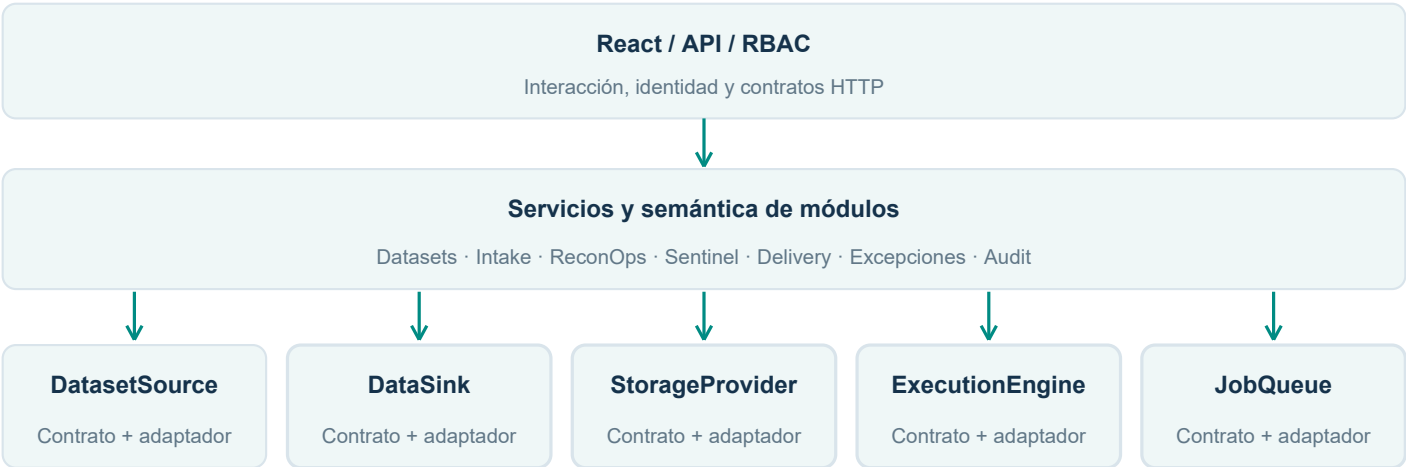
IMPLEMENTADO describe código funcional. PROBADO exige ejecución con evidencia; PREPARADO no sustituye una prueba. NOT_RUN_RESOURCE_LIMIT conserva un escalón opcional sin ejecutar. Un gate obligatorio pendiente/fallido impide declarar CERRADA la release. Las tablas de validación no heredan números de versiones anteriores ni reclasifican skips como PASS.

Las tablas reflejan el corte de evidencia anterior a la promoción del commit que contiene este PDF y al upgrade principal. El informe final externo registra el HEAD documental, todos sus jobs y la verificación efectiva de la instalación; completa esos dos gates sin volver a modificar los bytes ya revisados del PDF.

Quedan fuera de esta evolución conectores adicionales, Redis/Celery, mensajería externa, object storage remoto, Kubernetes/Helm/Terraform, masking, gobierno, observabilidad empresarial, retención avanzada y notificaciones de asignación. Standalone en el mismo PC prueba executors separados; no añade RAM/CPU física ni certifica escalamiento multinodo o capacidad productiva universal.

2. Arquitectura lógica estable

Trackvance conserva el monolito modular FastAPI/React. Los procesos comparten modelos, reglas y contratos, pero tienen funciones y credenciales distintas. PostgreSQL almacena metadata, cola, leases, ocurrencias y outbox; el volumen de artefactos contiene originales, partes, descriptores y evidencia verificable. No se introduce otro broker ni una cola independiente de adquisición.

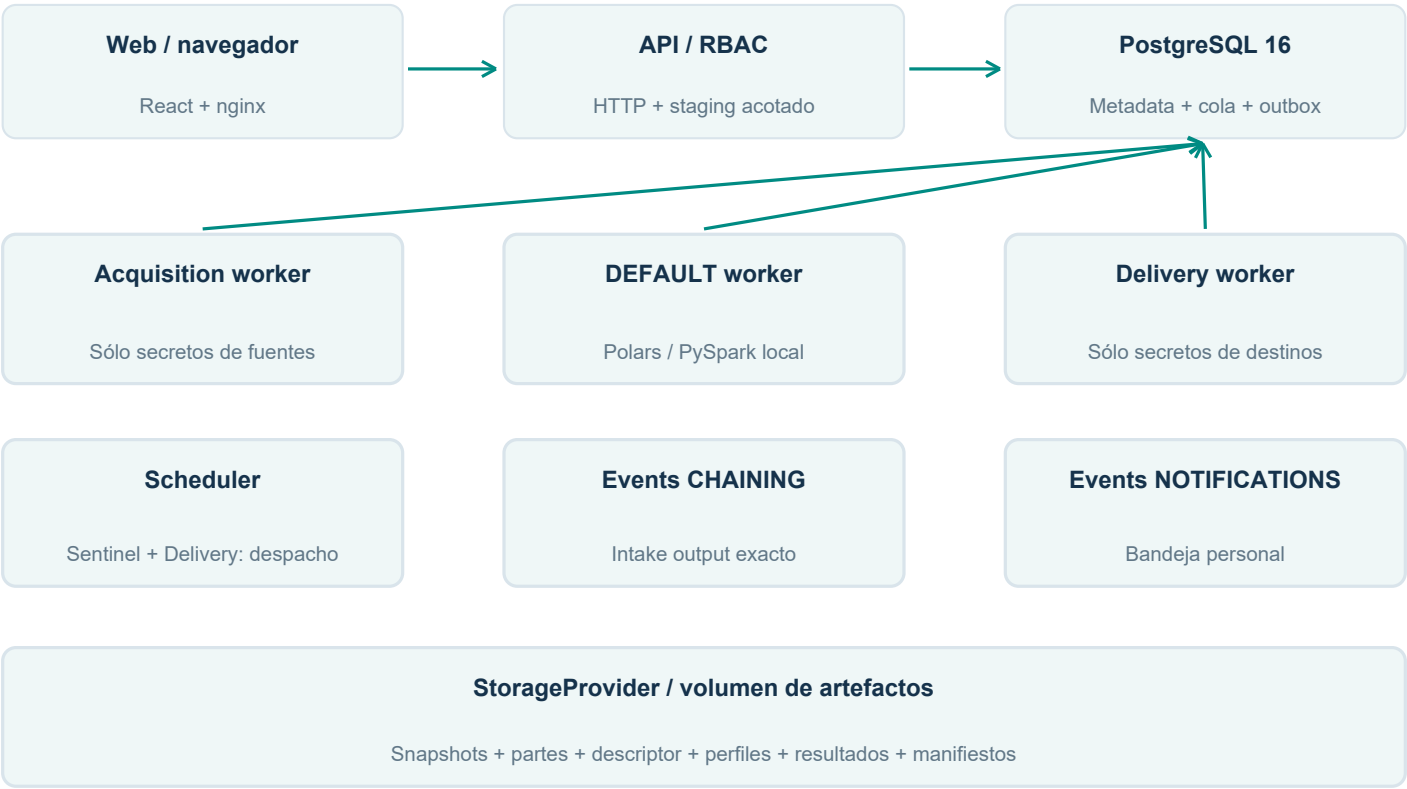


Puerto		Contrato operativo 0.7.0
DatasetSource		Abrir snapshot por revisión congelada; iterar lotes acotados y cerrar el cursor. Conectores PostgreSQL/SQL Server.
StorageProvider		put_file/materialize para históricos; put_dataset/dataset_paths para conjuntos ordenados y verificados. Temporales pertenecen al proveedor que los asignó.
ProcessingEngine		Ejecutar semántica portátil de reglas, transforms, condiciones y métricas sobre entradas materializadas. Polars y PySpark.
ExecutionEngine		Integrar Run/Job, leases, cancelación, almacenamiento, evidencia, decisión y publicación. No equivale a una SparkSession.
ExecutionPlanner		Congelar selección AUTO/POLARS/PYSPARK y explicar capacidades, costes, recursos y rechazos.
JobQueue		Identidad Run XOR AcquisitionRun, claim persistente, heartbeats, lease, idempotencia y recuperación por lane.
DataSink		Validar y preparar por lotes; ejecutar transacción SQL exclusiva en delivery-worker.
Outbox / consumidores		Eventos durables y consumos independientes CHAINING/NOTIFICATIONS, deduplicación, leases y reintentos limitados.

El servicio de aplicación revalida identidades, construye evidencia y hace el fence de publicación. El engine calcula resultados; los executors no escriben metadata ni tablas de destino. Un fallo de notificación no cambia el proceso original, y repetir consumo no debe crear una entrega lógica adicional.

3. Despliegue local de referencia

Compose inicia nueve servicios explícitos. Todos conservan restart: "no". La instalación principal mantiene PostgreSQL 16, su proyecto/volúmenes y su puerto/origen. La evolución no requiere Standalone ni puertos administrativos.



Servicio	Responsabilidad / acceso
postgres	Metadata, Run/AcquisitionRun, Job, outbox, inbox, ocurrencias e inventarios.
api	Autenticación, permisos, staging, inspección, registro durable, preview/paginación y controles HTTP. Acceso separado a ambos SecretStore para administrar.
worker	Lane DEFAULT: Intake, ReconOps y Sentinel, Polars/PySpark. Sin secretos fuente/destino.
acquisition-worker	Lane ACQUISITION: abrir fuentes, leer, materializar, perfilar y publicar. Sólo secretos fuente.
delivery-worker	Lane DELIVERY: preflight persistido, preparación y DataSink. Sólo secretos destino.
scheduler	Despachar Sentinel y Delivery. No procesa datasets ni abre sinks.
events-chaining	Consumir eventos Intake, resolver política y registrar ocurrencias/Jobs.
events-notifications	Crear resumen personal por transición, con autorización vigente al consultar.
web	React compilado, nginx, proxy /api y único puerto localhost del perfil estándar.

Dimensionamiento observado y aplicado

El host revisado dispone de 16 CPU y 32 GiB RAM; Docker Desktop informa 16 CPU y aproximadamente 15,2 GiB. Se observó margen de RAM y más de 1 TiB libre de disco. No se modificó globalmente el presupuesto Docker. Los límites Compose son aplicados por cgroups, mientras los presupuestos del planner son estimaciones. API tiene 2 GiB/2 CPU; DEFAULT 3 GiB/2 CPU; adquisición y Delivery 1,5 GiB/1 CPU cada uno; PostgreSQL 1 GiB/1 CPU; web 256 MiB/0,5 CPU; cada proceso ligero 512 MiB/0,25 CPU. No son reservas de memoria consumidas permanentemente.

Cada worker procesa un Job a la vez. Un despliegue con más réplicas comparte claims/fences, pero debe sumar su presupuesto físico y no puede crear dos dueños vigentes. El perfil Standalone añade master y dos executors, sin sustituir el driver DEFAULT ni aumentar recursos del host. Se activa sólo de manera explícita.

Salud y separación operativa

La API prueba DB, Alembic y escritura de un temporal propio. Los workers tienen heartbeat por lane. Scheduler y consumidores tienen heartbeats separados; component_status exige actualización reciente. Un Spark largo no impide que los procesos ligeros sigan despachando o notificando. OFFLINE no debe ocultarse como un worker de otra lane activo.

Las sondas de esos tres procesos importan component_health, un lector de JSON sin motores analíticos ni conexión a DB. La antigüedad admitida es $0 \leq \text{edad} < 30$ segundos; fechas futuras, JSON inválido o rutas resueltas fuera del storage devuelven OFFLINE. Con 0,25 CPU y 256 MiB se midieron 0,292 segundos de importación, frente a 4,082 segundos al importar dispatcher. Compose conserva timeout 5 segundos y los límites declarados. El diagnóstico aislado registra exit code/duración de sondas, OOM y límites, sin guardar entorno ni texto de errores con credenciales. Las esperas previas en CI se conservan como fallos; no se atribuyen a OOM o a una causa completa que no fue demostrada.

El modo directo PowerShell inicia los mismos procesos backend separados y web Vite. Las rutas de secretos se aíslan por proceso. Java 17 es necesario para PySpark; el contenedor backend lo incluye. En un host sin Java, el planner rechaza PYSPARK con ENGINE_UNAVAILABLE; no hace fallback silencioso.

4. Código, stack y dependencias

El runtime fija dependencias mediante backend/uv.lock y frontend/pnpm-lock.yaml. Backend integra FastAPI, SQLAlchemy/Alembic, Polars, PyArrow, DuckDB, PySpark 4.0.3, psycopy y el adaptador SQL Server existente; frontend React/TypeScript usa componentes y TanStack Query del producto. La imagen backend incorpora Java OpenJDK 17 y Python soportado; se validan las versiones reales en cada ejecución. La documentación oficial de Apache Spark 4.0.3 especifica Java 17 o 21 y Python 3.9+; las pruebas Docker fijan el conjunto compatible, sin imagen latest para Spark/PySpark ni instalación a voluntad en cada carga del usuario.

Responsabilidades y archivos de la evolución

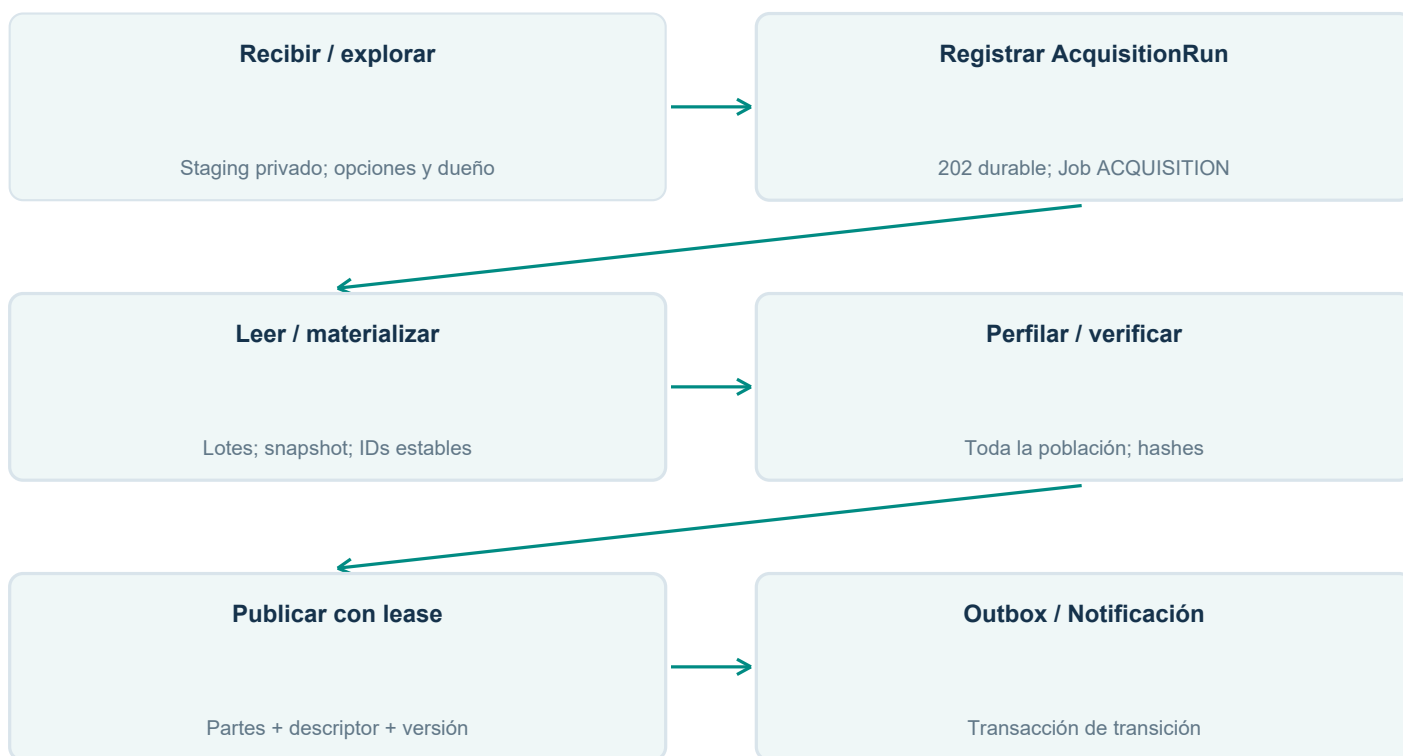
Archivos / grupo	Responsabilidad
acquisition*, batch_readers, dataset_scans	Registro, recepción, límites, lectura acotada, perfil global, cancelación y publicación.
artifactstore	Verificación de archivos únicos y descriptor de conjunto; registro de partes y linaje.
planner, spark_engine, spark_execution, services	Coste real de partes, selección, ejecución distribuida de semántica portable, resultado y fence.
delivery_streams, delivery_validation, delivery_service, data_sinks	Escaneo repetible, claves globales, preparación sellada, preflight durable y transacción remota.
automation*, dispatcher, scheduler	Revisiones, políticas de entrada, responsable, ticks, ocurrencias y serialización de target.
events, notifications_api	Outbox atómica, consumos CAS, retries, resumen personal y consultas autorizadas.
permissions, api, worker	Contratos HTTP, autoridad vigente, lanes y protección contra publicaciones de owners vencidos.
frontend/features/datasets, delivery, notifications, runs	Recepción/seguimiento, automatización, preflight, bandeja y selector de engine.
scripts/verify_storage, docker_state, backup_local	Huella state 6, conjuntos, separación de secretos y compatibilidad estricta.

Las reglas de negocio permanecen en el núcleo portable. PySpark distribuye llamadas a esos kernels y las operaciones globales de agrupación/referencias; no se mantiene un catálogo de reglas diferente dentro de funciones SQL Spark. Este diseño preserva Decimal y contratos Unicode/temporales que el tipo Decimal nativo de Spark no representa universalmente. Los dominios que exceden cotas explícitas de fila/grupo se rechazan antes de producir resultados incorrectos.

5. Fuentes y lectores de datasets

Recorrido asíncrono vigente

La UI normal permite elegir archivo o conexión, inspeccionar y confirmar opciones, registrar una AcquisitionRun, seguir lectura/materialización/perfil y abrir su DatasetVersion completa. La recepción y el procesamiento son dos fases distintas. XHR informa bytes realmente transferidos; después del 202 la UI consulta el historial persistido y no inventa porcentaje si el total es desconocido.



Recepción, identidad y límites

POST /datasets/uploads/stage escribe application/octet-stream a un archivo privado por chunks, mientras verifica tamaño observado, SHA-256, usuario, organización, formato y TTL. Content-Length desconocido no desactiva la cota. La adquisición consume el upload_id propio una sola vez y congela opciones, overrides, límites y request_hash. No existe una DatasetVersion ficticia antes de materializar. Una transferencia incompleta no se publica ni registra como recibida; cambiar de página después de registrar el Job no lo cancela.

CSV/TXT/TSV conservan registros CSV multilínea y número estable del origen. JSONL/NDJSON lee registros por línea; Parquet recorre batches de sus partes. Un lote tiene doble cota de filas y bytes observados; una sola celda gigante produce rechazo funcional. XLSX y JSON array/objeto conservan lectura limitada de 10 MiB y 100.000 filas por defecto; no se anuncian como streaming ilimitado. Opciones de hoja y delimitador permanecen explícitas e inmutables en la versión.

Snapshots SQL reales y coherencia

La revisión de conexión y la selección schema/objeto quedan congeladas. El worker revalida cuenta, permisos y conexión habilitada antes de abrir la lectura. Sólo acquisition-worker obtiene el secreto de esa revisión; calidad/Spark trabaja sobre snapshots materializados. PostgreSQL usa cursor server-side bajo una transacción REPEATABLE READ de sólo lectura. SQL Server usa lectura transaccional SERIALIZABLE y fetchmany, con implicaciones de bloqueo documentadas. No afirma una snapshot isolation que no esté configurada en el servidor.

Las expresiones de volumen OCTET_LENGTH/DATALENGTH previenen que una celda SQL arbitrariamente ancha se materialice en el driver antes del control de tamaño. No se corta el valor ni se convierte en null. Cancelar o perder el lease cierra cursor/transacción y descarta el intento privado. Recuperar crea una extracción entera nueva; no concatena lotes de snapshots diferentes. La posición SQL es número de registro del snapshot, nunca línea física de la tabla remota.

Perfil completo y publicación

La materialización conserva todos los valores observados y añade identidad interna de registro. DuckDB calcula counts/distintos/duplicados/perfiles sobre el conjunto completo con memoria y spill limitados. Una muestra sólo alimenta preview. Los tipos y columnas de negocio excluyen los identificadores internos. La publicación verifica partes, esquema, conteos y hashes; DatasetVersion, relaciones y SUCCESS/outbox se confirman bajo fence en la misma transacción de metadata. Los archivos aún no referenciados pertenecen al intento privado.

Consultar el perfil presenta las estadísticas globales persistidas y una muestra de hasta veinte filas y 8 MiB de JSON UTF-8. La muestra se lee con Arrow, con un lote limitado por la cota global de anchura antes de convertirlo a objetos Python; una versión histórica sin esa cota usa un registro por lote. La verificación de hashes y esquema se conserva, y los conteos físicos de las partes se obtienen del footer Parquet. Esta presentación no ejecuta un nuevo perfil analítico.

Fallos de formato, corrupción, recursos, cuenta revocada, timeout o cancelación no publican una versión parcial ni cambian la anterior. El cleanup toma los locks Acquisition→Job→Upload, conserva leases/trabajos activos y artifacts referenciados, y aplica grace/TTL únicamente a staging abandonado del dueño verificado.

Estado, reintentos y diagnóstico

QUEUED→RUNNING→SUCCESS/FAILED/CANCELLED. Las etapas READING, MATERIALIZING, PROFILING y PUBLISHING describen avance real. processed_rows/processed_bytes son cantidades observadas; total_rows/total_bytes pueden ser null. attempt_id separa extracciones. output_version_id sólo existe al éxito completo. La UI muestra fecha, duración, error sanitizado, cancelación e identidad de salida. El request idempotente devuelve la misma adquisición; reutilizar la misma clave con cuerpo diferente se rechaza. Refresh registra una nueva adquisición sobre la revisión vigente que queda congelada al registrar, sin reescribir la anterior.

Decisión de adquisición: ADR 0020: adquisición asíncrona y snapshots multipart

- Estado: implementado; certificación de volumen registrada por tier y motor.
- Fecha: 2026-10-03.

Decisión y diagnóstico

La adquisición anterior devolvía un DataFrame completo, perfilaba toda la población en memoria y publicaba un Parquet único dentro de la petición HTTP. Aunque SQL utilizaba fetchmany, concatenaba los registros; el límite de 100.000 filas no constituía un contrato para millones de registros. El parser multipart también recibía la carga antes de aplicar el límite del endpoint.

La recepción nueva utiliza un cuerpo binario directo con tamaño anunciado y observado, reserva de disco y limpieza de transferencias incompletas. La respuesta 201 confirma exclusivamente la transferencia y una inspección de hasta 100 registros. Las opciones de hoja/delimitador y los tipos confirmados se registran después mediante 202, con AcquisitionRun, límites efectivos congelados y un Job de lane ACQUISITION. La petición no materializa ni perfila la población completa. Las rutas históricas sincrónicas conservan sus contratos.

El worker dedicado posee únicamente secretos de fuentes. Congela revisión de conexión, selección de tabla/vista, esquema nativo y usuario; revalida permiso y estado antes de publicar. PostgreSQL utiliza transacción read-only REPEATABLE READ y cursor de servidor. SQL Server utiliza FreeTDS dbnextrow, fetchmany acotado y SERIALIZABLE: conserva una lectura consistente mediante locks y puede bloquear escrituras concurrentes. No declara aislamiento SNAPSHOT no habilitado. Las consultas CASE con sentinel detectan celdas demasiado grandes en el servidor antes de enviarlas al driver; el sentinel falla toda la adquisición y nunca publica columnas truncadas. Los timeouts de consulta de cada conexión continúan vigentes, además del deadline acumulado de adquisición.

Lectura y perfil

CSV/TXT y NDJSON se recorren por registros. NDJSON descubre completamente la unión de columnas sin conservar registros y vuelve a recorrer el archivo para generar lotes. Parquet verifica footer, tamaño expandido y tamaño de grupos de lectura, materializando ventanas acotadas. XLSX y JSON no lineal tienen un contrato menor explícito: 10 MiB y 100.000 filas por defecto; su incumplimiento falla y recomienda un formato de volumen, sin truncar la población. Los límites se configuran mediante `TRACKVANCE_ACQUISITION_*` y se conservan en cada adquisición.

Los defaults de volumen son 1 GiB de archivo, cinco millones de registros, 2 GiB de valores UTF-8 observados, lotes de 5.000 filas / 8 MiB, celdas de 64 KiB, presupuesto analítico de 256 MiB y reserva de 512 MiB de disco. Los fetches además reducen filas por el peor ancho permitido; la cantidad de registros del lote nunca es una autorización para un lote ilimitado en bytes. Un grupo Parquet mayor que la mitad del presupuesto de memoria se rechaza antes de descomprimirlo.

La inferencia final, `null/distinct/uniqueness` y las validaciones de overrides cubren todas las partes. DuckDB realiza las agregaciones globales con memoria limitada, un hilo y `spill` privado; no se suman distintos calculados por lote. Estadísticas monetarias se acumulan con Decimal y precisión suficiente para todo el conjunto. Pruebas comparan el resultado entero con `profile_frame`, incluyendo decimales de más de 28 dígitos. Se conservan `null` frente a texto vacío, Unicode compuesto/descompuesto, espacios, ceros iniciales y valores temporales observados.

Los decimales científicos se comprueban antes de convertirlos a notación fija: signo, cantidad de dígitos y exponente determinan la longitud ASCII exacta, incluidos cero y escala. Un literal corto como `1e999999999` se rechaza por la cota de 64 KiB sin construir su representación expandida ni convertir a float. Los archivos devuelven `ACQUISITION_CELL_LIMIT`; los conectores traducen ese límite a `SOURCE_SIZE_LIMIT` y cierran cursor/conexión. Las regresiones verifican el borde de 65.536 bytes, signo/escala y JSON/JSONL con pico de asignación Python menor que 1 MiB para ese rechazo; no atribuyen ese pico al proceso de adquisición completo.

Almacenamiento, publicación y numeración

StorageProvider publica un descriptor JSON versión 1, MIME `application/vnd.trackvance.parquet-set+json`, y registra cada Parquet como Artifact inmutable. El descriptor declara esquema físico, orden, filas, bytes, SHA-256 y locators relativos de cada parte. `dataset_paths` valida límites, traversal, identidad, hash/tamaño, esquema y conteos antes de escanear. Las partes tienen linaje `DATASET_PART`; versiones y resultados conservan linaje de fuente, run y parent. Los Parquet únicos anteriores mantienen su identidad y lectura.

Las partes físicas contienen `__tv_record_number Int64`; el esquema de negocio y la muestra excluyen esa columna. CSV/TXT conserva la línea física inicial de cada registro, incluyendo campos multiline; JSON/Parquet usa ordinal de registro; SQL declara `SNAPSHOT_ROW` sin inventar una línea física de tabla. Accepted output conserva el número del origen en lugar de reenumerarlo. El contrato de bytes de DatasetVersion sigue asociado al Artifact que define su SHA; el costo materializado se declara separadamente como `ingestion_metadata.canonical_size_bytes`.

La publicación final bloquea Acquisition → Job → Dataset, comprueba owner, lease, attempt y cancelación, y confirma versión READY, artifacts, linaje y estados terminales en una sola transacción. Un fallo, cancelación o lease perdido conserva las versiones anteriores y no expone una versión parcial. El helper de publicación no hace commit; Spark lo utiliza bajo el fence del caller. Los intentos perdidos pueden dejar bytes inmutables huérfanos, pero no sustituir otros artifacts.

La recuperación reclama un lease vencido con attempt nuevo, hasta tres intentos. La limpieza usa locks en el mismo orden y elimina sólo intentos privados muertos después de gracia, transferencias RECEIVED vencidas sin adquisición y staging de adquisiciones FAILED/CANCELLED vencidas sin salida. Excluye leases vivos, trabajo QUEUED y toda referencia Artifact/DatasetVersion. No recorre directorios de negocio.

Interfaz y certificación

La UI muestra transferencia real de XHR, confirmación de recepción, opciones, registro 202, etapa, filas, bytes observados, duración, error seguro y cancelación cooperativa. Polling e historial consultan el estado durable; salir de la pantalla no cancela un Job registrado. Sólo SUCCESS selecciona la versión publicada.

`scripts/tests/volume_cycle.py` y `frontend/tests-e2e/volume.spec.ts` registran fixtures streaming reproducibles y resultados por fase/tier. Un límite de recurso o una fase no ejecutada figura explícitamente; los tiers mayores no se certifican por extrapolación. Las pruebas rápidas usan DB y almacenamiento desechables.

5A. Módulo Conexiones

IMPLEMENTADO. Permite crear, probar, editar, deshabilitar y eliminar lógicamente conexiones PostgreSQL y SQL Server. La conexión registra nombre, motor, host, puerto, base, usuario y opciones controladas. La contraseña se envía únicamente al guardar/probar y nunca vuelve en una respuesta. El formulario exige una prueba vigente y backend vuelve a verificar antes de guardar. La deshabilitación puede hacerse aunque la fuente esté caída.

Después de conectar, el usuario explora schemas, tablas y vistas accesibles. La vista previa muestra nombres, tipos nativos/lógicos, nulabilidad y hasta 100 registros. La detección del esquema usa catálogos; no escanea la tabla completa. El recorrido normal registra una AcquisitionRun con la selección congelada; el worker publica el Dataset, su binding de origen y una DatasetVersion canónica Parquet mediante StorageProvider. La ruta síncrona legacy conserva su contrato para cargas pequeñas. Los motores de calidad siguen leyendo versiones locales inmutables.

Límites de responsabilidad

El PostgreSQL interno guarda solo metadata del sistema. PostgreSQL/SQL Server externos son fuentes del usuario y tienen credenciales independientes. Ningún adaptador escribe filas en la fuente. No se recibe SQL arbitrario; se seleccionan identificadores previamente descubiertos y citados según el motor. PostgreSQL usa transacciones read-only. En SQL Server la intención read-only es informativa: es obligatorio usar una cuenta con permisos SELECT mínimos, sin permisos de escritura.

Las fuentes externas requieren acceso de red desde API. El overlay offline restringe esa salida; el Compose base permite conexión a fuentes accesibles, sin añadir dependencias cloud. Para fuentes en el PC bajo Docker Desktop puede usarse host.docker.internal. La configuración local conserva restart=no.

Representación y límites

El preview aplica LIMIT/TOP en origen; el snapshot lee por lotes de 100 hasta 100.000 filas, 100 columnas y 64 MiB de valores normalizados. Cada celda tiene límite de 64 KiB. Una fila adicional detecta el exceso y se rechaza la importación completa: nunca se guarda una muestra parcial como si fuera un dataset completo. Timeout configurable de conexión 1..15 segundos; consulta 1..60 segundos. Este contrato describe la ruta legacy síncrona. El recorrido normal 0.7.0 usa adquisición asíncrona y sus cotas/consistencia documentadas en el capítulo 5.

Los valores se conservan como texto/null: Decimal sin redondeo float, fechas ISO, Unicode y espacios originales. Los identificadores siguen la política id/*_id y permiten override. Los binarios son Base64. SQL Server timestamp se interpreta como rowversion; datetimeoffset (hasta escala 6) y PostgreSQL timestampz conservan TIMESTAMP con offset. timestamp without time zone, datetime, datetime2 y smalldatetime se adquieren como STRING: no se inventa una zona horaria y Delivery conserva el texto exacto. datetimeoffset(7) también se adquiere como STRING para conservar el séptimo dígito. El orden DATABASE_UNSPECIFIED depende del motor; SNAPSHOT_ROW es posición en el snapshot, no un número físico en la tabla externa.

Versiones y evidencia

La conexión tiene ID estable y revisión para concurrencia optimista. Cada edición crea ExternalConnectionVersion inmutable con hash de configuración y referencia opaca de credencial. Un binding conserva dataset, conexión, schema, tabla/vista y overrides. Actualizar desde fuente usa la configuración vigente y crea una versión nueva. Fallos de acceso no modifican versiones anteriores.

ingestion_metadata.source conserva connection_id, connection_version_id, connection_version, source_type, schema_name, object_name, object_kind, config_hash y captured_at. El linaje SOURCE_SNAPSHOT apunta a la revisión de conexión; REFRESH_OF apunta al snapshot anterior. Los manifests de runs incluyen la identidad de fuente de cada DatasetVersion. El run original y su hash permanecen inmutables aunque cambie la conexión.

SecretStore, permisos y auditoría

El contrato put/get/delete permite sustituir el proveedor local por Azure Key Vault, AWS Secrets Manager o HashiCorp Vault. La implementación local cifra con Fernet autenticado, vincula credencial a organización/referencia y persiste en connection_credentials; la clave maestra reside en connection_keys. En Linux los directorios son 0700 y archivos 0600; Windows directo hereda ACL del usuario. El backup debe proteger ambos volúmenes por separado del almacenamiento de artifacts. Una clave perdida no se reemplaza silenciosamente si ya existen secretos. Rotación automática de clave maestra permanece pendiente.

La contraseña guardada solo se reutiliza en pruebas o ediciones cuando host, puerto, base, usuario y modo TLS coinciden con la versión vigente. Cambiar cualquiera exige una contraseña explícita; su ausencia devuelve 422

PASSWORD_REQUIRED_FOR_ENDPOINT_CHANGE antes de contactar el destino. Nombre y timeouts pueden cambiar conservando el secreto. Esto impide redirigir una credencial guardada a otro servidor desde un formulario de edición.

Transporte cifrado por defecto: PostgreSQL sslmode=require y SQL Server encryption=require. PostgreSQL permite verify-ca/verify-full con CA configurada. Requerir cifrado no equivale a verificar identidad del servidor; la confianza de certificados debe administrarse en el driver. Los modos disable/off se reservan para pruebas locales. FreeTDS comparte timeouts a nivel proceso; el adaptador SQL Server serializa conexiones para evitar carreras.

RBAC separa connections:read, connections:use y connections:manage. Administradores y responsables de datos administran; Analista puede usar. Los roles de consulta solo ven metadata. Todas las rutas verifican organización. La auditoría registra creación, edición, prueba, preview, baja y adquisición con actor estable; nunca password, DSN ni filas de preview. Los errores de drivers se convierten a códigos controlados y mensajes constantes.

Fuentes futuras y frontera de salida

Un nuevo adaptador implementa DataSource y su normalización, registra opciones permitidas y aporta pruebas de permisos, límites, tipos y trazabilidad. Fuentes no SQL tendrán DTOs específicos, sin forzar el concepto de schema/tabla sobre buckets o APIs. El motor de calidad no cambia. Data Delivery implementa su puerto DataSink independiente; DataSource continúa sin incorporar write ni entrega hacia sistemas externos.

5B. Data Delivery

IMPLEMENTADO. Data Delivery publica una DatasetVersion canónica e inmutable en PostgreSQL o SQL Server. La versión 0.5.1 endurece la evidencia, la interpretación operativa de UNKNOWN y las métricas. No incorpora transformación funcional, conectores nuevos ni una transacción distribuida. La automatización vigente 0.7.0 se describe en el capítulo 21.

Adaptación vigente 0.7.0 a volumen

El preflight conserva los mismos códigos y condiciones funcionales de bloqueo, con mensajes centralizados distintos para éxito/fallo. PERMISSIONS no atribuye un privilegio faltante cuando el adaptador sólo ofrece un booleano general. Ser Administrator de Trackvance no concede privilegios SQL. Las comprobaciones conservan referencia segura y jamás incorporan DSN, excepción del driver ni filas. La corrección de mensajes fue implementada y probada en un commit separado antes de adaptar la preparación. En los apartados siguientes las estrategias/auditoría continúan vigentes; los antiguos límites síncronos se sustituyen por los controles de esta sección cuando se utiliza el recorrido asíncrono.



Preflight persistido y reutilización segura

POST /delivery/validations crea un Run DELIVERY_PREFLIGHT con Job lane DELIVERY y configuración privada VALIDATION_PRIVATE. Devuelve 202 durable; no crea un DeliveryAttempt ni ejecuta DDL/DML. Su historial es personal/organización incluso para Administrator. Puede seguirse después de navegar, cancelarse y abrirse desde un aviso. El resultado separa SUCCESS técnico del PASS/FAIL de comprobaciones.

El worker recorre toda la fuente verificada por DatasetRecords. El preview de ocho filas por defecto y hasta veinte no certifica tipos, longitudes, nullability ni claves del resto. Escanea partes con DuckDB y memoria limitada, sin lista total de filas. Tipos/precisión Decimal/Unicode se validan en todos los registros. La unicidad UPSERT usa un índice SQLite global en disco para descubrir duplicados incluso entre partes/lotes; las claves Decimal se comparan con precisión completa y las fechas/zona con la semántica vigente. La comprobación nativa posterior usa tipos/collation del target.

Una validación sólo publica una configuración grande si fue personal, SUCCESS+PASS y coincide exactamente el draft_hash, DatasetVersion y SHA canónico. Cambiar destino, revisión, mapping, target, estrategia, keys o versión invalida la reutilización. El endpoint síncrono permanece compatible para hasta 100.000 filas efectivas; por encima devuelve PREFLIGHT_ASYNC_REQUIRED y remite al job persistido. Antes de ejecutar se repiten permisos/destino/drift. El PASS previo no congela el servidor remoto ni la autorización actual de la cuenta.

PreparedDelivery en disco y frontera STARTED

DataSink.prepare_batched convierte lotes acotados y escribe PreparedRows en SQLite privado, con escalares tipados JSON, hash por fila, conteo, hash de archivo y binding. No usa float para Decimal ni pickle. La preparación se vincula a Run, artefacto/schema fuente, hash de configuración, revisión/hash del destino, columnas tipadas, target, estrategia, claves y conteo. verify comprueba ese binding completo y los hashes antes de STARTED. Un payload de otra configuración, un archivo alterado o un límite local produce FAILED_PRECONDITION, no UNKNOWN.

El control cooperativo verifica cancelación, lease, presupuesto temporal y reserva de disco durante scans/preparación. Revalida el usuario real y obtiene target guard en la transacción fenced anterior a STARTED. La preparación sellada se elimina al terminar la llamada de DataSink, también ante UNKNOWN. El owner antiguo no puede registrar otro resultado. La preparación no es una autorización para replay: la semántica del intento remoto sigue siendo la autoridad.

Lotes y una transacción remota

Los inserts usan executemany de lotes acotados, agregando sólo conteos fiables del adaptador. CREATE_AND_LOAD, APPEND, OVERWRITE y UPSERT conservan un único commit final y rollback conocido. OVERWRITE conserva la tabla y su población anterior si la transacción falla; no hace drop/recreate. Los staging de claves UPSERT cubren toda la fuente y usan igualdad nativa bajo tipos/collation/constraints del destino. No se admite IGNORE_DUP_KEY ni una restricción incompatible para ocultar duplicados.

fechalngesta conserva la semántica temporal del intento y usuario el username interno capturado en la Run. Un disparador SYSTEM no sustituye esa identidad por una etiqueta Responsable. rows_prepared, rows_written y rows_inserted/updated se definen separadamente; N/D sigue siendo null, nunca cero estimado. PostgreSQL 16/17 conserva N/D donde no tiene descomposición fiable; PostgreSQL 18 usa OLD documentado, y SQL Server conserva la lógica transaccional sin MERGE.

COMMITTED+PENDING_REPAIR confirma la escritura y conserva un pendiente local. Reparar evidencia reconstruye receipt/manifest sin contactar DataSink. UNKNOWN requiere revisión externa; review no cambia la historia ni reenvía filas. En 0.7.0 la reanudación del target es otra decisión explícita, autorizada y auditada, asociada a revisión concluyente. Todos los UNKNOWN legacy reconocidos deben decidirse individualmente antes de permitir una nueva operación compatible.

A. Propósito y flujo funcional

El usuario escoge una versión concreta, un destino versionado, el target y el mapping. La vista previa explica la proyección; el preflight inspecciona el destino sin escribir. Publicar fija una configuración inmutable. Ejecutar crea una Run y un Job DELIVERY; el worker repite las comprobaciones antes de abrir la transacción. El resultado durable precede a receipt y manifest.



La fuente no se consulta otra vez durante la entrega: el mismo snapshot conserva sus valores aunque una tabla fuente o un archivo original cambien. Una versión aceptada de Intake también puede ser input; su linaje sigue apuntando al Intake que la produjo. La entrega no significa que las reglas de negocio del receptor hayan aprobado el contenido.

B. Identidad y versiones del destino

DeliveryDestination mantiene identidad estable, nombre, motor, estado enabled, revisión optimista y último test. DeliveryDestinationVersion conserva una revisión inmutable con configuración técnica y hash. Una configuración fija ambos IDs; cambiar la contraseña, host o nombre del destino no reescribe runs históricos.

La API recibe password pero no la devuelve ni expone secret_reference. La contraseña cifrada vive en delivery_credentials y la clave en delivery_keys, volúmenes diferentes de los secretos fuente. La referencia opaca participa en el hash interno, no aparece como credencial pública. Deshabilitar o eliminar lógicamente el destino impide nuevas entregas; no borra su historia.

C. Contrato DataSink y límites de responsabilidad

Operación	Responsabilidad
test / schemas / tables	Comprobar conectividad y descubrir objetos autorizados; no escribir negocio.
table_metadata / permissions	Obtener tipos, constraints, collation y capacidades necesarias.
prepare	Validar identificadores, convertir representación técnica del mismo tipo lógico y congelar PreparedDelivery localmente.
deliver_prepared	Ejecutar la estrategia y confirmar o clasificar el resultado de la transacción remota.
DeliveryResult	Métricas de la operación confirmada y referencia remota opcional; no inventario físico final.

DatasetSource sólo lee. StorageProvider sólo administra evidencia interna. DataSink no evalúa reglas Intake/Recon/Sentinel ni modifica el Parquet canónico. PreparedDelivery contiene schema, tabla, columnas, tuplas, estrategia, claves y bytes preparados. La preparación ocurre antes del marcador STARTED: un error local de tipos no debe convertirse en una supuesta transacción ambigua.

D. Mapping, selección y orden

El mapping declara source_name, target_name, target_type, ordinal y nullable; STRING añade length y DECIMAL admite precision/scale. Puede seleccionar, reordenar y renombrar. No puede inventar columnas, repetir nombres o interpretar STRING como número, fecha, timestamp o booleano.

Tipo lógico	Target nuevo PostgreSQL	Target nuevo SQL Server	Regla de conservación
STRING	text / varchar	nvarchar con collation suplementaria	Texto Unicode exacto; identificadores conservan ceros iniciales.
INT64	bigint	bigint	Entero dentro del rango; no redondeo.
DECIMAL	numeric(p,s)	decimal(p,s)	Valor exacto, escala y dígitos enteros suficientes.
DATE	date	date	Fecha ISO inequívoca; no zona horaria.
TIMESTAMP	timestamptz(6)	datetimeoffset(6)	Offset explícito, instante conservado, hasta microsegundos.
BOOLEAN	boolean	bit	Booleano lógico; no interpretación libre de texto.

En target existente se inspecciona el tipo real y su capacidad. DECIMAL no se escribe en real/float aproximado. Un MONEY fuera del rango remoto puede superar las comprobaciones locales y fallar en la transacción: rollback comprobado, no éxito parcial. Identificadores SQL se validan y citan por dialecto; no se admite SQL libre en nombres ni una plantilla de consulta escrita por el usuario.

E. Texto y temporales: ejemplos exactos

STRING acepta almacenamiento variable Unicode exacto: text/varchar PostgreSQL y nvarchar SQL Server. Se rechazan NUL, surrogates no emparejados, tipos fixed y familias no Unicode. La longitud se comprueba conservadoramente en unidades UTF-16; caracteres suplementarios exigen collation _SC/_UTF8 en SQL Server. Tablas nuevas usan Latin1_General_100_CI_AS_SC.

Fuente nativa	Lógico conservado	Ejemplo / consecuencia
PostgreSQL timestamptz(0..6)	TIMESTAMP	2026-09-20T11:30:00.123456-05:00 conserva el instante; la representación del offset puede normalizarse.

Fuente nativa	Lógico conservado	Ejemplo / consecuencia
PostgreSQL timestamp without time zone(0..6)	STRING	2026-09-20 11:30:00.123456 no adquiere UTC implícita.
SQL Server datetimeoffset(0..6)	TIMESTAMP	Zona explícita y fracción hasta seis cifras.
SQL Server datetimeoffset(7)	STRING	CONVERT estilo127 conserva el séptimo dígito y puede representar el instante normalizado a UTC, por ejemplo 2026-09-20T16:30:00.1234567Z.
SQL Server datetime2(0..7), datetime, smalldatetime	STRING	Sin zona; conservar texto nativo, no atribuir un instante.

La regresión 0.5.1 recorre adquisición, refresh, lectura histórica, Intake, Sentinel, Recon y Delivery en ambos motores. El objetivo no es cambiar esta política sino detectar pérdida de fracción, offset o coerción implícita. Si una versión conserva un temporal como STRING, Data Delivery sólo lo publica como STRING; convertirlo a TIMESTAMP requiere otra capacidad explícita, fuera de este endurecimiento.

F. Target nuevo o existente

EXISTING_TABLE exige schema y tabla existentes, columnas compatibles, permisos y constraints adecuados. CREATE_TABLE se combina exclusivamente con CREATE_AND_LOAD. create_schema=true significa crear un schema que todavía no existe; no autoriza adoptar uno aparecido concurrentemente.

El preflight no reserva el nombre. La transacción vuelve a comprobarlo: si otra sesión creó el schema o target entre validación y escritura, falla de forma controlada. Ni la carrera ni un reintento convierten CREATE_AND_LOAD en overwrite. No se ejecuta DROP TABLE para acomodar una estructura incompatible.

G. CREATE_AND_LOAD

Crea el target con la definición técnica validada y carga la población preparada en una transacción. Cuando se solicitó, incluye la creación del schema. Un error de DDL o una fila rechazada revierte el conjunto conforme a las garantías del motor; no se publica un receipt COMMITTED antes de confirmar commit.

Para cero filas puede crear una tabla vacía: filas preparadas y enviadas son cero. Esto no demuestra que otros volúmenes sean soportados. La metadata de la nueva tabla se gobierna por el mapping, no por una inferencia distinta dentro del sink.

Precondiciones y permisos: schema existente con USAGE/CREATE en PostgreSQL, o CREATE de base si se crea schema; SQL Server requiere CREATE TABLE y ALTER del schema existente, o CREATE SCHEMA si es nuevo. El target debe estar ausente. La operación usa los locks DDL del motor, no reserva previamente el nombre. TARGET_ALREADY_EXISTS o SCHEMA_NOT_FOUND abortan ante drift; fallos de permisos o constraints no se convierten en carga parcial confirmada.

Idempotencia y métricas: reabrir la misma Run no recrea la tabla ni reenvía datos. Otra Run deliberada contra el mismo nombre falla por existencia. Tras COMMITTED, rows_written=N; rows_inserted usa el conteo conocido del driver o null y rows_updated=0. Confirmación de commit perdida produce UNKNOWN, no un replay DDL.

H. APPEND

Inserta todas las filas preparadas en un target existente. No elimina ni compara el contenido anterior. Dos runs deliberadamente diferentes pueden anexas las mismas filas: la idempotencia de Trackvance evita duplicar una misma solicitud identificada, no convierte APPEND en deduplicación remota.

rows_written describe la población fuente enviada en la operación confirmada. Si había 100 filas y se enviaron 20, el contador no se convierte en 120. Un trigger puede además suprimir, transformar o generar filas; ese efecto no se deduce del contador.

Precondiciones y permisos: estructura compatible y columnas requeridas satisfechas; PostgreSQL exige INSERT, SQL Server INSERT+SELECT. PostgreSQL toma ROW EXCLUSIVE para estabilizar el target; SQL Server TABLOCKX/HOLDLOCK hasta fin de transacción. El lock SQL Server puede bloquear otros escritores/lectores y no implica un rendimiento ilimitado. Constraints, timeout o target ausente producen error saneado y rollback conocido cuando el motor lo confirma.

Resultado y métricas: no hay actualizaciones explícitas (rows_updated=0); inserciones sólo se reportan si el driver conoce su conteo. La población previa no se suma. La pérdida de confirmación conserva UNKNOWN y exige verificar el destino antes de otra Run, pues volver a anexas puede duplicar filas.

I. OVERWRITE

Ejecuta DELETE e inserción en una misma transacción; conserva la tabla y no hace drop/recreate. Sustituye toda la población autorizada del target. Si falla la carga, un rollback conocido debe conservar la población anterior.

Se rechazan RLS activa en PostgreSQL y FILTER security policies en SQL Server, porque podrían dejar filas no visibles al usuario y falsear la idea de reemplazo. Permisos INSERT y DELETE son necesarios; SQL Server requiere también SELECT y VIEW DEFINITION para las verificaciones. Un input vacío borra la población existente dentro de la transacción: debe ser una elección deliberada.

Locks: SHARE ROW EXCLUSIVE en PostgreSQL; TABLOCKX/HOLDLOCK en SQL Server. La validación de políticas se repite bajo lock. UNSAFE_ROW_SECURITY o falta de visibilidad completa impiden la operación; no se considera válido borrar sólo las filas que una policy deja ver. IGNORE_DUP_KEY en SQL Server también se rechaza. Puede haber contención, crecimiento de log y fallos de constraints; la estrategia no deshabilita triggers ni integridad referencial.

Métricas e idempotencia: rows_written cuenta N filas enviadas, no filas borradas; rows_inserted depende del driver y rows_updated=0. Otra Run es otra sustitución, no una continuación automática. Si no se conoce el resultado del commit, la revisión externa precede a cualquier nueva sustitución deliberada.

J. UPSERT y claves compuestas

Exige claves explícitas, incluidas en el mapping, sin null ni duplicados en la fuente. El destino debe respaldarlas con PK o UNIQUE compatible. Una clave compuesta es una tupla ordenada con comparación nativa; no se concatena texto con un separador que pueda provocar colisiones.

El staging de claves usa tipos y collation del target para descubrir colisiones que la igualdad de Python no detectaría. No se acepta una restricción distinta porque parezca equivalente después de un cambio concurrente. Un input vacío sigue validando y bloqueando el target; cero filas no desactiva la seguridad.

Permisos: INSERT+UPDATE+SELECT en ambos motores, más TEMPORARY de base en PostgreSQL y acceso a #temp conforme a tempdb en SQL Server. PostgreSQL toma ROW EXCLUSIVE, revalida el árbitro nombrado y ejecuta ON CONFLICT; SQL Server conserva TABLOCKX/HOLDLOCK y UPDLOCK/HOLDLOCK para la búsqueda/actualización, sin MERGE. TARGET_KEY_NOT_UNIQUE y drift de constraint impiden confirmar una coincidencia ambigua; los errores de constraints y duplicados revierten el lote.

Una repetición como Run nueva puede modificar otra vez valores o disparar triggers: UPSERT no garantiza ausencia universal de efectos secundarios. rows_written=N; insertadas/actualizadas siguen las capacidades descritas en K/L, no se infieren de una consulta previa. Vacío informa cero acciones conocidas.

K. PostgreSQL: transacción y conteos fiables

El adaptador conserva el lock del target y revalida nombre/columnas de la constraint después de adquirirlo. UPSERT exige una PK/UNIQUE nombrada no diferible y usa ON CONFLICT ON CONSTRAINT; no sustituye la operación atómica por un SELECT seguido de UPDATE/INSERT susceptible a carreras.

En PostgreSQL 18 la cláusula RETURNING documentada permite observar OLD/NEW. La detección de inserción usa la ausencia del registro OLD completo, no sólo una clave nullable; un registro existente cuyos campos sean null sigue siendo un registro existente. Los conteos son acciones DML de nivel superior reportadas por el motor. BEFORE triggers que suprimen filas no se cuentan como inserciones físicas exitosas; los efectos secundarios de triggers no son un censo final.

En PostgreSQL 16/17 no hay una descomposición fiable equivalente para UPSERT no vacío con columnas actualizables: rows_inserted y rows_updated quedan null/N/D. La excepción es el mapping formado sólo por claves, que ejecuta DO NOTHING ante conflictos: las inserciones usan el affected-row count conocido y las actualizaciones son cero. Una operación vacía también puede afirmar cero porque no hubo acciones. No se usa xmax, estadísticas aproximadas ni un SELECT previo para fabricar precisión.

Referencias primarias: <https://www.postgresql.org/docs/18/dml-returning.html> y <https://www.postgresql.org/docs/18/sql-insert.html>. La base metadata y el destino baseline siguen en PostgreSQL 16; el runner añade un destino desechable 18, sin actualizar silenciosamente la instalación del usuario.

L. SQL Server: concurrencia y conteos

El adaptador evita MERGE. Usa transacción, staging #temp con tipos/collation del target, locks y comprobación por clave; SELECT @@ROWCOUNT valida que una acción no afecte más de una fila. La disponibilidad de tablas temporales depende de la política de tempdb. Un índice unique con IGNORE_DUP_KEY se rechaza para no silenciar duplicados.

Los conteos sólo se exponen cuando el adaptador puede sostenerlos. Si el driver no proporciona un affected-row count conocido, se utiliza null, no el tamaño de la fuente ni un cero ficticio. Los efectos secundarios de triggers no se deducen de esos contadores. La semántica de rows_written continúa siendo filas fuente enviadas en la operación confirmada, no la suma de todas las filas tocadas por triggers.

M. Familias de preflight y fallos

Familia	Comprobación / riesgo prevenido
Fuente e integridad	DatasetVersion propia, artifact canónico materializable, tamaño y SHA-256; no entregar bytes sustituidos.
Destino	Revisión exacta, hash, enabled y conexión; no usar otro endpoint por una edición posterior.
Mapping y valores	Columnas, preservación de tipo, identificadores, nullability, longitudes, precisión y escala.
Target	Existencia/ausencia esperada, metadata, columnas generadas/identity/default y compatibilidad real.
Permisos	Privilegios específicos de estrategia, schema, tabla y temporales.
UPSERT	PK/UNIQUE compatible; claves fuente no nulas, no duplicadas y collation nativa.
Recursos locales	Conteos/schema completos, máximo Delivery efectivo, memoria de scan, tamaño de lote, reserva de disco, timeout y autorización/lease. Las cotas se verifican durante preparación.

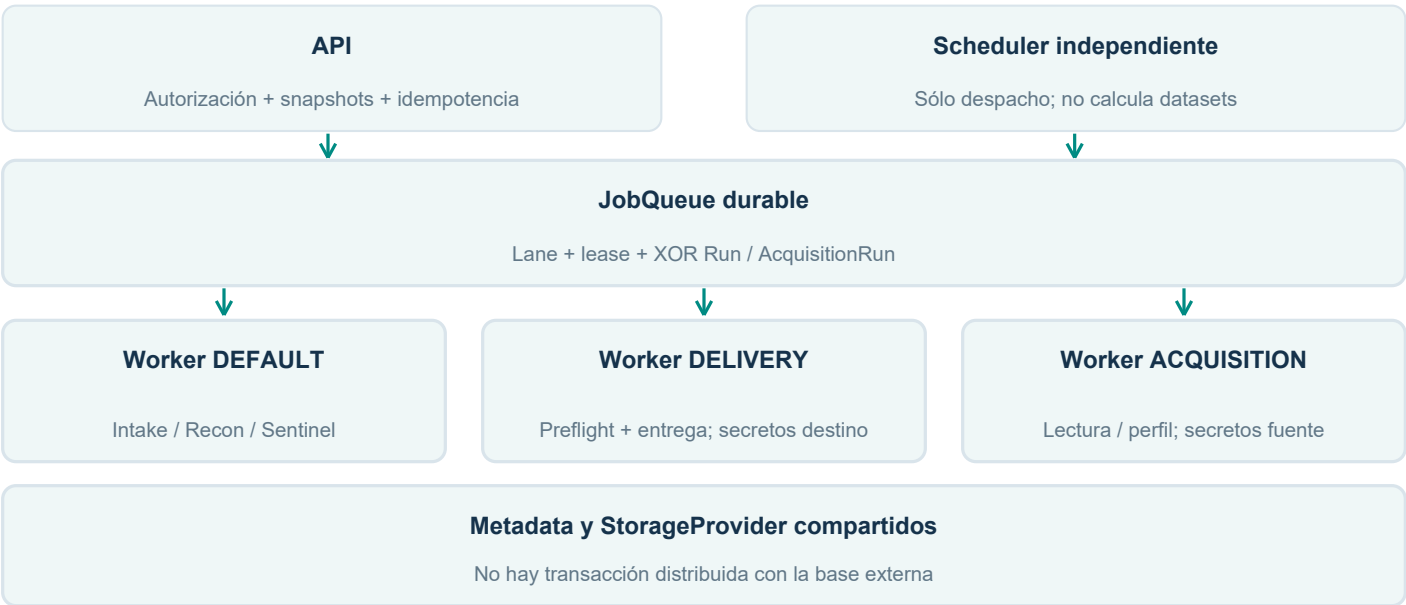
POST /preflight realiza lectura técnica; el resultado no garantiza que nada cambie después. El worker lo repite, y los guards transaccionales son la última autoridad ante drift. FAILED_PRECONDITION es un estado del Run anterior al intento remoto: no debe confundirse con DeliveryAttempt.FAILED después de STARTED.

N. Preview, configuración y publicación

POST /preview muestra una muestra acotada antes/después del mapping, orden y nombres. No envía datos al destino ni crea un intento. No certifica que todas las filas satisfagan constraints externas. La publicación de configuración valida el draft y fija schema_version=1, DatasetVersion, DestinationVersion, target, columns, write_strategy y upsert_keys.

Una nueva revisión es otra configuración inmutable. La UI no puede cambiar target_type para forzar una conversión lógica prohibida. El propietario visible es una etiqueta operativa y no sustituye permisos o identidad de usuario. config_hash y stored_config_hash permiten verificar lo publicado.

O. Workers, lanes y frontera de commit



DEFAULT procesa Intake/Recon/Sentinel. DELIVERY procesa preflights persistentes y entregas; ACQUISITION procesa AcquisitionRun. El scheduler independiente despacha horarios y no calcula datasets ni espera al worker. Las tres lanes comparten metadata y StorageProvider; el delivery-worker sólo monta secretos de destino y no secretos fuente. El Job incluye lane y una reclamación condicionada al lease, con XOR entre Run y AcquisitionRun.

La adquisición/reconciliación conserva el orden Run -> Job. Antes de STARTED se comprueba de nuevo que el worker conserva la reclamación. Una cancelación previa impide iniciar la escritura; un COMMITTED o FAILED ya conocido prevalece sobre una cancelación tardía. PostgreSQL interno y el motor receptor no participan en 2PC: perder la confirmación exige expresar incertidumbre, no repetir.

P. Estados del intento y del Run



Hecho verificable	Attempt	Run / decisión
Preflight o preparación local fallidos	No se inicia intento	FAILED_PRECONDITION / FAILED
Marcador durable previo al remoto	STARTED	RUNNING
Commit remoto confirmado y persistido	COMMITTED	SUCCESS / COMMITTED
Rechazo o rollback conocido	FAILED	FAILED / FAILED

Hecho verificable	Attempt	Run / decisión
Confirmación perdida o STARTED recuperado sin prueba	UNKNOWN	UNKNOWN / UNKNOWN
Commit conocido; publicación local incompleta	COMMITTED	SUCCESS / COMMITTED + PENDING_REPAIR

UNKNOWN no es una forma de FAILED. El sistema no afirma rollback ni commit cuando no puede demostrarlos. La recuperación de un attempt ya UNKNOWN no completa ni corrige retrospectivamente sus campos históricos. La revisión operacional es un registro aparte.

Q. UNKNOWN: revisión operacional append-only

GET y POST /api/v1/delivery/runs/{run_id}/reviews consultan/anexan observaciones. El cuerpo exige delivery_attempt_id, outcome y note; verified_at es opcional, con zona, no futuro ni anterior al inicio del intento. Si se omite, el servidor registra el instante actual. La nota debe contener texto y tiene un máximo de 4.000 caracteres.

outcome	Interpretación, nunca nuevo estado del intento
REMOTE_COMMIT_OBSERVED	El operador aporta una observación externa compatible con commit.
REMOTE_NOT_COMMITTED_OBSERVED	El operador aporta una observación externa de ausencia de commit.
INCONCLUSIVE	La comprobación no permite decidir.

DeliveryOperationalReview conserva org, Run, attempt, reviewer_id, snapshot de nombre, outcome, nota, verified_at y created_at. La migración 0009_delivery_reviews, introducida en 0.5.1, creó sólo esta tabla con FKs, índices y CHECK de outcomes; permanece intacta en 0.7.0 junto con 0001..0012. No hay PATCH/DELETE de revisiones: una corrección se anexa como nueva observación. No existe aprobación de cuatro ojos ni validación remota automática de lo escrito por el operador.

La UI muestra historial y advertencia de incertidumbre. Guardar una revisión no cambia Run/attempt a COMMITTED o FAILED, no encola un Job, no crea otro Run y no llama al DataSink. Si luego procede una nueva entrega, debe crearse deliberadamente con una nueva decisión operativa; no hay botón de replay ciego.

R. COMMITTED + PENDING_REPAIR: sólo evidencia local

POST /api/v1/delivery/runs/{run_id}/repair-evidence reconstruye receipt/manifest cuando existe Run DELIVERY SUCCESS/COMMITTED y un único attempt COMMITTED coherente. No se admite reparar UNKNOWN, FAILED, otro módulo o una identidad ajena. El resultado es REPAIRED o ALREADY_VALID con los IDs de ambos artifacts.

La operación bloquea la Run, valida organización, configuración/hash, identidad del destino/version, source schema/hash, artifact canónico y métricas persistidas. Usa los datos ya confirmados; no solicita contraseña, no descifra secretos, no instancia DataSink, no abre conexión remota y no repite DDL/DML/commit.

Los IDs nuevos de evidencia se derivan de org/Run/kind. Se reutilizan bytes y enlaces válidos; las restricciones de linaje y la publicación sin reemplazo evitan duplicación. Ante publicación parcial puede completar lo ausente. Un archivo registrado ausente sólo se reconstruye si los bytes coinciden con su SHA histórico; un archivo corrupto o una discrepancia no se sobrescribe para ocultar el problema. El servicio falla cerrado y registra el fallo.

Los planes 0.5.1 fijan además source_identity y la versión del motor de evidencia. Planes 0.5.0 que no contienen ese bloque se verifican con los registros retenidos; no se inventa un hash independiente que nunca fue almacenado. Artefactos 0.5.0 válidos se conservan byte por byte. La UI ofrece “Reparar evidencia”, nunca “Reintentar entrega”, y mantiene visible el commit confirmado.

Existe una ventana normal entre el commit durable y la publicación local. Si SUCCESS/COMMITTED todavía no incluye receipt_artifact_id ni PENDING_REPAIR, la UI muestra “Publicando evidencia local”, deshabilita descargas y consulta el Run cada 1,5 segundos durante hasta 60 segundos. Una respuesta lenta no se cancela para iniciar otra. Al vencer ese plazo ofrece Actualizar estado, sólo lectura; no marca la entrega como fallida ni ejecuta reparación o replay automáticamente. Si llega PENDING_REPAIR, pasa a la acción explícita de reparación con su permiso.

S. Idempotencia y concurrencia

Idempotency-Key identifica una solicitud de ejecución con el mismo payload y scope; repetirla devuelve la misma Run, no otra transacción. No es una clave de deduplicación de negocio remota ni autoriza repetir UNKNOWN. Configuración, DatasetVersion y destino versionado forman el contexto verificable.

La reparación es idempotente respecto a artifacts, enlaces y resultado; cada petición puede añadir auditoría de la acción, que no debe confundirse con otra entrega. La serialización local, los IDs estables y los guards de publicación protegen incluso una interrupción entre bytes y metadata. La revisión UNKNOWN, en cambio, es un historial de observaciones: cada POST válido añade un registro.

T. Métricas: significado y límites

Campo	Significado 0.5.1
rows_attempted	Filas fuente preparadas para el intento.
rows_written	Filas fuente enviadas sin error en una operación cuyo commit fue confirmado. No es COUNT(*) remoto.
rows_inserted	Acciones INSERT fiables reportadas por el adaptador; null cuando no puede garantizarlas.
rows_updated	Acciones UPDATE fiables reportadas por el adaptador; null cuando no puede garantizarlas.
bytes_sent	Tamaño UTF-8 preparado de valores no null; no bytes de red/TLS ni tamaño final de la base.
preflight_seconds	Duración monótonica del preflight del worker en runs nuevos.
write_seconds	Duración monótonica de deliver_prepared, incluida confirmación remota, excluida publicación local.

null se muestra como N/D. Cero es un conteo conocido, nunca una sustitución cosmética de null. Los nuevos DTOs/evidencias añaden metric_semantics versión 1; la lectura de manifests 0.5.0 sin el bloque sigue siendo válida y no reescribe bytes. Los timings históricos ausentes no se rellenan con cero.

U. Receipt y manifest: ejemplo abreviado

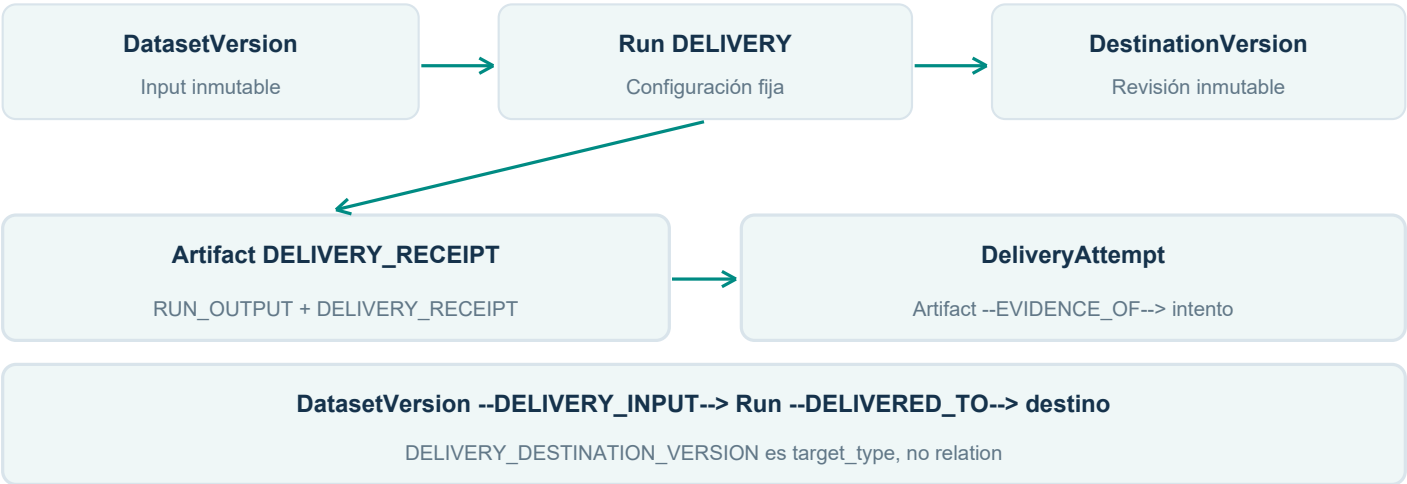
Ejemplo de forma, no resultado de una certificación. IDs y hashes son marcadores; los conteos elegidos ilustran UPSERT PostgreSQL 16 con desglose no disponible.

```
{
  "schema_version": 1,
  "kind": "DELIVERY_RECEIPT",
  "run_id": "<run>",
  "dataset_version_id": "<snapshot>",
  "canonical_artifact_id": "<artifact>",
  "canonical_sha256": "<sha256>",
  "destination_id": "<destino>",
  "destination_version_id": "<revision>",
  "sink_type": "POSTGRESQL",
  "target": {
    "mode": "EXISTING_TABLE",
    "schema_name": "public",
    "table_name": "records",
    "create_schema": false
  },
  "write_strategy": "UPSERT",
  "delivery_attempt_id": "<attempt>",
  "attempt_number": 1,
  "rows_attempted": 3,
  "rows_written": 3,
```

```
"rows_inserted": null,
"rows_updated": null,
"result": "COMMITTED"
}
```

El receipt completo añade fuente, destino/config_hash, nombres/versiones, fechas, bytes y metric_semantics. El manifest schema 2 incluye inputs, configuration, processing, actor, engine_version, metrics y result_artifacts. Su sección delivery incluye destination, target, write_strategy, metrics y attempt. Repara con los valores persistidos del commit, no con un nuevo timestamp de ejecución.

V. Linaje persistido y auditoría



source_type -> target_type	relation real
DATASET_VERSION -> RUN	DELIVERY_INPUT
RUN -> DELIVERY_DESTINATION_VERSION	DELIVERED_TO
RUN -> ARTIFACT (receipt)	DELIVERY_RECEIPT
ARTIFACT (receipt) -> DELIVERY_ATTEMPT	EVIDENCE_OF
RUN -> ARTIFACT (receipt o manifest)	RUN_OUTPUT

DELIVERY_DESTINATION_VERSION es un tipo de entidad, no la relación. Esta corrección documental no migra enlaces históricos ni cambia sus direcciones. Los tests contrastan los triples persistidos y la reconstrucción de evidencia. El backfill legacy de arranque no añade RUN_INPUT genérico a Runs Delivery ni DELIVERY_PREFLIGHT; preserva los vínculos históricos que ya existan y no reemplaza la reparación explícita. El restore compara el grafo completo, sin omitir enlaces para aprobar.

Auditoría conserva DESTINATION_CREATED/TESTED, DELIVERY_CONFIGURATION_PUBLISHED, DELIVERY_RUN_QUEUED, DELIVERY_STARTED/COMMITTED/FAILED/UNKNOWN y, en 0.5.1, DELIVERY_EVIDENCE_REPAIR_STARTED, DELIVERY_EVIDENCE_REPAIRED, DELIVERY_EVIDENCE_REPAIR_FAILED y DELIVERY_UNKNOWN_REVIEWED. No se incluye la nota libre de revisión en metadata de audit, aunque permanece accesible en su recurso autorizado.

W. Permisos, secretos y aislamiento

Delivery usa delivery:read/configure/execute/overwrite/alter_target/review_unknown/repair_evidence; los destinos usan destinations:read/use/manage. Receipt exige artifacts:download y delivery:read. Configuración y ejecución validan dependencias y el recurso real. La autoridad está en el backend y el rol persistido; la UI refleja los grants de /me. Los capítulos 25 y 30 detallan protección de Administrator, permisos no delegables, CSRF, scope, UNKNOWN, DDL y auditoría. Las revisiones conservan actor estable y no permiten cruzar attempt/Run/organización.

X. Backup, restauración y compatibilidad

El backup conserva PostgreSQL, artifacts, secretos fuente/destino y ambas claves. Manifest mantiene schema 2; state 6 incorpora las 42 tablas y conjuntos multipartes. State 5/31 tablas de 0.6.1 es antecedente y se conserva su proyección estricta documentada. El restore nativo exige igualdad exacta de filas, hashes, relaciones y secretos. Los secretos OIDC sólo utilizan variables externas: .env y client secrets no forman parte del backup.

Backups 0.5.1/0009/state4, 0.5.0/0008/state3 y 0.4.x/0007/state2 se actualizan a 0015 y comparan su proyección legacy-v4/v3/v2 más adiciones permitidas 0.7.0. La proyección valida primero integridad y migración de identidad, admite roles/grants sembrados y elimina sólo adiciones esperadas. Rechaza actividad nueva en tablas que debían estar vacías; no oculta filas, errores de referencias ni alteraciones de campos históricos.

El drill 0.5.1 preservó nueve artifacts, dos secretos, 164 relaciones, dos intentos y una revisión. Antes del backup, dos peticiones de reparación concurrentes devolvieron REPAIRED/ALREADY_VALID con los mismos IDs y bytes. Después del restore, otra reparación devolvió ALREADY_VALID; el target original conservó cuatro filas y no se repitió la escritura. Se destruyó el origen desechable antes de restaurar en un proyecto nuevo. La revisión UNKNOWN fue un fixture explícito
SIMULATED_UNKNOWN_NO_REMOTE_IO: no certifica una pérdida real exacta de confirmación post-commit; ese escenario se reporta separadamente, sin convertir NOT_RUN en PASS.

Y. Benchmark dedicado y operación en UI

scripts/tests/delivery_benchmark_cycle.py levanta destinos PostgreSQL y SQL Server reales y desechables, usa contenido SHAKE256 determinista variado y recorre CREATE_AND_LOAD, APPEND, OVERWRITE y UPSERT. Smoke: 1 MiB y 1.000 filas. Representativo acotado: 8 MiB y 20.000 filas. Ninguno certifica capacidad de producción. En ese ensayo histórico, los límites de 10 MiB/100.000 filas no aumentaron; las cotas asíncronas vigentes están descritas al inicio de este capítulo.

Registra bytes/filas/columnas/hash de origen; tiempo total, preflight y escritura; filas/s y MB/s; CPU, memoria, I/O y crecimiento de storage/temporales; outcome, conteos y motivo real de parada. Los picos muestreados pueden omitir ráfagas breves y los contadores cgroup incluyen actividad incidental del contenedor. Recurso insuficiente produce NOT_RUN_RESOURCE_LIMIT, nunca PASS. Los resultados medidos se publican en el informe específico y JSON sanitizados 0.5.1.

Resultados locales del 25 de septiembre de 2026

El smoke real de 1.074.923 bytes / 1.000 filas aprobó las ocho combinaciones en 143,59 s de wall time. El representativo de 8.917.809 bytes / 20.000 filas también aprobó ocho de ocho en 207,70 s. Ambos tienen cuatro columnas y payload distinto por fila; el representativo usa celdas de 420 caracteres, frente a 1.049 en smoke. No son repeticiones estadísticas ni una progresión a ancho de fila constante.

Motor / estrategia	Escritura smoke s	Escritura 20.000 filas s	Filas/s representativo
PostgreSQL 16 / CREATE_AND_LOAD	0,0255	0,2082	96.071
PostgreSQL 16 / APPEND	0,0168	0,1945	102.807
PostgreSQL 16 / OVERWRITE	0,0247	0,2191	91.273
PostgreSQL 16 / UPSERT	0,0392	0,6329	31.599
SQL Server 2022 / CREATE_AND_LOAD	0,4034	7,1269	2.806
SQL Server 2022 / APPEND	0,4097	7,1833	2.784
SQL Server 2022 / OVERWRITE	0,4727	7,2688	2.751
SQL Server 2022 / UPSERT	1,7187	31,7338	630

Escritura incluye conexión, locks/checks, DML y commit dentro de deliver_prepared; no incluye el preflight anterior, cola ni publicación local. El total de caso sí incluye esos pasos y polling, pero excluye arranque, upload y SQL independiente de verificación. El informe dedicado contiene ambos preflights, total, MB/s, CPU/I/O por contenedor y método, sin mezclar estas ventanas.

En la carga representativa se prepararon/enviaron 20.000 filas por caso; UPSERT SQL Server reportó 10.000 insertadas y 10.000 actualizadas. PostgreSQL 16 devolvió null/null, aunque el SQL independiente verificó la mezcla esperada. APPEND dejó 40.000 filas físicas; las demás estrategias dejaron 20.000. Esto confirma por qué filas enviadas no significa tamaño final de la tabla.

El pico de memoria agregado muestreado fue 1.251.307.681 bytes en smoke y 2.050.125.461 en representativo; CPU agregada máxima 141,38 % y 213,85 % de un núcleo, respectivamente, sobre 16 CPU lógicas asignadas a Docker. Hubo 28 y 40 muestras, sin OOM ni corte del watchdog. El delivery-worker alcanzó 217.186.304 bytes en el representativo; storage de artifacts creció de 15.325.905 a 15.407.176 bytes. El temporal observado fue cero, sin garantizar ausencia de picos entre muestras. El cgroup memory.peak es acumulado, no exclusivo del caso.

En el ciclo histórico 0.5.1, los tiers de 100/500 MiB y 1/2/5 GiB quedaron NOT_RUN_RESOURCE_LIMIT por su límite de entrada de 10 MiB, no por una medición de incapacidad del hardware. Ese resultado no describe las cotas actuales 0.7.0. El primer smoke falló antes de escribir porque el mapping del fixture usaba INT64 donde la inferencia produjo DECIMAL; se conserva como FAIL y su repetición es un resultado separado. La corrección no alteró tipos ni límites del producto.

Evidencia: docs/development/delivery-benchmark-results-0.5.1.md y los directorios delivery-benchmark-smoke, delivery-benchmark-smoke-retry y delivery-benchmark-representative bajo docs/development/evidence/0.5.1.

Recorrido funcional de la interfaz

Pasos de UI: crear/probar destino; escoger versión; configurar target/mapping; revisar preview/preflight; publicar; ejecutar y abrir detalle. El detalle muestra intento, filas enviadas/N/D, receipt y manifest. Ante PENDING_REPAIR, comprobar el commit y pulsar Reparar evidencia. Ante UNKNOWN, verificar externamente, elegir outcome, explicar la observación y guardar; revisar el historial sin ejecutar una entrega adicional implícita.

Z. Límites y criterio de aceptación

No hay garantía exactly-once distribuida, replay automático de UNKNOWN, SQL arbitrario, transformación funcional en Delivery, nuevo conector, SHIST/SCD, HA ni certificación de volumen productivo. Los efectos secundarios del receptor quedan fuera del conteo físico de Trackvance.

La aceptación exige tests nuevos de reparación sin DataSink, historial UNKNOWN inmutable, semántica nullable, conteos reales por versión de motor, temporalidad sin coerción, linaje exacto, benchmark y code splitting. Además se repiten las suites base, migraciones, conexiones, Delivery, recovery, navegador y CI. El apartado de validación contiene resultados ejecutados; este diseño no sustituye esa evidencia.

6. Carga, esquema y versionado

El dataset es una identidad lógica con versiones inmutables. La adquisición registra una intención durable separada; no se incrementa version_count hasta publicación completa. Las rutas legacy de carga/refresh permanecen explícitas y conservan sus respuestas históricas 201 y límites pequeños. La UI normal ofrece el recorrido de adquisición 202, y la alternativa legacy queda identificada.

Una versión contiene id/dataset/version, tipo de origen, nombre, tamaños, SHA-256 del artefacto de identidad, schema_hash, columnas/perfil completos y metadata de lectura. Si existe un original conservado, sha256/size_bytes identifican ese original; si no, identifican el artefacto canónico, incluido el descriptor de un conjunto multipart. La metadata de ingestión conserva canonical_size_bytes/data_size_bytes y observados para explicar el tamaño de los datos y alimentar el planner. No se utiliza el tamaño de JSON del descriptor como coste físico del conjunto.

Los overrides no hacen cast silencioso de valores: conservan exactitud y dejan un esquema lógico explícito para reglas posteriores. Identificadores de texto, espacios y ceros iniciales siguen siendo EXACT_OBSERVED. El perfil tiene conteos globales, no suma de distintos independientes por lote. La muestra se marca como muestra, limitada, con referencia a versión/schema/perfil que la produjo.

Los IDs internos de registro se conservan al procesar, ordenar y distribuir. En CSV multilínea el origen conserva su numeración aplicable; en SQL es posición estable de la extracción materializada. Spark no asigna identidad de negocio según orden accidental de partición. Las salidas aceptadas Intake fijan padre y output_version_id de la Run concreta y conservan sólo filas aceptadas completas.

7. Profiling, identificadores y transforms

La política actual es EXACT_OBSERVED, metric_definition_version=2. No se recortan espacios, cambia case ni normaliza Unicode para contar valores distintos. "Cliente 12" y " Cliente 12 " son valores distintos; las diferencias de mayúsculas y de representación Unicode se conservan. Null y texto vacío tienen semánticas propias del lector y del perfil; no se equiparan mediante un trim implícito.

Inferencia inequívoca

DATE se infiere cuando los valores no nulos válidos cumplen YYYY-MM-DD y representan fechas reales. Una fecha inválida o una mezcla incompatible impide inferir DATE. TIMESTAMP exige su semántica ISO diferenciada, con zona explícita según las reglas portables; no se adivinan fechas ambiguas como 01/02/2026.

IDENTIFIER tiene prioridad sobre la interpretación como magnitud. Los nombres id y *_id activan una heurística documentada; el usuario puede confirmar/override el esquema. "001234567" conserva su cero inicial frente a "1234567". No se convierten universalmente todas las columnas numéricas a identificadores. El esquema nativo Parquet ayuda a inferir tipos, incluso con columnas nulas, pero no invalida el override explícito.

En la carga inicial y al crear una versión, el selector de identificadores muestra las columnas realmente inspeccionadas y permite elegir algunas o todas. La API continúa recibiendo los mismos overrides; no se crea una segunda fuente de nombres ni se reescriben los esquemas de versiones históricas.

Los perfiles incluyen conteos de nulos y distintos, tasas y razón de unicidad, además de método/versión. Las series históricas LEGACY_NORMALIZED/1 se mantienen separadas de EXACT_OBSERVED/2 para no comparar métricas semánticamente incompatibles.

La vista de perfil identifica su muestra con sampled_rows, sample_bytes, sample_limited y sample_byte_limit. Como máximo presenta veinte registros y 8 MiB; una muestra vacía por exceso de anchura no significa que el dataset esté vacío. Los lotes Arrow se acotan antes de convertir valores a Python, mientras que las estadísticas persistidas siguen describiendo la población completa.

Un valor Decimal nativo del Parquet se representa en la muestra HTTP como texto decimal exacto, por ejemplo "12345678901234567890.12345678", sin pasar por float. El presupuesto sample_bytes utiliza esa misma codificación JSON UTF-8; conserva comillas, escapes y separadores. Esta presentación no modifica archivos, esquemas, hashes ni estadísticas globales persistidas.

Transformaciones funcionales

Intake y ReconOps admiten listas ordenadas de transforms {column,type,parameters}: trim, case, unicode_normalization, empty_to_null, id_padding, remove_characters, decimal_parse y date_parse. Recon mantiene listas independientes para origen y destino. Sólo se ejecutan cuando el contrato las declara. El upload original, Parquet de entrada y perfil histórico no se reescriben; los resultados conservan los parámetros efectivos.

La interfaz presenta esos mismos tipos con nombres funcionales: Eliminar espacios externos, Convertir mayúsculas/minúsculas, Normalizar texto Unicode, Convertir texto vacío en nulo, Completar identificador, Eliminar caracteres, Interpretar número decimal e Interpretar fecha. Cada elemento muestra una explicación y un ejemplo recalculado con sus parámetros. Un preview de hasta ocho valores reales aplica las transformaciones de la columna en el mismo orden del contrato y muestra Antes / Después. Los controles permiten hacer explícito ese orden. Un parseo decimal o de fecha inválido muestra "No se pudo interpretar; la transformación conservó el valor que recibió". Los pasos posteriores siguen ejecutándose en el orden declarado. La vista previa de fechas cubre únicamente las directivas determinísticas que el navegador reproduce con certeza. Para formatos Python no soportados (por ejemplo %b, %j o %Z) muestra "Vista previa no disponible para este formato; la ejecución usará el formato declarado", sin convertir esa limitación en un fallo. datetime.strptime del backend conserva la autoridad sobre la ejecución. La vista previa se calcula sólo para informar: no crea artifacts, no modifica el DatasetVersion y no forma parte del resultado ni del hash de configuración.

Las configuraciones nuevas se validan y normalizan a schema_version=2. configuration_hash usa JSON determinista con claves ordenadas y SHA-256; stored_config_hash conserva además la identidad de la configuración almacenada. La adaptación legacy se centraliza en config_semantics y manifests, evitando condiciones de versión dispersas en módulos.

8. Reglas portables y Data Intake

RuleDefinition declara type, column o columnas en parameters, severity ERROR/WARNING, enabled, parameters, when opcional, code/message opcionales y rule_id estable. El código ausente se deriva de type. Publicar una regla nueva asigna rule_id; la lectura de configuraciones históricas nunca fabrica IDs ni altera sus fingerprints. La declaración se compila a una representación portable antes de evaluarse con el engine. No se admite scripting arbitrario ni Python eval/exec.

Tipo / código	Parámetros y semántica
required / REQUIRED	Campo presente, no null y no vacío. Whitespace no es vacío sin trim declarado.
not_null / NOT_NULL	Rechaza null.
unique / UNIQUE	Incumplen todos los registros duplicados; null_policy explícita.
compound_unique / COMPOUND_UNIQUE	Tupla de columns; todos los miembros de una combinación repetida incumplen, sin concatenar valores con separadores ambiguos.
numeric / NUMERIC	Decimal finito con punto; conserva precisión.
positive / POSITIVE	Decimal estrictamente mayor que cero.
type / TYPE	logical_type STRING, INT64, DECIMAL, DATE, TIMESTAMP o BOOLEAN.
range / RANGE	gt/gte y lt/lte, límites Decimal inequívocos.
length / LENGTH	min/max enteros inclusivos en puntos de código Unicode observados, sin trim ni conteo por bytes.
allowed_values / ALLOWED_VALUES	values con hasta 1000 escalares; pertenencia exacta.
regex / REGEX	Subconjunto portable Rust/RE2, flags i/m/s, máximo 500 caracteres.
date_rule / DATE_RULE	not_future, min/max ISO, timezone UTC y null_policy.
column_compare / COLUMN_COMPARE	Columna comparada contra other_column mediante eq/ne/gt/gte/lt/lte y logical_type STRING/DECIMAL/DATE/TIMESTAMP declarado.
reference / REFERENCE	Una o varias columns deben existir en reference_columns de una DatasetVersion explícita e inmutable.

null_policy admite ALLOW, FAIL o IGNORE según la regla. IGNORE excluye la fila de evaluated_count; no se cuenta como una evaluación aprobada. Los shorthand históricos numeric/positive conservan FAIL. El instante de referencia de fecha se fija al inicio del Run para que una misma ejecución no cambie de criterio entre registros.

Condiciones y referencias seguras

when acepta una condición tipada o un árbol all/any: hasta cuatro niveles, 64 nodos y diez condiciones por grupo. Los operadores son eq/ne/gt/gte/lt/lte/is_null/not_null. No hay código de usuario. Los literales se validan al publicar; una fecha o número mal formado no convierte silenciosamente una condición en falsa.

La condición filtra la población antes de evaluar la regla. En unicidad condicional se comparan únicamente las filas elegibles. Las excluidas por condición o IGNORE quedan en skipped_count. Ejemplo: country eq CO -> required department. El editor permite construir una condición simple con columnas reales; la API admite árboles acotados. El editor conserva una condición compuesta existente sin reinterpretarla.

REFERENCE fija dataset_version_id y correspondencia de columnas en el contrato. La API valida organización, existencia del snapshot y columnas; el worker materializa exclusivamente el Parquet verificado mediante StorageProvider. La comparación de tuplas conserva texto/null y no conecta desde el motor a una BD externa. El plan incluye el tamaño de las referencias; el manifest registra references y ArtifactLink registra RUN_REFERENCE. Un dataset de referencia vacío rechaza filas aplicables; null se gobierna por la política declarada.

Editor y resultados

Al elegir dataset, el editor carga su esquema. Columnas obligatorias, sin duplicados, numéricas y positivas usan multiselect con Todos; las positivas sólo permiten columnas numéricas. Las reglas avanzadas seleccionan columnas reales y muestran parámetros por tipo. Las referencias permiten elegir dataset, versión y columnas con su esquema persistido. El sistema detecta candidatos, pero el usuario decide las reglas. Actualizar esquema no obliga a escanear datasets completos. Responsable funciona como un catálogo deduplicado de owners de datasets y configuraciones del módulo; el usuario puede seleccionar uno o agregar una etiqueta nueva de hasta 120 caracteres. El contrato sigue persistiendo el mismo string owner: no es una cuenta, rol ni asignación de Excepciones.

Intake devuelve total_rows, valid_rows, error_rows, warning_rows, acceptance_rate y decisión APPROVED/APPROVED_WITH_WARNINGS/REJECTED según severidad y max_error_rate. Cada regla conserva rule_id cuando existe, code, columnas, severity, status, evaluated_count, failed_count, skipped_count, parámetros y condición. La evidencia identifica valor recibido y motivo. Una fila puede producir varios errores; error_rows cuenta registros afectados, no la suma de mensajes. WARNING conserva la fila aceptada y su advertencia. Una regla sin filas evaluadas no demuestra corrección técnica de una excepción.

El output reutilizable se guarda como Parquet INTAKE_ACCEPTED, con source_type=INTAKE_OUTPUT, canonical_artifact_id, parent_version_id y source_run_id. No tiene upload original ficticio. La UI muestra Artefacto derivado/Fuente de la versión y el linaje enlaza el input y el IntakeRun.

Caso operativo: aceptación y evidencia reutilizable

Para un archivo de pagos, required valida identificación, positive exige monto positivo y reference compara la cuenta con una versión del catálogo. El usuario declara cuando corresponda trim o decimal_parse; el motor nunca los añade por su cuenta. Se publica el contrato y se ejecuta sobre una versión concreta.

SUCCESS + REJECTED significa que la evaluación terminó y encontró incumplimientos; no debe repetirse como si hubiera ocurrido un error de infraestructura. La fila rechazada queda explicada por regla/valor/motivo. Las aceptadas generan una versión derivada trazable; su entrega posterior no necesita consultar de nuevo el archivo ni el catálogo remoto. Una regla WARNING no elimina la fila.

9. ReconOps y normalización declarada

ReconOps compara versiones inmutables de origen y destino. key_columns admite una o varias columnas. El control conserva key_normalization en su versión, config hash, manifest, plan y diagnostics.

El editor explica que las columnas clave identifican el mismo registro en ambos lados y que, al seleccionar varias, forman una clave compuesta cuyo orden se respeta. También aclara que la normalización se aplica antes de buscar coincidencias y no modifica los datasets originales. Las opciones visibles son “Conservar como están / Eliminar espacios externos”, “Conservar como están / Convertir a MAYÚSCULAS / Convertir a minúsculas” y “No normalizar / Normalización estándar (NFC) / Normalización de compatibilidad (NFKC)”. La ayuda Unicode explica la diferencia entre representación visual e interna.

```
{"trim": false, "case": "NONE", "unicode_normalization": "NONE"}
```

case admite NONE/UPPER/LOWER y unicode_normalization NONE/NFC/NFKC. Los controles nuevos usan NONE por defecto. Los históricos sin el campo conservan el comportamiento legacy de trim mediante adaptación versionada; no se cambia silenciosamente el resultado de runs anteriores.

Una vista previa Antes / Después utiliza hasta diez registros reales de origen y destino para mostrar la clave interna tras la normalización. En claves compuestas representa todas las columnas como conjunto ordenado y permite contrastar ambos lados. La muestra y el ejemplo dinámico se calculan en UI; no escriben datos ni alteran trim booleano, case (NONE, UPPER, LOWER) ni unicode_normalization (NONE, NFC, NFKC) persistidos.

Regla	Semántica
EXACT_COMPARE	Igualdad textual exacta tras la normalización declarada, con configuración independiente por par de columnas.
NUMERIC_TOLERANCE	Comparación Decimal con abs y percent opcional; denominador SOURCE/TARGET/MAX_ABS y política de denominador cero EXACT_ONLY. Umbral inclusivo.

Regla	Semántica
DATE_TOLERANCE	Diferencia temporal contra hours o days, con zona UTC y equal_nulls.
AGGREGATE_COMPARE	1:N agregando destino o N:1 agregando origen, con una o varias operaciones SUM/COUNT sobre el mismo lado. comparison_rules consumen las columnas resultantes.

EXACT_MATCH se conserva como alias explícito legacy de NUMERIC_TOLERANCE. Una tolerancia numérica cero sigue comparando magnitudes: "01" y "1" pueden coincidir. Esto es distinto de EXACT_COMPARE. amount_column/tolerance históricos se adaptan sin modificar el snapshot.

Políticas y orden de ejecución

Cada comparación admite null_policy MATCH_NULLS, MISMATCH o INVALID; el adaptador conserva equal_nulls de controles históricos. Con MATCH_NULLS dos null coinciden, uno solo produce diferencia; MISMATCH considera cualquier null una diferencia; INVALID clasifica el registro como inválido. Un valor numérico/temporal mal formado conserva INVALID y nunca se equipara a un null válido.

El orden determinista es transforms de origen/destino, normalización de claves, agrupación explícita y comparaciones. Se configuran las transformaciones por columna y lado; no existen conversiones implícitas adicionales. Los parámetros se persisten en control, config hash, diagnostics, manifest y Excel.

Las agregaciones nuevas sólo conservan claves y outputs definidos. Una comparación no puede tomar arbitrariamente el primer valor no agregado de un grupo. SUM inválido se propaga a la comparación que consume ese output; COUNT puede evaluarse independientemente. Los controles históricos que usaban aggregation singular se leen con su comportamiento anterior; una nueva publicación debe satisfacer las validaciones actuales. No se admite N:M ni fuzzy matching.

Clasificaciones estables

MATCH, VALUE_MISMATCH, SOURCE_ONLY, TARGET_ONLY, DUPLICATE_SOURCE, DUPLICATE_TARGET e INVALID. Las filas duplicadas no se convierten en coincidencias arbitrarias. Las comparaciones conservan valores, regla, diferencia, tolerancia y líneas origen/destino; agregaciones conservan las líneas de sus integrantes.

metrics incluye source_rows/target_rows, matched, mismatched, faltantes, duplicados, invalid, total_rows, match_rate, counts por clasificación y evaluadas/fallidas/inválidas por comparación. La decisión es CONFORME o WITH_FINDINGS. Los hallazgos agrupan categorías distintas de MATCH; UI y Excel permiten revisar el detalle verificable de cada comparación y el linaje de ambos inputs.

La precisión monetaria se implementa con Decimal y contexto suficiente para sumas, diferencias y porcentajes. Las expresiones tienen pruebas de paridad Polars/DuckDB donde se declara soporte. No se infiere equivalencia de todo el pipeline entre motores a partir de estas pruebas.

Caso operativo: claves, tolerancia y diferencias

Un control de conciliación puede fijar (cuenta, fecha) como clave compuesta y comparar importe con tolerancia Decimal. Antes de ejecutar se eligen los dos snapshots y se revisan transforms y null_policy. Una clave duplicada en origen no se empareja arbitrariamente con la primera fila destino; conserva la clasificación de duplicidad y su evidencia.

El resultado puede ser SUCCESS + WITH_FINDINGS. MATCH no implica igualdad de texto cuando se eligió tolerancia numérica; EXACT_COMPARE sí conserva la representación después de las transforms declaradas. Para temporales STRING, una prueba de igualdad no los convierte en instantes; DATE_TOLERANCE requiere su propio contrato temporal compatible.

10. Sentinel y métricas históricas

Sentinel ejecuta monitores sobre una DatasetVersion y produce checks explicables. Los controles legacy SCHEMA_REQUIRED, NULL_RATE, FRESHNESS y VOLUME_CHANGE permanecen compatibles. Una columna ausente se distingue de una columna presente con nulls.

En el editor, “Columnas requeridas” y “Columnas a revisar por nulos” son selectores múltiples alimentados por el esquema de la DatasetVersion elegida. Pueden conservar selecciones históricas mientras se publica una versión nueva; backend sigue validando que los nombres y la organización sean correctos.

Control	Parámetros / cálculo
SCHEMA_REQUIRED	Lista de columnas obligatorias del monitor.
NULL_RATE	Columnas observadas y máximo permitido.
FRESHNESS	Edad respecto a fecha de versión y max_age_hours.
VOLUME_CHANGE	Cambio porcentual respecto a versión anterior.
DISTINCT_COUNT	Columna y min/max de valores distintos.
DISTINCT_RATE	Columna y límites en fracción 0..1.
UNIQUENESS_RATIO	Proporción de registros únicos, con límites 0..1.
SCHEMA_TYPE	Tipo esperado o comparación con el esquema de la versión anterior.
METRIC_THRESHOLD	Métrica/columna, operador y threshold o min/max.
HISTORICAL_BAND	window, min_history, iqr_multiplier y fallback explícito.

Las reglas por registro de Intake también pueden evaluarse como checks de Sentinel con conteos evaluados/fallidos/excluidos. El resultado indica code, name, status, actual, expected, severity, rule_id cuando existe y message. La severidad declarada llega a Findings y Excel. La salud es la fracción de checks aprobados; cero fallos da HEALTHY y cualquier fallo da ALERT, incluso cuando su severidad es WARNING.

Compatibilidad de series

SentinelMetricHistory conserva run_id, monitor_id, metric_key, dimensiones y hash, numeric_value, method, metric_definition_version y observed_at. Sólo se comparan ejecuciones SUCCESS con método/versión compatibles. El historial no mezcla perfiles legacy normalizados con observaciones nuevas exactas.

La banda histórica utiliza mediana e IQR con el método MEDIAN_IQR_LINEAR_V1. Cuando no hay historia suficiente, el resultado identifica FIXED_THRESHOLD_FALLBACK o PREVIOUS_COMPATIBLE_METRIC según la política. El expected muestra qué método se aplicó; no inventa una referencia estadística.

El monitor puede detectar schema drift sin que el worker falle técnicamente. El Run original conserva ALERT aunque una versión posterior vuelva a estar saludable. Las excepciones asociadas sólo se validan cuando una ejecución posterior del mismo snapshot de monitor termina HEALTHY.

Programación local e historia visible

El proceso scheduler independiente consulta monitor_schedules y registra Jobs; el worker DEFAULT consume la cola de calidad. La programación referencia una Configuration inmutable. Una edición crea MonitorScheduleVersion con intervalo, activación, inicio y actor; no mueve programaciones a una nueva versión del monitor automáticamente. Se admiten intervalos de un minuto a 31 días. La UI recibe hora local y la convierte a UTC; la API exige zona explícita.

En cada despacho se fija la última DatasetVersion registrada. No se consulta ni actualiza automáticamente la fuente externa; el refresh de una conexión sigue siendo adquisición separada. MonitorOccurrence conserva configuración, revisión de horario, DatasetVersion, fecha prevista, despacho y Run. El Run aporta inicio real, finalización, estado, métricas y evidencia. La metadata schedule se incluye en execution_plan y manifest.

COALESCE_LATEST agrupa intervalos vencidos tras una pausa; SKIP_WHILE_ACTIVE evita solapar runs del mismo monitor. NO_DATASET_VERSION documenta la ausencia de entrada. Cursor, ocurrencia, Run, Job y auditoría se confirman en una sola transacción con compare-and-swap y unicidad por fecha prevista. El actor es SYSTEM / trackvance:local-scheduler.

Mientras el scheduler está detenido no hay nuevos ticks; mientras DEFAULT está detenido no se ejecutan los Jobs de calidad. Un Job largo puede retrasar el inicio de la siguiente Run, pero no detiene los ticks independientes. La diferencia entre fecha prevista e inicio real queda visible. Las series agrupan muestras por métrica, dimensiones, método y versión, con un máximo de 2000 muestras recientes. Las alertas internas son Findings y aparecen en Sentinel y Centro de Control. La bandeja interna 0.7.0 publica resúmenes personales mediante outbox. NotificationDelivery conserva metadata histórica; Email, Teams, Slack y Webhook no están integrados.

Caso operativo: monitoreo y comparación histórica

Un monitor fija columnas, tasas permitidas y checks. Ejecutarlo contra un nuevo snapshot conserva ambos IDs y la definición exacta de cada métrica. Una alerta de null rate conduce al detalle del Run y al Finding, no modifica la versión observada. Deshabilitar una programación impide próximos despachos, sin borrar ocurrencias ni series ya registradas.

Al cambiar la definición de una métrica no se mezclan valores incompatibles bajo una sola tendencia. El lector agrupa por método/versión/dimensiones. En 0.5.1 los temporales sin zona siguen siendo STRING en los checks de schema; la apariencia de fecha en una celda no autoriza reinterpretar la columna.

11. Excepciones con validación técnica

Una excepción nace de un Finding y conserva finding_id, run_id de origen y configuration_id del snapshot que lo produjo. El Run de origen nunca cambia su decisión porque un caso se cierre o los datos se corrijan.



Estado	Función
OPEN / Abierta	Hallazgo convertido en caso, pendiente de gestión.
ASSIGNED / Asignada	Caso asignado a un usuario activo y autorizado de la organización.
INVESTIGATING / En gestión	Responsable investiga causa y prepara corrección.
PENDING_VALIDATION	Espera evidencia técnica posterior.
RESOLVED / Resuelta	Evidencia válida, causa y resolución documentadas.
DISCARDED / Descartada	Cierre administrativo con motivo obligatorio.
ACCEPTED / Aceptada	Riesgo o situación aceptada con motivo obligatorio.
NOT_APPLICABLE / No aplica	Cierre administrativo justificado.
REOPENED / Reabierta	Caso cerrado que vuelve a gestión con comentario y nueva fecha de referencia.

Criterios de elegibilidad

Se requiere un Run posterior SUCCESS, de la misma organización y el mismo configuration_id. Publicar otra versión de configuración crea otro snapshot y no valida automáticamente la excepción anterior. Se selecciona la candidata elegible más reciente; no se acepta una validación positiva obsoleta cuando existe una ejecución posterior que vuelve a fallar.

- Intake: la regla con rule_id que originó el hallazgo debe estar presente, evaluar al menos una fila y tener cero incumplimientos. En un caso histórico abierto sin ID se identifica la regla mediante su código/columna y fingerprint compatible, pero también se exigen métricas suficientes: evaluated_count mayor que cero y failed_count igual a cero. La mera ausencia de otro Finding no demuestra que la regla se haya evaluado.
- ReconOps: el control debe quedar conforme o desaparecer la clasificación asociada al hallazgo.
- Sentinel: el monitor debe volver a HEALTHY.

La UI deshabilita Resolver mientras no se cumplen los criterios y muestra el motivo. Una validación correcta se presenta como Validada técnicamente y enlaza el Run confirmatorio. Resolver sólo se acepta desde PENDING_VALIDATION y exige causa raíz y resolución. La API reevalúa antes de guardar.

El worker actualiza evidencia de casos PENDING_VALIDATION al terminar un Run. Conserva validation_run_id, validated_at, validation_evidence y evento TECHNICAL_VALIDATION. auto_resolve_enabled es una política por caso deshabilitada por defecto; habilitarla requiere permiso de cierre. Cuando está activa y la misma comprobación técnica es válida, el worker puede resolver el caso con actor SYSTEM, evento EXCEPTION_AUTO_RESOLVED, política CASE_OPT_IN y Run confirmatorio. No inventa una causa raíz humana y no duplica cierres en reintentos.

Gestión operativa

Cada caso admite assigned_user_id, prioridad, SLA en horas, due_at, comentarios y adjuntos. La prioridad es independiente de la severidad histórica. El SLA se calcula desde creación o reapertura, con UTC; una fecha objetivo explícita prevalece. No hay calendario laboral implícito. Los filtros backend cubren estado, módulo, severidad, prioridad, responsable, vencimiento y texto.

Reabrir exige comentario; conserva el cierre anterior en timeline y reinicia el plazo. Una ejecución creada antes de la reapertura no prueba la corrección actual. Se preservan las transiciones históricas OPEN -> INVESTIGATING y reapertura hacia OPEN para compatibilidad.

Los adjuntos permitidos se limitan a 10 MiB, se publican inmutables mediante StorageProvider y registran Artifact EXCEPTION_ATTACHMENT, hash y relación EXCEPTION_EVIDENCE. No se ejecutan ni se usan como reglas. La descarga exige organización/permisos, verifica integridad, fuerza attachment/nosniff y genera auditoría. Nombre y descripción se sanitizan; el filename nunca define una ruta física.

Las mutaciones usan version para concurrencia optimista. Timeline conserva actor estable, fecha, estado anterior/nuevo, comentario y referencia de validación. Los cierres administrativos no equivalen a RESOLVED ni fabrican evidencia. WAITING_EXTERNAL/FALSE_POSITIVE y resoluciones históricas permanecen legibles; los registros anteriores sin prueba se identifican como históricos, sin inventar un Run confirmatorio.

Si el Run candidato no contiene métricas suficientes para identificar y probar la regla histórica, el nuevo cierre queda bloqueado con motivo de evidencia insuficiente. Se requiere ejecutar nuevamente el mismo control y obtener evidencia verificable. Esto no modifica casos ya RESOLVED ni sus timelines históricos; endurece únicamente las nuevas decisiones de resolución.

Caso operativo: cierre respaldado por nueva ejecución

Asignar una excepción a una persona no cambia la severidad del Finding original. El responsable documenta causa, corrección y evidencia; PENDING_VALIDATION espera un nuevo Run elegible de la misma configuración. Una ejecución anterior a la corrección o una regla con cero filas evaluadas no demuestra resolución.

Resolver conserva vínculo al Run confirmatorio. Descartar, aceptar o marcar no aplicable es un cierre administrativo con motivo, no una aprobación técnica. Una reapertura deja historia y exige nueva validación posterior. Estos estados son distintos de la revisión UNKNOWN de Delivery: no se recicla Exception como sustituto de su registro operacional específico.

12. Centro de Control y navegación

El Centro de Control es un cockpit operativo calculado por backend. El usuario puede identificar qué falla, qué requiere atención y abrir la acción correspondiente. Las cifras no son constantes de presentación.

Elemento	Comportamiento actual
Filtros globales	Período 7d/30d/90d/all, dataset, módulo, estado operativo y criticidad.
Indicadores superiores	Salud general, controles fallidos, excepciones abiertas y datasets afectados. Variación contra período anterior cuando existe una comparación válida.
Requiere tu atención	Excepciones, hallazgos y fallos priorizados; muestra dataset, módulo, problema, fecha y acción directa.
Salud de los datos	Evolución temporal y desglose Intake/Recon/Sentinel.
Datasets afectados	Salud, hallazgos, última ejecución y tendencia.
Resumen por módulo	Ejecuciones/controles/monitores con fallos y alertas.
Ejecuciones recientes	Dataset, registros, hallazgos y duración cuando están disponibles; estado técnico separado de decisión.
Accesos rápidos	Carga de dataset y ejecución de operaciones.

health_score se pondera por unidades evaluadas según la agregación del backend. Las variaciones devuelven previous/delta o null cuando no hay base válida. Los filtros respetan organización y los enlaces navegan al recurso correspondiente. No se presenta ausencia de historia como mejora o deterioro.

Rutas SPA

/; /datasets; /datasets/:id; /connections; /connections/:id; /delivery/*; /intake/*; /recon/*; /sentinel/*; /runs; /runs/:id; /notifications; /exceptions/*; /rules; /audit; /settings*. Delivery incluye listado, /delivery/new, /delivery/destinations y /delivery/destinations/:id, /delivery/automation y /delivery/automation/:id, y el detalle de preflight persistido /delivery/validation/:id. El historial de adquisiciones se integra en Datasets y en el detalle del dataset; un aviso abre /datasets/:id?acquisition=:acquisition_id. /acquisitions identifica la API, no una pantalla SPA independiente. La administración de usuarios está integrada en Configuración. El menú conserva los módulos y el diseño navy/teal. Estados de carga, vacío y error tienen tratamiento específico; se conserva request_id en errores para diagnóstico.

Cerrar sesión confirma /auth/logout, limpia token CSRF, caché de consultas y estado de sesión, y navega con reemplazo a /. El usuario vuelve a la pantalla inicial de Trackvance (incluido "Entrar al entorno demo" cuando esté habilitado) sin mantener una pantalla autenticada en el historial inmediato.

Los detalles de dataset separan archivo original, fuente derivada, esquema/perfil y linaje. Los detalles de run separan Completada de Rechazado/Con hallazgos/Alerta. Los labels de negocio acompañan reason codes técnicos en resultados y evidencia. La numeración se muestra como Línea del archivo cuando representa línea física o Registro de la versión cuando corresponde.

Carga diferida y continuidad de la navegación

React.lazy y Suspense separan las rutas pesadas sin introducir librerías. El shell, identidad y navegación permanecen disponibles mientras se descarga el módulo. El detalle Delivery se carga por separado para evitar que Runs importe todo el builder de salida. La frontera de error ofrece recuperación visible si falla la carga de un chunk; no deja una región vacía como éxito aparente.

La medición 0.5.1 reduce el JavaScript inicial de 611,41 kB/179,47 kB gzip a 364,97 kB/116,77 kB gzip: aproximadamente 40,3% y 34,9% respectivamente. Es una reducción del arranque, no de la suma de todos los assets; las features siguen descargándose al usarlas. El build deja de emitir la advertencia de chunk mayor que 500 kB. Las rutas directas, recarga y permisos se prueban en navegador.

13. StorageProvider, artefactos y linaje

Archivo único y conjunto lógico

Artifact conserva id, kind, organización, ruta relativa, tamaño, SHA-256 y media type. Los archivos históricos se verifican y leen sin modificar bytes o hash. El media type application/vnd.trackvance.parquet-set+json identifica descriptor schema_version=1, kind=PARQUET_DATASET. Incluye esquema y partes en orden explícito: ordinal, artifact_id, path, sha256, size_bytes, row_count, más totales y metadata. El SHA del descriptor representa inequívocamente el conjunto de identidades; dataset_paths verifica descriptor y cada parte antes de devolver sus paths.

put_dataset copia/registra partes inmutables y publica el descriptor al final, en la misma transacción de metadata que sus referencias DATASET_PART. Una caída puede dejar archivos privados/no referenciados, pero no una DatasetVersion de conjunto incompleto disponible. Un cambio en bytes, path o conteo no se acepta como la misma identidad. La limpieza no borra partes referenciadas bajo la etiqueta de staging. Los paths del cliente no determinan rutas del proveedor.

Resultados, evidencia y consultas

Intake errors/accepted, Recon resultados y Sentinel resultados pueden publicarse como múltiples partes. La evidencia guarda sus propias referencias y hashes. La consulta de resultados usa scans en disco, orden estable y paginación; las operaciones globales se completaron antes de publicar. No vuelve a cargar toda la población en un DataFrame local para exportar o generar el preview.

La descarga de conjunto devuelve ZIP_STORED con descriptor.json exacto y las partes en los paths declarados. Se comprueba integridad y reserva de disco, y el temporal de descarga se elimina al terminar. El paquete contiene la población completa, no una muestra. El CSV completo de resultados se escribe por iteración acotada, se registra como EXPORT_CSV con hash/EXPORT_OF y permite descarga nativa del navegador. Excel tiene sus límites explícitos y no trunca para aparentar éxito. El inventario y el backup siguen todos los componentes del descriptor.

Linaje y preservación

DATASET_PART relaciona descriptor→Artifact parte; las relaciones de entrada, salida y evidencia conservan identidades históricas. DELIVERY_INPUT vincula la DatasetVersion exacta con la Run; DELIVERED_TO conserva como target_type DELIVERY_DESTINATION_VERSION. Una ocurrencia encadenada usa output_version_id del Intake concreto. No se sustituye por la última versión al consumir el evento.

Los temporales pertenecen al proveedor que los asignó, incluso un puerto de almacenamiento opaco. No se valida un path temporal remoto contra la raíz global de FileArtifactStore. El contrato materialize es la autoridad para acceder a referencias. Esta regla conserva la sustituibilidad de StorageProvider.

14. Manifest schema 2 y auditoría

Cada Run completado publica un manifest inmutable con schema_version=2. Excepción operativa explícita: si una entrega ya COMMITTED pierde la publicación local, conserva el commit y marca evidence_status=PENDING_REPAIR; nunca repite la transacción para fabricar la evidencia. read_manifest/adapt_manifest centraliza la lectura de v1 histórico; la adaptación se realiza en memoria, sin sobrescribir archivos anteriores.

```
{
  "schema_version": 2,
  "run_id": "<id>",
  "module": "recon",
  "initiated_by": {
    "type": "USER", "id": "<id estable>",
    "display_name": "Equipo Trackvance"
  },
  "result_artifacts": [{
    "artifact_id": "<id>", "kind": "RECON_RESULTS",
    "name": "results.parquet", "sha256": "<hash>",
```

```

    "size_bytes": 1234
  }
}
```

El contrato completo incluye `started_at/finished_at`, identidad/legacy, versión de Trackvance, plan processing con engine/version y schedule cuando corresponde, inputs con DatasetVersion/SHA/schema hash/origen/ingestion_metadata, configuración efectiva/id/version/config_hash/stored_config_hash, métricas, output_version_id y result_artifacts. references añade identidad y hashes de DatasetVersions usadas para integridad referencial. Delivery añade una sección de livery con DestinationVersion, target, estrategia, intento, resultado y receipt; es una extensión aditiva de schema 2. Los snapshots anteriores no se reescriben.

La versión del producto y la del motor se distinguen: engine_version de nivel superior identifica Trackvance en el manifest actual; processing.engine_version identifica la versión real del motor seleccionado, Polars o PySpark. La UI y exports muestran ambas con su contexto.

Identidad y eventos

AuditEvent conserva actor_type, actor_id estable, actor visible, actor_legacy, event_type, subject_type/id, request_id, run_id, metadata sanitizada y timestamp. Actor puede representar USER, WORKER o SYSTEM. Los nombres visibles no sustituyen la identidad persistente.

DATASET_UPLOADED, DATASET_DERIVED, CONFIGURATION_PUBLISHED, RUN_QUEUED, RUN_COMPLETED, EXCEPTION_CREATED, EXCEPTION_UPDATED y EXCEPTION_VALIDATION_CHECKED documentan el ciclo. EXPORT_DOWNLOADED, EVIDENCE_DOWNLOADED y ARTIFACT_DOWNLOADED registran el acceso relevante a evidencia. La UI enlaza recursos desde los eventos.

La administración añade USER_CREATED, USER_UPDATED, USER_CREDENTIALS_REGENERATED y USER_PASSWORD_CHANGED. La gestión operativa audita comentarios, adjuntos y resolución automática; la programación conserva revisiones y eventos de despacho. Delivery registra publicación, encolado, STARTED, COMMITTED, FAILED o UNKNOWN con actor estable y referencias, nunca contraseña ni filas. Las filas de negocio recibidas, credenciales, hashes de contraseña y sesiones nunca forman parte del payload público de estos eventos.

Se excluyen claves de passwords, tokens y secretos de metadata/evidencia. Los casos legacy sin identidad demostrable se marcan como legacy; no se atribuyen a un UUID de usuario inventado. Las auditorías de exportación agregan evidencia sin alterar el manifest original del Run.

15. Reportes Excel de negocio y resultado completo

Los reportes Excel conservan resumen, reglas, entradas, trazabilidad y seguridad contra fórmulas. La evolución no extiende arbitrariamente el dominio de Excel: un máximo de 100.000 filas, 500.000 celdas o 16 MiB de valores limita el workbook local; las restricciones de longitud/columnas del generador continúan aplicándose. No se cortan filas/celdas para devolver un archivo aparentemente completo.

Cuando excede la cota, el endpoint devuelve 422 EXPORT_LIMIT_EXCEEDED y una referencia complete_download al CSV de la Run. El detalle de ejecución ofrece CSV completo además de Excel/manifiesto. CSV recorre todos los resultados y protege celdas con inicio =,+,-,@,tab/CR mediante prefijo seguro; JSON complejo se serializa explícitamente. La descarga registra Artifact/hash/linaje/auditoría. El navegador recibe CSV completo mediante enlace de descarga, sin Blob gigante.

La paginación de pantalla sigue siendo 50 filas por consulta. El usuario debe poder distinguir esa vista del artefacto completo. Ni las métricas ni los resultados de un millón de filas dependen de la página visible.

La página de resultados limita el payload UTF-8 a 16 MiB antes de decodificar JSON. Una solicitud que excede la cota recibe 422 RESULT_PAGE_BYTE_LIMIT, sin truncar la página ni incorporar parámetros nuevos. El CSV completo sigue disponible por el recorrido acotado en disco. /system/engines informa esta cota y la de la muestra del perfil entre los límites efectivos.

La muestra del perfil presenta hasta 20 filas y 8 MiB de JSON UTF-8 observado. sampled_rows/sample_bytes informan lo mostrado; sample_limited señala el límite de bytes y sample_byte_limit declara su cota. Una muestra menor o vacía no cambia las estadísticas persistentes de la población completa.

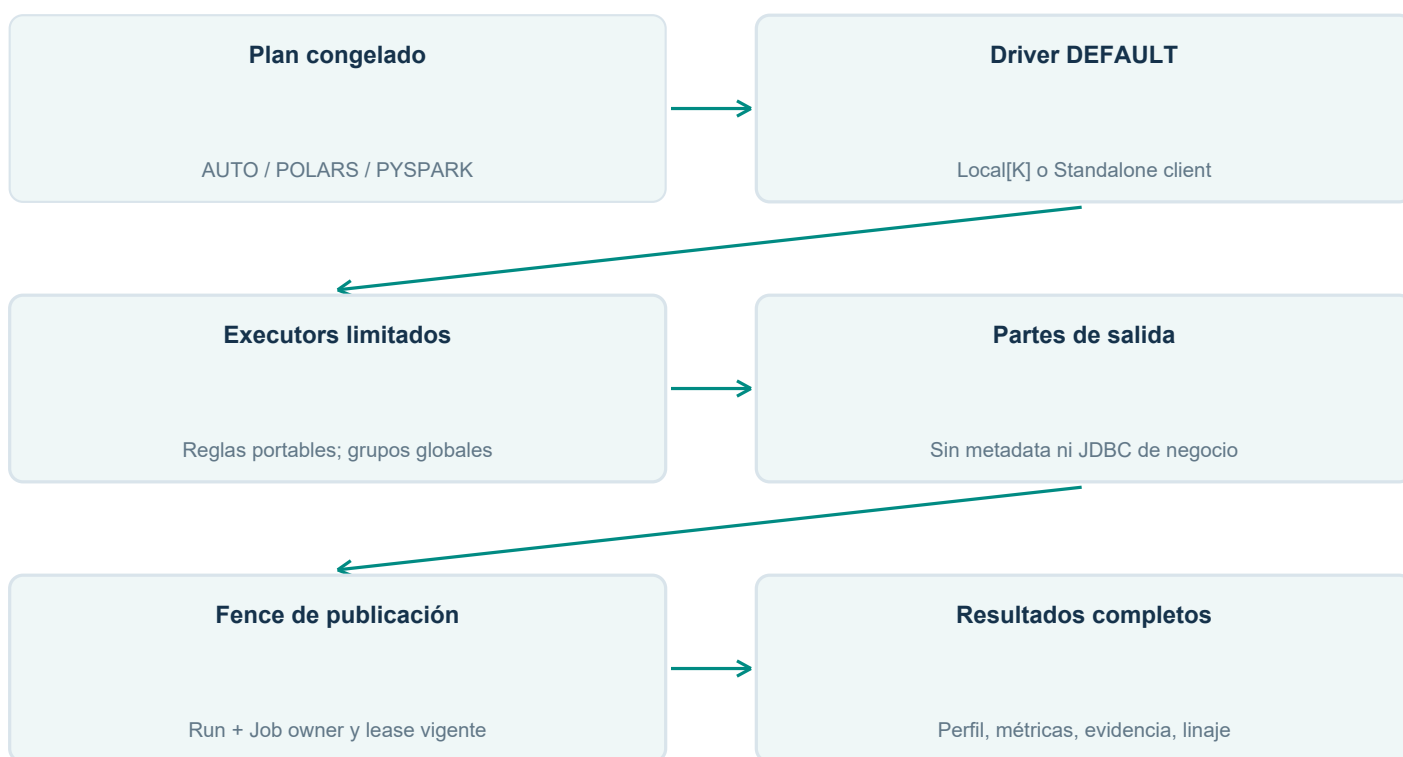
16. Ejecución, planificación, PySpark y recuperación

Selección y congelación del plan

La UI ofrece Automático, Polars y PySpark. `requested_engine` conserva la elección y el plan `schema_version=3` explica engine efectivo, versión, `reason_code`, `allowed/rejection_code`, input bytes, working set estimado, disk requerido, presupuesto y parámetros. AUTO usa Polars dentro de su presupuesto y PySpark cuando la carga exige ejecución acotada y el runtime puede hacerlo. Una selección explícita también verifica recursos; un fallo nunca cambia silenciosamente de engine.

El planner incluye fuente, target y DatasetVersions de referencia deduplicadas, con el tamaño físico de partes y ancho observado del perfil. No suma sólo el JSON del descriptor. Distingue `memory_soft_bytes` estimado de límites cgroup y cotas realmente aplicadas de batch/row/group/timeout/disco. `ENGINE_UNAVAILABLE` y `RESOURCE_MEMORY_INSUFFICIENT`, `RESOURCE_SPARK_MEMORY_INSUFFICIENT` o `RESOURCE_DISK_INSUFFICIENT` son decisiones visibles anteriores a ejecutar.

`estimated_working_set_bytes` incorpora `estimated_polars_evidence_bytes`: cada regla Intake habilitada puede fallar en todas las filas, y Recon conserva detalles por comparación. La cota considera valores, parámetros, condiciones y estructuras de salida coexistentes; no extrapola el número de fallos desde la muestra. AUTO puede elegir Spark con un archivo pequeño y muchas reglas. POLARS explícito que supera el presupuesto se rechaza antes de materializar la población. Esta estimación conservadora sigue separada del límite duro real del proceso.



ProcessingEngine y ejecución real

Polars preserva su recorrido estándar pequeño. PySpark 4.0.3 ejecuta Intake, ReconOps y Sentinel sobre partes materializadas, con resultados completos en particiones, métricas exactas y kernels portables. Referencias, unicidad y joins son globales; Recon agrupa claves completas entre particiones y conserva duplicados/cardinalidad. No usa `collect/toPandas/to_dicts/coalesce(1)` sobre la población para regresar al procesamiento local. Sólo reduce métricas, muestras y listas de IDs estrictamente limitadas para filtrar fallos escasos.

Las referencias y la unicidad usan una unión ordenada que consume las claves de forma incremental, sin retener una población completa con la misma clave. Sentinel consulta sólo las ventanas históricas compatibles que cada check necesita.

Decimal se conserva como Decimal, incluidas fracciones fuera del dominio Decimal(38) de Spark; no se convierte a float. Null no sustituye errores de interpretación. Unicode, regex, normalización explícita, fechas, zona horaria y fracciones temporales siguen los contratos portables. La agrupación por clave tiene cotas efectivas de 10.000 registros y 16 MiB por grupo por defecto; una clave sesgada que las exceda se rechaza completa con RESOURCE_RECON_GROUP_LIMIT. La fila de Spark tiene máximo 64 KiB observado, batch 2.048 registros/8 MiB y result bytes del driver 16 MiB por defecto. No se presenta un grupo cortado como conciliación correcta.

El lote Recon reserva bytes de input y evidencia prevista por comparación. Usa la cardinalidad real del grupo: duplicados no generan comparaciones y una agregación conserva todos sus números de origen. Si una clave indivisible excede el presupuesto de evidencia por grupo, falla antes de construir el resultado portable. La previsión es conservadora y no equivale a un pico RSS medido.

Spark local

El perfil por defecto utiliza local[2], driver/executor 768 MiB, 2 cores totales, 4 particiones y Java 17. Se limita CPU/RAM del contenedor DEFAULT. Local[K] ejecuta en una JVM local con threads; no certifica distribución entre hosts. El worker registra runtime/version/master/parámetros efectivos y medidas de ejecución. Si Java/PySpark no están disponibles, el plan rechaza el engine.

Standalone opcional y ubicación del driver

deploy/compose.spark-standalone.yml añade master y dos workers/executors. El driver Python permanece en worker DEFAULT, modo client soportado para PySpark. Driver host worker y bind 0.0.0.0, puertos internos 7078/7079 y master 7077 permiten conectividad privada. API y worker comparten TRACKVANCE_SPARK_MASTER para no congelar un plan local y ejecutarlo en Standalone. Todos ven el mismo volumen de snapshots; los executors acceden al almacenamiento y código portable, sin metadata ni credenciales de fuentes/destinos. No se publican administradores web al host.

```
docker compose -f compose.yml -f deploy/compose.spark-standalone.yml --profile spark-standalone up -d --wait
```

El comando se ejecuta con el .env/proyecto correcto, sin activar el perfil por defecto. Dos executors/procesos y sus application IDs prueban reparto dentro del mismo host; no constituyen capacidad multinodo. La evidencia muestra memoria separada de driver, master y executors y ausencia de OOM.

Lease, cancelación, timeout y publicación

JobQueue reclama por lane con owner y lease. El heartbeat y un controlador independiente revisan cancelación, deadline, disco y autoridad durante Spark. Cancelar usa job group; perder el worker puede dejar archivos privados, pero no autoriza publicar. El caller toma lock Run y CAS de Job RUNNING, owner vigente y lease no vencido antes de publicar, manteniendo el fence hasta commit. La transacción registra partes, perfil de accepted output, Findings, manifiesto, linaje, estado terminal y outbox. Un owner antiguo no puede sobrescribir al nuevo.

Una cancelación anterior a una escritura remota concluye CANCELLED y limpia staging. Para Delivery la frontera STARTED conserva sus estados remotos; un commit conocido conserva COMMITTED pese a una cancelación tardía. El scheduler continúa en otro proceso mientras DEFAULT permanece ocupado.

Paridad y límites comprobados

La comparación es lógica: filas, clasificaciones, métricas y decisiones, no identidad de bytes entre engines. Cada archivo tiene su propio SHA verificable. Las suites dedicadas ejecutan opt-ins JVM sin SKIP y prueban nulls, Unicode, temporales, referencias, duplicados entre particiones, condiciones, transforms, agregaciones y comparaciones exactas. Las pruebas de volumen y la integración con Run/Job/publicación se registran por separado de una prueba directa de kernel.

17. Modelo persistente, restricciones y migraciones

Migraciones aditivas y baseline conservada

0013_async_acquisition añade acquisition_uploads/acquisition_runs y amplía Job para Run XOR AcquisitionRun con lane ACQUISITION. Run deja de ser obligatoria sólo para esa identidad alternativa; la restricción persistente impide ambas asociaciones o ninguna. FK e índices de claim/lease permiten usar la misma cola. Una adquisición no inventa DatasetVersion para satisfacer una FK.

0014_automation_outbox añade delivery_automations, delivery_automation_versions, delivery_occurrences, delivery_input_claims, delivery_target_guards, delivery_target_decisions, outbox_events, event_consumptions e internal_notifications. Los UNIQUE preservan identidad de versión, trigger, input no repetido, target, dedupe/event-consumer y event-recipient. Las tablas del negocio histórico no se convierten en estos objetos ni se generan eventos por escanearlas durante la migración.

0015_sentinel_execution_identity añade responsible_user_id a monitor_schedule_versions. monitor_schedules recibe legacy_enabled_before_identity; el responsable efectivo se consulta desde su revisión vigente, no desde una columna duplicada en schedule. Sólo backfilla Actor USER que corresponde a un usuario verificable de la misma organización. Los schedules sin identidad se pausan; no se inventa un dueño. legacy_enabled_before_identity conserva el enabled anterior necesario para proyección estricta y recuperación del estado. La UI permite asignación explícita posterior sin reescribir actores de ocurrencias/ejecuciones históricas.

El inventario final consta de 42 tablas. Las migraciones 0001..0012 permanecen byte a byte. Los backups identifican revisión real y state 6; la compatibilidad con state 5 proyecta únicamente los defaults/columnas documentados. Tablas nuevas deben estar vacías al comparar una migración legacy antes de iniciar despachos; una tabla física desconocida o una diferencia fuera de esa proyección se rechaza.

Campos y vínculos del inventario completo

El inventario siguiente documenta las 42 tablas y sus 509 campos, 95 índices, PK/FK, nulabilidad y restricciones CHECK/UNIQUE. Se genera desde el ORM vigente e identifica las once tablas nuevas, las tres estructuras evolucionadas y las 28 estructuras preservadas. Los cambios de datos permitidos por migración y la proyección legacy se describen arriba; «preservada» se refiere al esquema, sin afirmar que una operación nueva no pueda añadir historia legítima.

Todas las entidades de Record conservan id, organization_id y created_at. Los campos JSON son snapshots/contratos versionados, no ubicaciones de secretos. AcquisitionRun.source_snapshot y effective_limits son inmutables durante un intento; los progresos son observaciones actualizables. El payload de outbox contiene referencias y decisiones sanitizadas, nunca valores de negocio. EventConsumption almacena lease y attempts hasta cinco; InternalNotification almacena recipient_user_id/read_at y enlace personal. DeliveryTargetDecision vincula review histórica, actor real y nota, sin cambiar el intento UNKNOWN.

acquisition_runs

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
dataset_id	VARCHAR(64); obligatorio; FK datasets.id
upload_id	VARCHAR(64); nullable; FK acquisition_uploads.id
connection_version_id	VARCHAR(64); nullable; FK external_connection_versions.id
source_type	VARCHAR(30); obligatorio
filename	VARCHAR(240); obligatorio
source_snapshot	JSON; obligatorio
reader_options	JSON; obligatorio
column_overrides	JSON; obligatorio
effective_limits	JSON; obligatorio

Campo	Tipo, nulabilidad y relación
request_hash	VARCHAR(64); obligatorio
idempotency_key	VARCHAR(128); nullable
initiated_by_id	VARCHAR(64); obligatorio; FK users.id
initiated_by_name	VARCHAR(200); obligatorio
status	VARCHAR(20); obligatorio
stage	VARCHAR(50); obligatorio
attempt_id	VARCHAR(64); nullable
attempts	INTEGER; obligatorio
processed_rows	BIGINT; obligatorio
processed_bytes	BIGINT; obligatorio
total_rows	BIGINT; nullable
total_bytes	BIGINT; nullable
cancel_requested	BOOLEAN; obligatorio
output_version_id	VARCHAR(64); nullable; FK dataset_versions.id
error_code	VARCHAR(80); nullable
error_message	VARCHAR(500); nullable
started_at	DATETIME; nullable
finished_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: (source_type = 'UPLOAD' AND upload_id IS NOT NULL AND connection_version_id IS NULL) OR (source_type IN ('POSTGRESQL','SQLSERVER') AND upload_id IS NULL AND connection_version_id IS NOT NULL)

Restricción: ForeignKeyConstraint(connection_version_id)

Restricción: ForeignKeyConstraint(dataset_id)

Restricción: ForeignKeyConstraint(initiated_by_id)

Restricción: ForeignKeyConstraint(output_version_id)

Restricción: ForeignKeyConstraint(upload_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,initiated_by_id,idempotency_key)

Restricción: UniqueConstraint(output_version_id)

Restricción: UniqueConstraint(upload_id)

Restricción: status IN ('QUEUED','RUNNING','SUCCESS','FAILED','CANCELLED')

Índice: ix_acquisition_org_dataset_created (organization_id, dataset_id, created_at)

Índice: ix_acquisition_runs_dataset_id (dataset_id)

Índice: ix_acquisition_runs_initiated_by_id (initiated_by_id)

Índice: ix_acquisition_runs_organization_id (organization_id)

Índice: ix_acquisition_runs_status (status)

acquisition_uploads

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
user_id	VARCHAR(64); obligatorio; FK users.id
filename	VARCHAR(240); obligatorio
path	TEXT; obligatorio
sha256	VARCHAR(64); obligatorio
size_bytes	BIGINT; obligatorio
source_format	VARCHAR(30); obligatorio
status	VARCHAR(20); obligatorio
expires_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: status IN ('RECEIVED','REGISTERED','CONSUMED','EXPIRED')

Índice: ix_acquisition_uploads_expires_at (expires_at)

Índice: ix_acquisition_uploads_organization_id (organization_id)

Índice: ix_acquisition_uploads_user_id (user_id)

artifact_links

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
relation	VARCHAR(40); obligatorio
source_type	VARCHAR(40); obligatorio
source_id	VARCHAR(64); obligatorio
target_type	VARCHAR(40); obligatorio
target_id	VARCHAR(64); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,relation,source_type,source_id,target_type,target_id)

Índice: ix_artifact_links_organization_id (organization_id)

Índice: ix_artifact_links_source_id (source_id)

Índice: ix_artifact_links_target_id (target_id)

artifacts

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
kind	VARCHAR(40); obligatorio
name	VARCHAR(240); obligatorio
path	TEXT; obligatorio
sha256	VARCHAR(64); obligatorio
size_bytes	BIGINT; obligatorio
media_type	VARCHAR(120); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(path)

Índice: ix_artifacts_kind (kind)

Índice: ix_artifacts_organization_id (organization_id)

audit_events

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
event_type	VARCHAR(80); obligatorio
actor	VARCHAR(120); obligatorio
actor_type	VARCHAR(20); obligatorio
actor_id	VARCHAR(64); obligatorio
actor_legacy	BOOLEAN; obligatorio
request_id	VARCHAR(128); nullable
run_id	VARCHAR(64); nullable
subject_type	VARCHAR(60); obligatorio
subject_id	VARCHAR(64); obligatorio
message	TEXT; obligatorio
metadata_json	JSON; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Índice: ix_audit_events_organization_id (organization_id)

Índice: ix_audit_events_request_id (request_id)

Índice: ix_audit_events_run_id (run_id)

configurations

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(160); obligatorio
module	VARCHAR(20); obligatorio
version	INTEGER; obligatorio
dataset_id	VARCHAR(64); obligatorio; FK datasets.id
target_dataset_id	VARCHAR(64); nullable; FK datasets.id
owner	VARCHAR(120); obligatorio
description	TEXT; obligatorio
status	VARCHAR(20); obligatorio
config	JSON; obligatorio
previous_version_id	VARCHAR(64); nullable; FK configurations.id
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(dataset_id)

Restricción: ForeignKeyConstraint(previous_version_id)

Restricción: ForeignKeyConstraint(target_dataset_id)

Restricción: PrimaryKeyConstraint(id)

Índice: ix_configurations_module (module)

Índice: ix_configurations_organization_id (organization_id)

Índice: ix_configurations_previous_version_id (previous_version_id) UNIQUE

dataset_source_bindings

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
dataset_id	VARCHAR(64); obligatorio; FK datasets.id
connection_id	VARCHAR(64); obligatorio; FK external_connections.id
schema_name	VARCHAR(128); obligatorio
object_name	VARCHAR(128); obligatorio
object_kind	VARCHAR(20); obligatorio
column_overrides	JSON; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio

Campo	Tipo, nulabilidad y relación
created_at	DATETIME; obligatorio
Restricción: ForeignKeyConstraint(connection_id)	
Restricción: ForeignKeyConstraint(dataset_id)	
Restricción: PrimaryKeyConstraint(id)	
Restricción: UniqueConstraint(dataset_id)	
Índice: ix_dataset_source_bindings_connection_id (connection_id)	
Índice: ix_dataset_source_bindings_organization_id (organization_id)	

dataset_versions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
dataset_id	VARCHAR(64); obligatorio; FK datasets.id
version	INTEGER; obligatorio
filename	VARCHAR(240); obligatorio
source_type	VARCHAR(30); obligatorio
sha256	VARCHAR(64); obligatorio
schema_hash	VARCHAR(64); obligatorio
size_bytes	INTEGER; obligatorio
row_count	INTEGER; obligatorio
column_count	INTEGER; obligatorio
profile_status	VARCHAR(20); obligatorio
original_path	TEXT; obligatorio
canonical_path	TEXT; obligatorio
schema_json	JSON; obligatorio
profile	JSON; obligatorio
ingestion_metadata	JSON; obligatorio
parent_version_id	VARCHAR(64); nullable
original_artifact_id	VARCHAR(64); nullable; FK artifacts.id
canonical_artifact_id	VARCHAR(64); nullable; FK artifacts.id
source_run_id	VARCHAR(64); nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio
Restricción: ForeignKeyConstraint(canonical_artifact_id)	
Restricción: ForeignKeyConstraint(dataset_id)	
Restricción: ForeignKeyConstraint(original_artifact_id)	
Restricción: PrimaryKeyConstraint(id)	

Restricción: UniqueConstraint(dataset_id,version)

Índice: ix_dataset_versions_dataset_id (dataset_id)

Índice: ix_dataset_versions_organization_id (organization_id)

Índice: ix_dataset_versions_source_run_id (source_run_id)

datasets

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(160); obligatorio
description	TEXT; obligatorio
domain	VARCHAR(80); obligatorio
owner	VARCHAR(120); obligatorio
criticality	VARCHAR(20); obligatorio
status	VARCHAR(20); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,name)

Índice: ix_datasets_organization_id (organization_id)

delivery_attempts

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
run_id	VARCHAR(64); obligatorio; FK runs.id
destination_version_id	VARCHAR(64); obligatorio; FK delivery_destination_versions.id
attempt_number	INTEGER; obligatorio
idempotency_key	VARCHAR(64); obligatorio
status	VARCHAR(20); obligatorio
target_locator	VARCHAR(300); obligatorio
system_audit	JSON; obligatorio
rows_attempted	BIGINT; nullable
rows_written	BIGINT; nullable
rows_inserted	BIGINT; nullable
rows_updated	BIGINT; nullable
bytes_sent	BIGINT; nullable
remote_reference	VARCHAR(300); nullable
error_code	VARCHAR(80); nullable

Campo	Tipo, nulabilidad y relación
error_message	TEXT; nullable
started_at	DATETIME; obligatorio
finished_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(destination_version_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(run_id,attempt_number)

Índice: ix_delivery_attempts_destination_version_id (destination_version_id)

Índice: ix_delivery_attempts_idempotency_key (idempotency_key)

Índice: ix_delivery_attempts_organization_id (organization_id)

Índice: ix_delivery_attempts_run_id (run_id)

Índice: ix_delivery_attempts_status (status)

delivery_automation_versions

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
automation_id	VARCHAR(64); obligatorio; FK delivery_automations.id
version	INTEGER; obligatorio
configuration_id	VARCHAR(64); obligatorio; FK configurations.id
responsible_user_id	VARCHAR(64); obligatorio; FK users.id
enabled	BOOLEAN; obligatorio
settings	JSON; obligatorio
actor_id	VARCHAR(64); obligatorio; FK users.id
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(actor_id)

Restricción: ForeignKeyConstraint(automation_id)

Restricción: ForeignKeyConstraint(configuration_id)

Restricción: ForeignKeyConstraint(responsible_user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(automation_id,version)

Índice: ix_delivery_automation_versions_automation_id (automation_id)

Índice: ix_delivery_automation_versions_organization_id (organization_id)

delivery_automations

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
name	VARCHAR(160); obligatorio
version	INTEGER; obligatorio
enabled	BOOLEAN; obligatorio
responsible_user_id	VARCHAR(64); obligatorio; FK users.id
next_run_at	DATETIME; nullable
updated_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(responsible_user_id)

Restricción: PrimaryKeyConstraint(id)

Índice: ix_delivery_automations_next_run_at (next_run_at)

Índice: ix_delivery_automations_organization_id (organization_id)

Índice: ix_delivery_automations_responsible_user_id (responsible_user_id)

delivery_destination_versions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
destination_id	VARCHAR(64); obligatorio; FK delivery_destinations.id
version	INTEGER; obligatorio
config	JSON; obligatorio
secret_reference	VARCHAR(240); obligatorio
config_hash	VARCHAR(64); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(destination_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(destination_id,version)

Índice: ix_delivery_destination_versions_destination_id (destination_id)

Índice: ix_delivery_destination_versions_organization_id (organization_id)

delivery_destinations

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(160); obligatorio
sink_type	VARCHAR(30); obligatorio
enabled	BOOLEAN; obligatorio
deleted	BOOLEAN; obligatorio
version	INTEGER; obligatorio
last_test_status	VARCHAR(20); obligatorio
last_test_at	DATETIME; nullable
last_test_message	VARCHAR(240); obligatorio
updated_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Índice: ix_delivery_destinations_organization_id (organization_id)

delivery_input_claims

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
automation_id	VARCHAR(64); obligatorio; FK delivery_automations.id
dataset_version_id	VARCHAR(64); obligatorio; FK dataset_versions.id
occurrence_id	VARCHAR(64); obligatorio; FK delivery_occurrences.id
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(automation_id)

Restricción: ForeignKeyConstraint(dataset_version_id)

Restricción: ForeignKeyConstraint(occurrence_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(automation_id,dataset_version_id)

Índice: ix_delivery_input_claims_organization_id (organization_id)

delivery_occurrences

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
automation_id	VARCHAR(64); obligatorio; FK delivery_automations.id
automation_version_id	VARCHAR(64); obligatorio; FK delivery_automation_versions.id
trigger_key	VARCHAR(180); obligatorio
request_hash	VARCHAR(64); nullable
origin	VARCHAR(20); obligatorio
planned_at	DATETIME; obligatorio
dispatched_at	DATETIME; obligatorio
source_run_id	VARCHAR(64); nullable; FK runs.id
dataset_version_id	VARCHAR(64); nullable; FK dataset_versions.id
run_id	VARCHAR(64); nullable; FK runs.id
status	VARCHAR(30); obligatorio
reason_code	VARCHAR(80); nullable
coalesced_intervals	INTEGER; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

- Restricción: ForeignKeyConstraint(automation_id)
- Restricción: ForeignKeyConstraint(automation_version_id)
- Restricción: ForeignKeyConstraint(dataset_version_id)
- Restricción: ForeignKeyConstraint(run_id)
- Restricción: ForeignKeyConstraint(source_run_id)
- Restricción: PrimaryKeyConstraint(id)
- Restricción: UniqueConstraint(automation_id,trigger_key)
- Restricción: UniqueConstraint(run_id)
- Índice: ix_delivery_occurrences_automation_id (automation_id)
- Índice: ix_delivery_occurrences_organization_id (organization_id)

delivery_reviews

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
run_id	VARCHAR(64); obligatorio; FK runs.id
delivery_attempt_id	VARCHAR(64); obligatorio; FK delivery_attempts.id
reviewer_id	VARCHAR(64); obligatorio; FK users.id
reviewer_name	VARCHAR(200); obligatorio
outcome	VARCHAR(40); obligatorio

Campo	Tipo, nulabilidad y relación
note	VARCHAR(4000); obligatorio
verified_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(delivery_attempt_id)

Restricción: ForeignKeyConstraint(reviewer_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: outcome IN ('REMOTE_COMMIT_OBSERVED', 'REMOTE_NOT_COMMITTED_OBSERVED', 'INCONCLUSIVE')

Índice: ix_delivery_reviews_delivery_attempt_id (delivery_attempt_id)

Índice: ix_delivery_reviews_organization_id (organization_id)

Índice: ix_delivery_reviews_reviewer_id (reviewer_id)

Índice: ix_delivery_reviews_run_id (run_id)

delivery_target_decisions

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
guard_id	VARCHAR(64); obligatorio; FK delivery_target_guards.id
run_id	VARCHAR(64); obligatorio; FK runs.id
review_id	VARCHAR(64); obligatorio; FK delivery_reviews.id
actor_id	VARCHAR(64); obligatorio; FK users.id
note	VARCHAR(2000); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(actor_id)

Restricción: ForeignKeyConstraint(guard_id)

Restricción: ForeignKeyConstraint(review_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(run_id)

Índice: ix_delivery_target_decisions_organization_id (organization_id)

delivery_target_guards

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
target_fingerprint	VARCHAR(64); obligatorio
active_run_id	VARCHAR(64); nullable; FK runs.id
unknown_run_id	VARCHAR(64); nullable; FK runs.id
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(active_run_id)

Restricción: ForeignKeyConstraint(unknown_run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,target_fingerprint)

Índice: ix_delivery_target_guards_organization_id (organization_id)

delivery_target_policies

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
destination_id	VARCHAR(64); obligatorio; FK delivery_destinations.id
target_fingerprint	VARCHAR(64); obligatorio
sink_type	VARCHAR(30); obligatorio
host_snapshot	VARCHAR(253); obligatorio
port	INTEGER; obligatorio
database	VARCHAR(128); obligatorio
schema_name	VARCHAR(128); obligatorio
table_name	VARCHAR(128); obligatorio
audit_columns_required	BOOLEAN; obligatorio
enabled_at	DATETIME; obligatorio
enabled_by_user_id	VARCHAR(64); obligatorio; FK users.id
enabled_by_username	VARCHAR(128); obligatorio
materialized_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(destination_id)

Restricción: ForeignKeyConstraint(enabled_by_user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,target_fingerprint)

Restricción: audit_columns_required = true

Índice: ix_delivery_target_policies_destination_id (destination_id)

Índice: ix_delivery_target_policies_organization_id (organization_id)

event_consumptions

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
event_id	VARCHAR(64); obligatorio; FK outbox_events.id
consumer	VARCHAR(20); obligatorio
status	VARCHAR(20); obligatorio
attempts	INTEGER; obligatorio
available_at	DATETIME; obligatorio
lease_owner	VARCHAR(64); nullable
lease_until	DATETIME; nullable
error_code	VARCHAR(80); nullable
completed_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(event_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(event_id,consumer)

Restricción: attempts >= 0 AND attempts <= 5

Índice: ix_event_consumer_pending (consumer, status, available_at)

Índice: ix_event_consumptions_organization_id (organization_id)

exception_attachments

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
exception_id	VARCHAR(64); obligatorio; FK exceptions.id
artifact_id	VARCHAR(64); obligatorio; FK artifacts.id
uploaded_by_id	VARCHAR(64); obligatorio; FK users.id
description	VARCHAR(500); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(artifact_id)

Restricción: ForeignKeyConstraint(exception_id)

Restricción: ForeignKeyConstraint(uploaded_by_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(artifact_id)

Índice: ix_exception_attachments_exception_id (exception_id)

Índice: ix_exception_attachments_organization_id (organization_id)

exceptions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
display_id	VARCHAR(30); obligatorio
finding_id	VARCHAR(64); nullable; FK findings.id
run_id	VARCHAR(64); obligatorio; FK runs.id
configuration_id	VARCHAR(64); obligatorio; FK configurations.id
validation_run_id	VARCHAR(64); nullable; FK runs.id
title	VARCHAR(240); obligatorio
module	VARCHAR(20); obligatorio
severity	VARCHAR(20); obligatorio
state	VARCHAR(30); obligatorio
owner	VARCHAR(120); obligatorio
assigned_user_id	VARCHAR(64); nullable; FK users.id
priority	VARCHAR(20); obligatorio
sla_hours	INTEGER; nullable
due_at	DATETIME; nullable
reopened_at	DATETIME; nullable
auto_resolve_enabled	BOOLEAN; obligatorio
root_cause	TEXT; obligatorio
resolution	TEXT; obligatorio
administrative_reason	TEXT; obligatorio
validation_evidence	JSON; obligatorio
validated_at	DATETIME; nullable
version	INTEGER; obligatorio
updated_at	DATETIME; obligatorio
events	JSON; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(assigned_user_id)

- Restricción: ForeignKeyConstraint(configuration_id)
- Restricción: ForeignKeyConstraint(finding_id)
- Restricción: ForeignKeyConstraint(run_id)
- Restricción: ForeignKeyConstraint(validation_run_id)
- Restricción: PrimaryKeyConstraint(id)
- Restricción: UniqueConstraint(display_id)
- Restricción: UniqueConstraint(finding_id)
- Índice: ix_exceptions_assigned_user_id (assigned_user_id)
- Índice: ix_exceptions_configuration_id (configuration_id)
- Índice: ix_exceptions_due_at (due_at)
- Índice: ix_exceptions_organization_id (organization_id)
- Índice: ix_exceptions_state (state)
- Índice: ix_exceptions_validation_run_id (validation_run_id)

external_connection_versions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
connection_id	VARCHAR(64); obligatorio; FK external_connections.id
version	INTEGER; obligatorio
host	VARCHAR(253); obligatorio
port	INTEGER; obligatorio
database	VARCHAR(128); obligatorio
username	VARCHAR(128); obligatorio
options	JSON; obligatorio
secret_reference	VARCHAR(240); obligatorio
config_hash	VARCHAR(64); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

- Restricción: ForeignKeyConstraint(connection_id)
- Restricción: PrimaryKeyConstraint(id)
- Restricción: UniqueConstraint(connection_id,version)
- Índice: ix_external_connection_versions_connection_id (connection_id)
- Índice: ix_external_connection_versions_organization_id (organization_id)

external_connections

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(160); obligatorio
source_type	VARCHAR(30); obligatorio
enabled	BOOLEAN; obligatorio
deleted	BOOLEAN; obligatorio
version	INTEGER; obligatorio
last_test_status	VARCHAR(20); obligatorio
last_test_at	DATETIME; nullable
last_test_message	VARCHAR(240); obligatorio
updated_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Índice: ix_external_connections_organization_id (organization_id)

external_identities

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
user_id	VARCHAR(64); obligatorio; FK users.id
provider	VARCHAR(30); obligatorio
issuer	VARCHAR(400); obligatorio
subject	VARCHAR(255); obligatorio
email_at_link	VARCHAR(200); obligatorio
linked_at	DATETIME; obligatorio
last_login_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(provider,issuer,subject)

Restricción: UniqueConstraint(user_id,provider)

Índice: ix_external_identities_organization_id (organization_id)

Índice: ix_external_identities_user_id (user_id)

findings

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
run_id	VARCHAR(64); obligatorio; FK runs.id
title	VARCHAR(240); obligatorio
code	VARCHAR(80); obligatorio
severity	VARCHAR(20); obligatorio
fingerprint	VARCHAR(64); obligatorio
details	JSON; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(run_id,fingerprint)

Índice: ix_findings_organization_id (organization_id)

Índice: ix_findings_run_id (run_id)

idempotency_keys

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
route	VARCHAR(180); obligatorio
key	VARCHAR(128); obligatorio
request_hash	VARCHAR(64); obligatorio
run_id	VARCHAR(64); obligatorio; FK runs.id
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,route,key)

Índice: ix_idempotency_keys_organization_id (organization_id)

internal_notifications

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
event_id	VARCHAR(64); obligatorio; FK outbox_events.id
recipient_user_id	VARCHAR(64); obligatorio; FK users.id
module	VARCHAR(20); obligatorio
origin	VARCHAR(20); obligatorio
status	VARCHAR(30); obligatorio
decision	VARCHAR(40); nullable
description	VARCHAR(500); obligatorio
resource_type	VARCHAR(30); obligatorio
resource_id	VARCHAR(64); obligatorio
detail_url	VARCHAR(200); obligatorio
read_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(event_id)

Restricción: ForeignKeyConstraint(recipient_user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(event_id,recipient_user_id)

Índice: ix_internal_notifications_organization_id (organization_id)

Índice: ix_notification_inbox (organization_id, recipient_user_id, read_at, created_at)

jobs

Evolucionada en 0.7.0

Campo	Tipo, nulabilidad y relación
run_id	VARCHAR(64); nullable; FK runs.id
acquisition_id	VARCHAR(64); nullable; FK acquisition_runs.id
status	VARCHAR(20); obligatorio
lane	VARCHAR(20); obligatorio
attempts	INTEGER; obligatorio
lease_owner	VARCHAR(64); nullable
lease_until	DATETIME; nullable
last_error	TEXT; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: (run_id IS NOT NULL AND acquisition_id IS NULL AND lane IN ('DEFAULT','DELIVERY')) OR (run_id IS NULL AND acquisition_id IS NOT NULL AND lane = 'ACQUISITION')

Restricción: ForeignKeyConstraint(acquisition_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(acquisition_id)

Restricción: UniqueConstraint(run_id)

Índice: ix_jobs_lane (lane)

Índice: ix_jobs_lane_status_created_at (lane, status, created_at)

Índice: ix_jobs_organization_id (organization_id)

Índice: ix_jobs_status (status)

metric_history

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
monitor_id	VARCHAR(64); obligatorio; FK configurations.id
run_id	VARCHAR(64); obligatorio; FK runs.id
metric_key	VARCHAR(200); obligatorio
dimensions	JSON; obligatorio
dimension_hash	VARCHAR(64); obligatorio
numeric_value	JSON; nullable
method	VARCHAR(60); obligatorio
metric_definition_version	INTEGER; obligatorio
observed_at	DATETIME; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(monitor_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(run_id,metric_key,dimension_hash)

Índice: ix_metric_history_metric_key (metric_key)

Índice: ix_metric_history_monitor_id (monitor_id)

Índice: ix_metric_history_observed_at (observed_at)

Índice: ix_metric_history_organization_id (organization_id)

Índice: ix_metric_history_run_id (run_id)

monitor_occurrences

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
schedule_id	VARCHAR(64); obligatorio; FK monitor_schedules.id
schedule_version_id	VARCHAR(64); obligatorio; FK monitor_schedule_versions.id
monitor_id	VARCHAR(64); obligatorio; FK configurations.id
dataset_version_id	VARCHAR(64); nullable; FK dataset_versions.id
run_id	VARCHAR(64); nullable; FK runs.id
planned_at	DATETIME; obligatorio
dispatched_at	DATETIME; obligatorio
status	VARCHAR(30); obligatorio
reason_code	VARCHAR(60); nullable
coalesced_intervals	INTEGER; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(dataset_version_id)

Restricción: ForeignKeyConstraint(monitor_id)

Restricción: ForeignKeyConstraint(run_id)

Restricción: ForeignKeyConstraint(schedule_id)

Restricción: ForeignKeyConstraint(schedule_version_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(run_id)

Restricción: UniqueConstraint(schedule_id,planned_at)

Índice: ix_monitor_occurrences_monitor_id (monitor_id)

Índice: ix_monitor_occurrences_organization_id (organization_id)

Índice: ix_monitor_occurrences_schedule_id (schedule_id)

monitor_schedule_versions

Evolucionada en 0.7.0

Campo	Tipo, nulabilidad y relación
schedule_id	VARCHAR(64); obligatorio; FK monitor_schedules.id
version	INTEGER; obligatorio
interval_seconds	INTEGER; obligatorio
enabled	BOOLEAN; obligatorio
starts_at	DATETIME; obligatorio
actor_id	VARCHAR(64); obligatorio
responsible_user_id	VARCHAR(64); nullable; FK users.id

Campo	Tipo, nulabilidad y relación
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(responsible_user_id)

Restricción: ForeignKeyConstraint(schedule_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(schedule_id,version)

Índice: ix_monitor_schedule_versions_organization_id (organization_id)

Índice: ix_monitor_schedule_versions_schedule_id (schedule_id)

monitor_schedules

Evolucionada en 0.7.0

Campo	Tipo, nulabilidad y relación
monitor_id	VARCHAR(64); obligatorio; FK configurations.id
version	INTEGER; obligatorio
enabled	BOOLEAN; obligatorio
next_run_at	DATETIME; obligatorio
updated_at	DATETIME; obligatorio
legacy_enabled_before_identity	BOOLEAN; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(monitor_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(monitor_id)

Índice: ix_monitor_schedules_next_run_at (next_run_at)

Índice: ix_monitor_schedules_organization_id (organization_id)

notification_deliveries

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
event_type	VARCHAR(80); obligatorio
template_key	VARCHAR(80); obligatorio
channel	VARCHAR(20); obligatorio
recipient_type	VARCHAR(20); obligatorio
recipient_user_id	VARCHAR(64); obligatorio; FK users.id
recipient_email_snapshot	VARCHAR(200); obligatorio

Campo	Tipo, nulabilidad y relación
provider_key	VARCHAR(40); obligatorio
attempt_number	INTEGER; obligatorio
status	VARCHAR(20); obligatorio
error_code	VARCHAR(80); nullable
sent_at	DATETIME; nullable
failed_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(recipient_user_id)

Restricción: PrimaryKeyConstraint(id)

Índice: ix_notification_deliveries_organization_id (organization_id)

Índice: ix_notification_deliveries_recipient_user_id (recipient_user_id)

oidc_login_attempts

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
state_hash	VARCHAR(64); obligatorio; PK
provider	VARCHAR(30); obligatorio
browser_hash	VARCHAR(64); obligatorio
nonce	VARCHAR(100); obligatorio
code_verifier	VARCHAR(128); obligatorio
expires_at	DATETIME; obligatorio
consumed_at	DATETIME; nullable

Restricción: PrimaryKeyConstraint(state_hash)

outbox_events

Nueva en 0.7.0

Campo	Tipo, nulabilidad y relación
dedupe_key	VARCHAR(220); obligatorio
event_type	VARCHAR(60); obligatorio
aggregate_type	VARCHAR(30); obligatorio
aggregate_id	VARCHAR(64); obligatorio
module	VARCHAR(20); obligatorio
payload	JSON; obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio

Campo	Tipo, nulabilidad y relación
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,dedupe_key)

Índice: ix_outbox_events_organization_id (organization_id)

role_permissions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
role_id	VARCHAR(64); obligatorio; PK; FK roles.id
permission_code	VARCHAR(80); obligatorio; PK

Restricción: ForeignKeyConstraint(role_id)

Restricción: PrimaryKeyConstraint(role_id,permission_code)

roles

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(120); obligatorio
normalized_name	VARCHAR(120); obligatorio
description	VARCHAR(2000); obligatorio
system_key	VARCHAR(40); nullable
active	BOOLEAN; obligatorio
deleted	BOOLEAN; obligatorio
version	INTEGER; obligatorio
updated_at	DATETIME; obligatorio
deleted_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(organization_id,normalized_name)

Restricción: UniqueConstraint(organization_id,system_key)

Índice: ix_roles_organization_id (organization_id)

runs

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
module	VARCHAR(20); obligatorio
name	VARCHAR(160); obligatorio
status	VARCHAR(30); obligatorio
decision	VARCHAR(40); nullable
config_id	VARCHAR(64); obligatorio; FK configurations.id
dataset_version_id	VARCHAR(64); obligatorio; FK dataset_versions.id
target_version_id	VARCHAR(64); nullable; FK dataset_versions.id
initiated_by	VARCHAR(64); obligatorio
initiated_by_type	VARCHAR(20); obligatorio
initiated_by_id	VARCHAR(64); obligatorio
initiated_by_legacy	BOOLEAN; obligatorio
progress_stage	VARCHAR(50); obligatorio
progress_percent	INTEGER; obligatorio
metrics	JSON; obligatorio
execution_plan	JSON; obligatorio
result_path	TEXT; nullable
evidence_path	TEXT; nullable
output_version_id	VARCHAR(64); nullable
error	TEXT; nullable
cancel_requested	BOOLEAN; obligatorio
started_at	DATETIME; nullable
finished_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(config_id)

Restricción: ForeignKeyConstraint(dataset_version_id)

Restricción: ForeignKeyConstraint(target_version_id)

Restricción: PrimaryKeyConstraint(id)

Índice: ix_runs_module (module)

Índice: ix_runs_organization_id (organization_id)

Índice: ix_runs_status (status)

sessions

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
token_hash	VARCHAR(64); obligatorio
user_id	VARCHAR(64); obligatorio; FK users.id
csrf_token	VARCHAR(100); obligatorio
expires_at	DATETIME; obligatorio
authentication_method	VARCHAR(30); obligatorio
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio
created_at	DATETIME; obligatorio

Restricción: ForeignKeyConstraint(user_id)

Restricción: PrimaryKeyConstraint(id)

Restricción: UniqueConstraint(token_hash)

Índice: ix_sessions_organization_id (organization_id)

users

Estructura histórica preservada

Campo	Tipo, nulabilidad y relación
name	VARCHAR(200); obligatorio
email	VARCHAR(200); obligatorio
role	VARCHAR(40); obligatorio
role_id	VARCHAR(64); obligatorio; FK roles.id
username	VARCHAR(80); obligatorio
first_name	VARCHAR(100); nullable
last_name	VARCHAR(100); nullable
password_hash	TEXT; obligatorio
active	BOOLEAN; obligatorio
deleted	BOOLEAN; obligatorio
deleted_at	DATETIME; nullable
must_change_password	BOOLEAN; obligatorio
temporary_password_expires_at	DATETIME; nullable
last_login_at	DATETIME; nullable
version	INTEGER; obligatorio
updated_at	DATETIME; obligatorio
password_changed_at	DATETIME; nullable
id	VARCHAR(64); obligatorio; PK
organization_id	VARCHAR(64); obligatorio

Campo	Tipo, nulabilidad y relación
created_at	DATETIME; obligatorio
Restricción: ForeignKeyConstraint(role_id)	
Restricción: PrimaryKeyConstraint(id)	
Restricción: UniqueConstraint(email)	
Restricción: UniqueConstraint(username)	
Índice: ix_users_organization_id (organization_id)	
Índice: ix_users_role_id (role_id)	

18. API, DTOs y errores

OpenAPI se genera desde la aplicación 0.7.0 y se versiona en backend/openapi.json. La API mantiene /api/v1, cookie HttpOnly, CSRF, IDs opacos y fechas ISO UTC. Las rutas legacy conservan su contrato; las nuevas operaciones asíncronas utilizan endpoints explícitos y 202. Health/ready continúa pública. Todo endpoint protegido debe estar en la matriz; una ruta no inventariada se deniega.

Adquisición y transferencia

POST /datasets/uploads/stage?filename=... recibe bytes con MIME octet-stream y devuelve 201 con upload id/hash/tamaño/formato/expiry. GET /datasets/uploads/{id}/inspect acepta opciones de lectura y muestra columnas/opciones admitidas. POST /datasets/{id}/acquisitions recibe upload_id, reader_options y column_overrides; POST /connections/{id}/acquisitions recibe selección SQL y nombre de dataset. Ambas registran 202; POST /datasets/{id}/acquisitions/refresh registra otra intención. GET /acquisitions admite filtros/paginación; GET /acquisitions/{id} y POST /cancel permiten seguimiento/acción autorizada. El DTO expone status/stage, cantidades observadas/totales opcionales, fechas, attempts, error sanitizado y output_version_id.

Ejecución y preflight

Intake/Recon/Sentinel aceptan requested_engine AUTO/POLARS/PYSPARK. El plan preview usa las mismas identidades y referencias de ejecución. GET /system/engines conserva worker DEFAULT por compatibilidad y añade workers ACQUISITION/DELIVERY, components y límites efectivos. POST /delivery/validations recibe un draft y 202; GET lista/detalle son personales; POST cancel pide cancelación cooperativa. Una configuración grande se publica con validation_run_id, unido al draft exacto.

Automatizaciones y bandeja

GET /delivery/automations y /{id}/occurrences listan revisiones/estado/historia. POST /delivery/automations y /{id}/versions conservan versiones; POST /{id}/dispatch registra ejecución deliberada, con repeat=true explícito cuando se necesita; el default es false. La política persistida settings.repeat_versions controla la repetición en futuras ocurrencias y también está deshabilitada por defecto. POST /delivery/runs/{id}/resume-target requiere review_id y nota, no crea otra Run. GET /notifications/inbox expone colección filtrada por usuario/org/permisos; GET /notifications/unread-count y POST /inbox/{id}/read /read-all persisten lectura. Los endpoints deprecated /notifications/status y /deliveries siguen HISTORICAL_ONLY.

Ejemplo sanitizado

```
POST /api/v1/datasets/<dataset>/acquisitions
Idempotency-Key: <clave de la solicitud>
X-CSRF-Token: <token de la sesión>
{"upload_id":"<upload>","reader_options":{"delimiter":";"}}
HTTP 202: {"id":"<acquisition>","status":"QUEUED"}
```

Los tokens representados son placeholders y no forman parte de evidencia. Errores usan {error:{code,message,details?,request_id}}. Una excepción inesperada registra sólo tipo/referencia en el log, sin excepción cruda del driver. Los códigos funcionales de recursos/cancelación/lease conservan significado específico.

Inventario ejecutable completo

Método	Ruta	Operación
GET	/health	Health
GET	/health/ready	Ready
GET	/api/v1/health	Health
GET	/api/v1/health/ready	Ready
POST	/api/v1/auth/demo	Demo Login
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/first-login/change-password	First Login Password
GET	/api/v1/me	Me
POST	/api/v1/auth/logout	Logout
GET	/api/v1/datasets	Datasets
POST	/api/v1/datasets	New Dataset
POST	/api/v1/datasets/uploads/inspect	Inspect Dataset Upload
GET	/api/v1/datasets/{dataset_id}	Dataset Detail
GET	/api/v1/datasets/{dataset_id}/schema	Dataset Schema
POST	/api/v1/datasets/{dataset_id}/versions/upload	Upload Version
GET	/api/v1/dataset-versions/{version_id}/profile	Profile
GET	/api/v1/monitors	Configurations
POST	/api/v1/monitors	Monitor
GET	/api/v1/recon/controls	Configurations
POST	/api/v1/recon/controls	Recon Control
GET	/api/v1/intake/contracts	Configurations
POST	/api/v1/intake/contracts	Intake Contract
POST	/api/v1/intake/runs	Intake Run
POST	/api/v1/recon/runs	Recon Run
POST	/api/v1/monitors/{monitor_id}/runs	Monitor Run
GET	/api/v1/runs	Runs
GET	/api/v1/runs/{run_id}	Run Detail
POST	/api/v1/runs/{run_id}/cancel	Cancel Run
GET	/api/v1/recon/runs/{run_id}/results	Results
GET	/api/v1/intake/runs/{run_id}/errors	Results
GET	/api/v1/runs/{run_id}/results	Results
GET	/api/v1/runs/{run_id}/evidence	Evidence
GET	/api/v1/artifacts/{artifact_id}/download	Artifact Download
GET	/api/v1/runs/{run_id}/export.xlsx	Export Excel
GET	/api/v1/runs/{run_id}/export.csv	Export Csv
GET	/api/v1/monitors/{monitor_id}/metrics	Monitor Metrics

Método	Ruta	Operación
GET	/api/v1/findings	Findings
POST	/api/v1/findings/{finding_id}/exceptions	Finding Exception
GET	/api/v1/audit-events	Audit Events
GET	/api/v1/rules	Rules
GET	/api/v1/system/engines	Engines
GET	/api/v1/dashboard	Dashboard
POST	/api/v1/monitors/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/recon/controls/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/intake/contracts/{configuration_id}/versions	Publish Configuration Version
GET	/api/v1/runs/{run_id}/execution-plan	Run Execution Plan
GET	/api/v1/runs/{run_id}/diagnostics	Run Diagnostics
POST	/api/v1/execution-plans/preview	Preview Execution
GET	/api/v1/connections	List Connections
POST	/api/v1/connections	Create Connection
POST	/api/v1/connections/test	Test Draft
GET	/api/v1/connections/{connection_id}	Connection Detail
PATCH	/api/v1/connections/{connection_id}	Update Connection
DELETE	/api/v1/connections/{connection_id}	Delete Connection
POST	/api/v1/connections/{connection_id}/test	Test Connection
GET	/api/v1/connections/{connection_id}/schemas	Schemas
GET	/api/v1/connections/{connection_id}/objects	Objects
GET	/api/v1/connections/{connection_id}/preview	Preview
POST	/api/v1/connections/{connection_id}/datasets	Register Source Dataset
POST	/api/v1/datasets/{dataset_id}/refresh-source	Refresh Source
GET	/api/v1/delivery/destinations	List Destinations
POST	/api/v1/delivery/destinations	Create Destination
POST	/api/v1/delivery/destinations/test	Test Destination Draft
GET	/api/v1/delivery/destinations/{destination_id}	Destination Detail
PATCH	/api/v1/delivery/destinations/{destination_id}	Update Destination
DELETE	/api/v1/delivery/destinations/{destination_id}	Delete Destination
POST	/api/v1/delivery/destinations/{destination_id}/test	Test Destination
GET	/api/v1/delivery/destinations/{destination_id}/schemas	Destination Schemas
GET	/api/v1/delivery/destinations/{destination_id}/tables	Destination Tables
GET	/api/v1/delivery/destinations/{destination_id}/table-metadata	Destination Table Metadata
GET	/api/v1/delivery/destinations/{destination_id}/target-policy	Destination Target Policy
POST	/api/v1/delivery/preview	Delivery Preview
POST	/api/v1/delivery/preflight	Delivery Preflight

Método	Ruta	Operación
GET	/api/v1/delivery/configurations	List Delivery Configurations
POST	/api/v1/delivery/configurations	Publish Delivery Configuration
POST	/api/v1/delivery/configurations/{configuration_id}/versions	Publish Delivery Configuration Version
GET	/api/v1/delivery/runs	List Delivery Runs
POST	/api/v1/delivery/runs	Create Delivery Run
GET	/api/v1/delivery/runs/{run_id}/attempts	Delivery Attempts
GET	/api/v1/delivery/runs/{run_id}/receipt	Delivery Receipt
POST	/api/v1/delivery/runs/{run_id}/repair-evidence	Repair Evidence
GET	/api/v1/delivery/runs/{run_id}/reviews	Delivery Reviews
POST	/api/v1/delivery/runs/{run_id}/reviews	Review Delivery Outcome
GET	/api/v1/roles/permissions	Permission Catalog
GET	/api/v1/users/roles	Roles
GET	/api/v1/roles	Roles
POST	/api/v1/roles	Create Role
GET	/api/v1/roles/{role_id}	Role Detail
PATCH	/api/v1/roles/{role_id}	Edit Role
DELETE	/api/v1/roles/{role_id}	Delete Role
GET	/api/v1/users	Users
POST	/api/v1/users	Create User
GET	/api/v1/users/{user_id}	User Detail
PATCH	/api/v1/users/{user_id}	Edit User
DELETE	/api/v1/users/{user_id}	Delete User
POST	/api/v1/users/{user_id}/reset-password	Regenerate Credentials (legacy)
POST	/api/v1/users/{user_id}/resend-credentials	Regenerate Credentials (legacy)
POST	/api/v1/users/{user_id}/regenerate-credentials	Regenerate Credentials
DELETE	/api/v1/users/{user_id}/external-identities/{identity_id}	Unlink Identity
GET	/api/v1/notifications/status	Notification Status (legacy)
GET	/api/v1/notifications/deliveries	Notification Deliveries (legacy)
GET	/api/v1/exceptions	Exceptions
GET	/api/v1/exceptions/assignees	Assignees
GET	/api/v1/exceptions/{case_id}	Detail
PATCH	/api/v1/exceptions/{case_id}	Update Case
POST	/api/v1/exceptions/{case_id}/validate	Validate
POST	/api/v1/exceptions/{case_id}/comments	Comment
POST	/api/v1/exceptions/{case_id}/attachments	Upload Attachment
GET	/api/v1/exceptions/{case_id}/attachments/{attachment_id}/download	Attachment Download
GET	/api/v1/monitors/{monitor_id}/schedule	Get Schedule

Método	Ruta	Operación
POST	/api/v1/monitors/{monitor_id}/schedule	Configure Schedule
GET	/api/v1/monitors/{monitor_id}/occurrences	Occurrences
GET	/api/v1/monitors/{monitor_id}/series	Metric Series
GET	/api/v1/monitors/{monitor_id}/alerts	Monitor Alerts
GET	/api/v1/auth/providers	Providers
GET	/api/v1/auth/sso/{provider}/start	Start
GET	/api/v1/auth/sso/{provider}/callback	Callback
POST	/api/v1/datasets/uploads/stage	Stage File
GET	/api/v1/datasets/uploads/{upload_id}/inspect	Inspect Staged
POST	/api/v1/datasets/{dataset_id}/acquisitions	Acquire File
POST	/api/v1/connections/{connection_id}/acquisitions	Acquire Source
POST	/api/v1/datasets/{dataset_id}/acquisitions/refresh	Refresh Source
GET	/api/v1/acquisitions	Acquisitions
GET	/api/v1/acquisitions/{acquisition_id}	Acquisition Detail
POST	/api/v1/acquisitions/{acquisition_id}/cancel	Cancel
GET	/api/v1/delivery/automations	List Automations
POST	/api/v1/delivery/automations	Create Automation
GET	/api/v1/delivery/automations/{automation_id}	Automation Detail
POST	/api/v1/delivery/automations/{automation_id}/versions	Automation Version
GET	/api/v1/delivery/automations/{automation_id}/occurrences	Automation Occurrences
POST	/api/v1/delivery/automations/{automation_id}/dispatch	Manual Dispatch
POST	/api/v1/delivery/runs/{run_id}/resume-target	Resume Target
GET	/api/v1/notifications/inbox	Inbox
GET	/api/v1/notifications/unread-count	Unread Count
POST	/api/v1/notifications/inbox/read-all	Read All
POST	/api/v1/notifications/inbox/{notification_id}/read	Read Notification
GET	/api/v1/delivery/validations	List Validations
POST	/api/v1/delivery/validations	Register Validation
GET	/api/v1/delivery/validations/{run_id}	Read Validation
POST	/api/v1/delivery/validations/{run_id}/cancel	Cancel Validation

19. Contratos HTTP, identidad y permisos

RBAC dinámico continúa resolviendo Role/RolePermission de la cuenta vigente. Administrator es un rol de sistema protegido y obtiene los códigos del catálogo; no obtiene acceso a inbox/validaciones ajenas. Users/roles manage son no delegables. La clausura transitiva se verifica en backend, además de botones de UI.

delivery:schedule depende de delivery:read/execute/configure y las dependencias transitivas de datasets/destinations. No se resembran grants de roles personalizados existentes. Los defaults ampliados aplican únicamente a organizaciones nuevas. Asignar schedule no equivale a autorizar OVERWRITE o creación/alteración de target: se siguen requiriendo permisos separados para la estrategia.

Adquisición requiere datasets:write para upload y connections:use para SQL. La entidad y upload son de usuario/organización, y al leer se revalida conexión activa/revisión y permisos. Automatización captura responsible_user_id real; despacho y pre-STARTED consultan cuenta/rol activos y destino vigente. SYSTEM describe el trigger y no recibe notificaciones ni permiso por sí mismo.

Notificaciones exige notifications:read y el permiso vigente del módulo enlazado. Lista, contador, read y read-all aplican el mismo filtro. Revocar permiso oculta el recurso y su aviso; cambiar de rol no conserva acceso por un token histórico. La lectura no expone una inbox ajena aunque exista un id conocido o Administrator.

Sesión local, primer acceso obligatorio, temporales de una presentación, bajas lógicas y SSO Microsoft/Google opcional se conservan. SSO permanece deshabilitado por defecto; no se reinstala SMTP ni Mailpit. Las cookies, contraseñas temporales y tokens no se capturan en evidencia pública, pantallas, vídeos o traces.

La matriz exhaustiva vigente se reproduce en el capítulo 34 y se genera desde permissions.py. Los vínculos de recursos aplican autorización además del permiso de ruta; esconder un botón no constituye control del servidor.

20. Parámetros, operación, backup y upgrade

Parámetros efectivos y unidades

Los enteros se validan por rango; API verifica configuración al arrancar y workers verifican el conjunto que usan. Bytes son unidades binarias; filas son registros; seconds son segundos; MiB de Spark se convierten explícitamente a bytes. Reiniciar/recrear servicios afectados aplica cambios; los jobs capturan límites efectivos y plan para que un cambio posterior no reinterprete su identidad.

Variable	Default, unidad, rango y control
TRACKVANCE_ACQUISITION_MAX_UPLOAD_BYTES	1073741824 bytes. 1 KiB..5 GiB. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_MAX_ROWS	5000000 registros. 1..100.000.000. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_MAX_OBSERVED_BYTES	2147483648 bytes UTF-8 observados. 1 KiB..20 GiB. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_BATCH_ROWS	5000 registros/lote. 1..50.000. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.

Variable	Default, unidad, rango y control
TRACKVANCE_ACQUISITION_BATCH_BYTES	8388608 bytes/lote. 64 KiB..64 MiB. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_BOUNDED_FORMAT_BYTES	10485760 bytes JSON no lineal/XLSX. 1 KiB..64 MiB. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_BOUNDED_FORMAT_ROWS	100000 filas JSON no lineal/XLSX. 1..100.000. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_MEMORY_BYTES	268435456 bytes del motor de perfil DuckDB. 64 MiB..4 GiB. API / acquisition-worker. Cota del motor analítico congelada en AcquisitionRun; no es límite de RSS del proceso ni de cgroup. Buffers del lector/writer, Python y file cache se miden aparte y el contenedor conserva su límite declarado. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_MIN_FREE_BYTES	536870912 bytes reserva de disco. 16 MiB..20 GiB. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_TIMEOUT_SECONDS	1800 segundos. 30..86.400. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_ACQUISITION_UPLOAD_TTL_SECONDS	86400 segundos. 300..604.800. API / acquisition-worker. Cota aplicada durante recepción, lectura, perfil o cleanup según el campo; congelada en AcquisitionRun. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_BATCH_ROWS	1000 filas/lote. 1..10.000. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_SCAN_MEMORY_BYTES	268435456 bytes scan DuckDB. 64 MiB..2 GiB. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_MAX_ROWS	5000000 filas. 1..100.000.000. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_RESERVE_DISK_BYTES	536870912 bytes reserva de disco. 16 MiB..20 GiB. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_SYNCHRONOUS_ROWS	100000 filas endpoint síncrono. 1..100.000. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_DELIVERY_PREPARATION_TIMEOUT_SECONDS	1800 segundos. 30..86.400. API / delivery-worker. Scan completo, spool/keys en disco y control cooperativo; no cambia garantías de commit remoto. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.

Variable	Default, unidad, rango y control
TRACKVANCE_SPARK_MASTER	local[2] URI/modo. local[K] K>=1 <=TOTAL_CORES; spark://host:port. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_DRIVER_MEMORY_MB	768 MiB JVM. 512..16.384. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_EXECUTOR_MEMORY_MB	768 MiB JVM/executor. 512..16.384. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_EXECUTOR_CORES	1 CPU/executor. 1..16. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_TOTAL_CORES	2 CPU total Spark. 1..64. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_PARTITIONS	4 particiones. 1..256. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_BATCH_ROWS	2048 filas. 1..10.000. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_BATCH_BYTES	8388608 bytes. 64 KiB..64 MiB. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_MAX_RECORD_BYTES	65536 bytes/fila. 1 KiB..1 MiB. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_MAX_GROUP_ROWS	10000 filas/grupo Recon. 1..100.000. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_MAX_GROUP_BYTES	16777216 bytes/grupo Recon. 64 KiB..64 MiB. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_MAX_RESULT_BYTES	16777216 bytes resultado driver. 1 MiB..64 MiB. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SPARK_MEMORY_BUDGET_BYTES	2147483648 bytes estimados. Entero positivo; Standalone overlay usa 3 GiB. API planner / DEFAULT / executors. Memoria JVM, recursos y cotas efectivas de fila/lote/grupo; budget estima y cgroup impone límite del contenedor. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.

Variable	Default, unidad, rango y control
TRACKVANCE_WORKER_MEMORY_SOFT_BYTES	536870912 bytes. Positivo. API / DEFAULT planner. Presupuesto estimado Polars; no límite duro de RSS. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_TEMP_MIN_FREE_BYTES	16777216 bytes. Positivo; Compose usa 536870912. API / DEFAULT. Reserva planner previa y control durante ejecución. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_RUN_TIMEOUT_SECONDS	300 segundos. Positivo; Compose usa 1800. DEFAULT. Deadline cooperativo; no terminación arbitraria de un commit SQL. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SCHEDULER_POLL_SECONDS	5 segundos. 1..60. scheduler. Intervalo aplicado al bucle ligero. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_SCHEDULER_BATCH_SIZE	100 ocurrencias/tick. 1..1.000. scheduler. Cantidad acotada por tick; no paralelismo de jobs. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_EVENT_LEASE_SECONDS	300 segundos. 60..3.600. events-consumers. Lease CAS, heartbeat y fence final. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.
TRACKVANCE_EVENT_POLL_SECONDS	1 segundo. 1..60. events-consumers. Intervalo del bucle independiente. Recrear/reiniciar servicios afectados; jobs conservan plan/límites capturados.

Cotas fijas y controles complementarios

Cota fija	Contrato y alcance
1	Concurrencia por proceso worker: 1 Job; múltiples réplicas requieren presupuesto adicional.
2	MAX_UPLOAD_BYTES = 10 MiB y TRACKVANCE_MAX_ROWS = 100000 permanecen en las rutas síncronas legacy.
3	Celdas de adquisición: 64 KiB; columnas: 100. El lote de Delivery admite 8 MiB observados además de BATCH_ROWS.
4	PreparedRows: binding, hash y conteos completos; no hay commit por lote.
5	Reintentos del consumidor de eventos: máximo 5, persistidos; las reservas y leases no autorizan replay de UNKNOWN.
6	Página de resultados: hasta 100.000 filas y 16 MiB UTF-8 antes de decodificar; el exceso produce HTTP 422 RESULT_PAGE_BYTE_LIMIT, sin truncar. Muestra de perfil: hasta 20 filas y 8 MiB de JSON UTF-8.

La cota legacy de 10 MiB/100.000 filas permanece en endpoints síncronos y formas JSON/XLSX limitadas. El perfil asíncrono default admite archivo 1 GiB, 5 millones de filas y 2 GiB de bytes observados. Los benchmarks usan perfil aislado declarado cuando el formato/overhead real excede el tamaño nominal del escalón. Eso no certifica 2 GiB ni cambia silenciosamente .env de la instalación real.

Backup nativo state 6

backup-manifest.json conserva schema 2; state.json evoluciona a schema 6 con el inventario exacto de 42 tablas, artifacts y relaciones. El verificador reconoce la revisión real de runtime, no sólo el script copiado. Paths, SHA/tamaños, descriptor y todas las partes se verifican; archivos no declarados o symlinks se rechazan. Metadata y los cinco volúmenes de archivos/secretos/claves se respaldan de manera coordinada. .env se conserva aparte y privado.

Antes de leer las filas, physical_schema_guard contrasta el catálogo SQL físico con Base.metadata de la versión instalada: tablas y columnas exactas, y claves foráneas completas con sus pares, agrupación y acciones. Sólo alembic_version queda fuera del modelo de aplicación; una columna o FK desconocida causa FAIL. No compara nombres de constraints ni grafías de tipos entre motores. Inspecciona el namespace de aplicación y los schemas explícitos del modelo; una base de negocio co-alojada no se incorpora implícitamente al backup de Trackvance. El verificador y este helper se copian juntos y se validan por SHA antes de

ejecutarse. El guard genérico también funciona sobre el ORM histórico auténtico sin modificar sus scripts; la actualización guarda primero el dump original, verifica ese catálogo y calcula la huella antes de iniciar el API antiguo.

El backup quiesce API, todos los workers, scheduler y consumidores; rechaza Jobs/Runs/Acquisition activos para evitar una foto inconsistentemente publicada. Un outbox pending durable puede respaldarse y reanudarse. Los secretos fuente quedan separados de los de destino y se valida que puedan descifrarse tras restore sin imprimirlos. Las fuentes/destinos restaurados de pruebas no se apuntan a servidores del usuario; los dispatchers permanecen detenidos mientras se verifica.

El backfill de artefactos conserva la compatibilidad de Intake, ReconOps y Sentinel históricos. Excluye DELIVERY y DELIVERY_PREFLIGHT: sus identidades de entrada y evidencia ya se fijan al crear/publicar la operación. Un preflight conserva SHA canónico y hash del draft en execution_plan; no recibe enlaces genéricos RUN_INPUT al reiniciar o restaurar. La primera restauración de volumen detectó tres enlaces adicionales de ese tipo y seis relaciones adicionales, pese a igualdad de las otras 41 tablas, archivos y secretos. Se corrigió el hook, sin borrar linaje existente ni ignorar esas diferencias. Tres regresiones focales de API con preflight persistido QUEUED, SUCCESS y CANCELLED comparan todos los valores de las 42 tablas tras dos backfills consecutivos.

La restauración física posterior vuelve a importar el dump original sin editar filas ni redefinir la huella esperada. Verifica 42 tablas/0015, 5.643 artefactos, cuatro secretos fuente y seis secretos Delivery; la huella origen/restaurada es ff709e8fe022e26b68f3ef65122412c41a5ff660accf99fc85219d1473c893f6. Multipartes, bytes históricos y comparación exacta PASS; los procesos automáticos permanecen detenidos hasta STOPPED_VERIFIED. La fase SQL/restauración/verificación posterior al fix tardó 150,541 s; no es el tiempo total del backup ni de los dos intentos fallidos conservados. El escaneo privado revisó 5.666 archivos y el dump SQL descomprimido, sin publicar su contenido. La instalación principal quedó intacta.

Compatibilidad estricta 0.6.1→0.7.0

La huella legacy state 5 conserva tablas, actores, versiones, hashes, profiles, configs, Run/Attempt y linaje. La comparación proyecta sólo campos nuevos documentados y defaults esperados, incluido enabled legacy pausado por identidad Sentinel no verificable. Exige tablas nuevas vacías antes de despachar. No ignora indiscriminadamente diferencias ni genera outbox al insertar historia restaurada. El listener observa transiciones actuales, no un escaneo/replay de db.new histórico.

La prueba de upgrade usa código auténtico 6fac26b y datos sintéticos creados con esa versión, backup compatible y restore en un proyecto nuevo. La prueba nativa incluye multipartes, usuarios/roles, fuentes/destinos/keys, schedules e inbox. Los resultados ejecutados y hashes se registran en evidencia, separados del upgrade posterior de la instalación de trabajo.

La ejecución auténtica aprobó el ciclo completo en 114,609 s: origen destruido antes de restaurar, proyección de las 31 tablas idéntica y once tablas nuevas vacías antes de activar procesos. Conservó 29 artefactos, un secreto fuente, un secreto destino y una notificación histórica; no generó notificaciones retroactivas. La huella histórica origen/proyección es f0b0e536e20da6893b68eccfb163534e9d81d3f40f017e327b873fe1e86f81d9. Las credenciales y conexiones restauradas se comprobaron después de la comparación. Un fallo inicial de build de 30,155 s queda retenido sin causa definitiva atribuida; el siguiente ciclo separado de build/arranque pasó, con diagnósticos acotados.

La revisión intermedia 191ff280 completó además las cuatro etapas de backup de GitHub Actions: operacional 0.7.0 y fuentes auténticas 0.5.1, 0.6.0 y 0.6.1. El paso operacional tomó 133 s según el reloj GitHub de resolución 1 s; los runners auténticos midieron 149,785 / 150,416 / 124,619 s respectivamente. El fixture SQL antiguo de 0.6.0 que falló en ba183 se corrigió y el reintento conservó su notificación y secreto, con huella histórica igual y sin replay. ci-191-recovery.json conserva resultados saneados y los enlaces anteriores; este PASS intermedio no sustituye los jobs del HEAD documental definitivo.

Actualización controlada de la instalación principal

La instalación se identifica por etiquetas/configuración, puerto 3100 y proyecto trackvance-certification, conservando sus volúmenes y PostgreSQL 16. Durante desarrollo ningún fixture/benchmark/fault injection usa sus recursos. Los guards comparan imagen/estado/restart/mounts antes y después de tests; el orden de mounts se normaliza, pero sus identidades/valores se comparan estrictamente.

Después de aprobar código/E2E/recovery/documentos y todos los jobs CI del HEAD final, se pausa nuevo despacho, comprueba ausencia de trabajos activos, respalda consistentemente con herramientas compatibles, registra inventario y verifica el backup. Sólo entonces recrea servicios desde revisión final, aplica 0013..0015 y conserva .env/puerto/origen/restart:no/demo seed false/SSO deshabilitado. No se usa down-v, prune, reset, fixture ni escritura de prueba en el proyecto del usuario.

La verificación final no destructiva comprueba health, Alembic, los tres workers, scheduler/consumidores, storage, API/UI 0.7.0, imágenes/revisión y la preservación estricta state 5→6. Si falla, se usa el procedimiento de recuperación probado, sin downgrade destructivo improvisado. El informe externo identifica backup privado, revisión realmente desplegada y servicios/URL final.

20A. Benchmark reproducible y límites comprobados

Objetivos y representación

El primer objetivo es un millón de registros. El fixture reproducible contiene identificador con ceros iniciales, Decimal fraccionario, región de cardinalidad declarada, valores observados con espacios/Unicode combinante/emoji/multilínea, null y payload de entropía determinista. Se registran columnas, ancho real, cardinalidad, seed, SHA y compresibilidad medida. CSV, NDJSON y Parquet no tienen el mismo tamaño físico; el escalón nominal no sustituye los bytes medidos.

La progresión obligatoria de 100 MiB, 500 MiB y 1 GiB se ejecuta en presupuesto observado, con adquisición, perfil global, procesamiento y Delivery medidos por separado y de extremo a extremo. Memoria de proceso, cgroup, CPU, spill/disco temporal, storage, rows/hashees y resultado remoto se registran. Para 2/5 GiB primero se evalúa RAM/disco/límites y se conserva NOT_RUN_RESOURCE_LIMIT si no hay margen. Un escalón obligatorio sin ejecutar/aprobar impide certificar la release.

Separación entre pruebas de engine e integración

spark_cycle comprueba un millón de registros en Intake/Recon/Sentinel local y Standalone con referencias, duplicados globales y conciliación Decimal no trivial. volume_cycle recorre la API normal stage/confirmación 202, workers, partes, perfil, Intake PySpark, accepted output exacto, automatización, DataSink SQL y bandeja personal. El hash ordenado de valores observados y el conteo del destino verifican toda la población; no basta que la UI muestre una muestra o que la Run diga SUCCESS.

Los informes distinguen kernel JVM, publicación Run/Job y E2E integrado. Standalone registra application IDs/driver+dos executors y memoria por proceso. El mismo host no certifica multinodo. Las mediciones no son un SLA ni un umbral universal para otros anchos, skew, redes, triggers o motores SQL.

Método de medición por etapa y recorrido integral

El primer volume_cycle agrupó recepción HTTP, espera de adquisición y perfil global en un tiempo CSV; se conservan sus cifras sin presentarlas como tiempos separados. El ensayo adicional acquisition_timing_cycle vuelve a recibir las tres fixtures CSV con datasets/targets nuevos. Comprueba igualdad completa del perfil y esquema persistidos, hash de toda la fuente, todos los aceptados y el destino SQL, y preservación de las versiones originales. No modifica targets ni versiones previos.

Observa la etapa durable cada 0.5 s y registra inicio/fin de cada solicitud HTTP. Una transición queda acotada entre la petición anterior y la primera respuesta que muestra la etapa nueva; se publican duración mínima/máxima e incertidumbre, no un tiempo exacto de worker. READING/MATERIALIZING se agrupan porque alternan. PROFILING debe observarse con ventana inferior positiva; una etapa breve no vista no recibe un cero ficticio. Los probes de recursos de perfil deben caer enteros dentro de esa ventana estable.

El reloj integral monotónico empieza antes de recibir CSV y termina después de Intake aprobado, Delivery COMMITTED, hash SQL completo e inbox personal. Incluye las pausas de verificación del harness y por ello no es un SLA de producto. Sus tiempos no se obtienen sumando fases. Cada fase de preflight, Spark y Delivery conserva además tiempo y recursos independientes. CPU se expresa como delta de cgroup entre probes retenidos de la misma identidad; una muestra insuficiente/reset produce null. RSS/cgroup/temp/spill son picos muestreados, no máximos atómicos; memory.peak vitalicio no se usa como pico de fase.

Los tres ensayos CSV adicionales terminaron PASS sobre 191ff2805f755f7b2d090ef3aeaa642fc2ae476a, con relojes integrales observados de 191.686482 / 237.246066 / 319.704406 segundos para 100/500/1024 MiB. Los intervalos de PROFILING fueron 12.498934-13.516264 / 11.999178-13.015148 / 16.004406-17.032454 segundos. Son ventanas observadas con polling de 0.5 s y 4/3/5 ciclos de recursos íntegramente dentro de su tramo estable; no son tiempos exactos del worker ni una división retrospectiva del primer volume_cycle. Se compararon completos perfil/esquema y hashees de un millón de registros en origen, accepted y SQL. Las versiones originales permanecieron intactas y la bandeja incluyó tres recursos por recorrido. Al concluir quedaron cero Jobs, adquisiciones, Runs y leases de Jobs/eventos; inventario principal UNCHANGED.

En PROFILING, RSS acquisition-worker fue 348119040 / 475639808 / 504508416 bytes; CPU entre sondas retenidas, 7.411996 / 7.084830 / 13.356226 segundos. El spill compartido máximo fue 0 / 395608064 / 884178944 bytes. El mayor pico conjunto cgroup entre muestreadores independientes fue 1911558144 / 4216512512 / 5981884416 bytes (1823.004 / 4021.180 / 5704.770 MiB), bajo el guard de 6144 MiB. La cota de 256 MiB del motor no equivale a RSS/cgroup y el mismo filesystem no se suma por servicio. Hubo presión de memoria: acquisition-worker acumuló 19206 eventos memory.events.max en PROFILING de 1 GiB; durante su integral, los incrementos fueron 40334 adquisición, 10182 API, 83 Delivery y 22373 DEFAULT. En el integral de 500 MiB, adquisición acumuló 3259. No hubo OOM/oom_kill, errores de medición ni reinicios de contadores CPU. No se certifica concurrencia adicional.

El JSON conserva CPU/RSS/cgroup/temporales/disco por rol y fase. La CPU de bandeja tiene una sola sonda y queda null; el spill separado por fase de cadena no fue retenido y queda null, aunque sí se midió en perfil/integral. Temporales y spill son ocupación observada, no bytes escritos acumulados ni tamaño total del catálogo. La medición usó fixtures ya existentes en el mismo host y no certifica caché fría. Conserva también otro intento FAIL por 504 al publicar configuración después de preflight: se comprobó su publicación antes de repetir una escritura; stack tardó inactivo y causa NOT_PROVEN. Tras cambiar sólo el descriptor/muestra de Delivery API a Arrow y recrear únicamente API, los tres recorridos se repitieron con UUID y reloj nuevos, sin elevar timeouts/cotas.

La fuente saneada de estas mediciones es acquisition-timing-certification.json (../development/evidence/0.7.0/acquisition-timing-certification.json); el detalle completo del método y las cifras anteriores está en acquisition-volume-0.7.0.md (../development/acquisition-volume-0.7.0.md).

Evidencia conocida y progresión

Ejecución	Medición / alcance
Spark local 1 M	PROBADO final: Intake 51.699 s, Recon 53.161 s, Sentinel 17.753 s; 999900 aceptados, 188 filas de resultados Intake (150 errores + 38 warnings), 1000038 resultados Recon y 6 checks Sentinel. Valores/posición y multiplicidad completos iguales al oráculo. Pico de RSS sumado por fase: 1737011200/1853841408/1814925312 bytes; pico cgroup de ciclo 2151247872 bytes, cota 3 GiB. Suite 35 PASS/0 SKIP (16 JVM + 19 guards), 57.828 s; ciclo 216.452 s, sin OOM.
Spark Standalone 1 M	PROBADO final: Intake 55.453 s, Recon 60.354 s, Sentinel 22.793 s; driver client y dos executors reales. RSS del árbol driver por fase: 640225280/812548096/747683840 bytes; cifras completas verificadas en spark-certification.json. Executors pico cgroup 792555520/796758016 bytes; master 169070592 bytes. Suite 35 PASS/0 SKIP (16 JVM + 19 guards), 65.767 s; ciclo 263.744 s, sin OOM. Cuatro huellas completas iguales a Local y oráculo. La suite mixta contiene 15 escenarios JVM en cluster y 1 publicación Local explícita; los tres módulos de volumen se ejecutan en Standalone.
Perfil completo	PROBADO final: población de 1 M; Local 8.947 s/RSS sumado 276983808 bytes y Standalone 9.130 s/RSS driver 278343680 bytes; son ciclos de engine separados de recepción API. La adquisición normal certifica además su perfil global persistente; los intervalos de su etapa PROFILING se registran en el ensayo adicional. RSS no equivale a cgroup/working set.
Recorrido 100 MiB	PROBADO: 1 M de registros. CSV: 121413300 bytes; JSONL: 184413159 bytes; Parquet: 59408757 bytes. Recepción+adquisición+perfil CSV en el primer ciclo: 73.302 s; preflight: 86.588 s; Intake Spark: 49.732 s; Delivery: 71.280 s. Coinciden la población y los hashes completos de fuente/aceptados/SQL. Ensayo CSV adicional 191ff280: integral real 191.686482 s; transferencia HTTP 0.660746 s, espera durable 29.526450 s; PROFILING acotado entre 12.498934 y 13.516264 s. Preflight/publicación 28.658 s, Intake/hash 54.904 s, Delivery/hash SQL 70.787 s, inbox 1.572 s. Perfil completo igual al original y fuente/accepted/SQL 1 M con hash completo igual.
Recorrido 500 MiB	PROBADO: 1 M de registros. CSV: 541413300 bytes; JSONL: 604413159 bytes; Parquet: 377525863 bytes. Recepción+adquisición+perfil CSV en el primer ciclo: 143.657 s; preflight: 148.765 s; Intake Spark: 56.721 s; Delivery: 80.407 s. Los hashes completos coinciden. Ensayo CSV adicional 191ff280: integral real 237.246066 s; transferencia HTTP 2.595792 s, espera durable 58.547290 s; PROFILING acotado entre 11.999178 y 13.015148 s. Preflight/publicación 29.439 s, Intake/hash 49.970 s, Delivery/hash SQL 86.945 s, inbox 1.431 s. Perfil completo igual al original y fuente/accepted/SQL 1 M con hash completo igual.
Tier 1 GiB	PROBADO: 1 M de registros. CSV: 1090413300 bytes; JSONL: 1153413159 bytes; Parquet: 791832404 bytes. Recepción+adquisición+perfil CSV en el primer ciclo: 235.536 s; preflight: 210.868 s; Intake Spark: 67.458 s; Delivery: 110.441 s. Fuente, aceptados y SQL tienen el mismo hash canónico completo. Se conserva el primer intento FAIL por HTTP 504, causa interna NOT_PROVEN; recrear únicamente API aislada permitió consultar y verificar JSONL (28.524 s) y completar Parquet (79.732 s), sin repetir la cadena CSV ya aprobada. Ensayo CSV adicional 191ff280: integral real 319.704406 s; transferencia HTTP 4.923375 s, espera durable 98.070818 s; PROFILING acotado entre 16.004406 y 17.032454 s. Preflight/publicación 36.536 s, Intake/hash 59.054 s, Delivery/hash SQL 107.521 s, inbox 1.487 s. Perfil completo igual al original y fuente/accepted/SQL 1 M con hash completo igual.
Recuperación/corrupción	PROBADO: worker interrumpido, 2 intentos y una versión completa, 129.848 s; hash idéntico. Staging alterado → ARTIFACT_INTEGRITY_ERROR, 0 versiones. PostgreSQL: 1 M/refresh/cancel PASS.
Opcionales 2/5 GiB	NOT_RUN_RESOURCE_LIMIT por guard conservador previo: nominal+64 MiB supera upload 2 GiB en ambos escalones; la estimación observada 2×nominal supera 4 GiB para 5 GiB. No se generaron ni midieron sus formatos físicos. No se cuentan como PASS ni se deduce capacidad universal de una codificación comprimida distinta; los tamaños no probados permanecen explícitamente no certificados.

Ejecución	Medición / alcance
Interpretación	Mediciones de este host/contexto y de perfiles explícitos de pruebas; no constituyen un SLA. La CPU, si se reinicia el contenedor, se marca COUNTER_RESET/null cuando no puede reconstruirse; no se inventan medidas. Los tiempos del primer ciclo agrupan recepción, adquisición y perfil; no se suman para inventar un reloj integral. El ensayo adicional registra los intervalos observados de PROFILING y un reloj monótonico real desde recepción hasta SQL e inbox, incluidas las pausas de verificación del harness.
Memoria del recorrido 1 GiB	PROBADO en el primer ciclo: máximo conjunto muestreado de cgroup backend 5465.9 MiB; presupuesto de certificación agregado 6 GiB. Se suman memory.current de servicios en el mismo ciclo de probes secuenciales (~1-2 s); no es un snapshot atómico ni la suma de sus máximos individuales. Incluye file cache y spill. Durante Spark: 5370.9 MiB conjuntos; RSS muestreado API 683.2 MiB, acquisition 205.3 MiB, delivery 291.9 MiB y worker 1262.9 MiB. Worker alcanzó la cota propia 3 GiB sin OOM. Estos picos no constituyen un SLA.
Recursos de PROFILING CSV adicional	PROBADO 191ff280, objetivos 100/500/1024 MiB: 4/3/5 ciclos con probes enteros dentro de las ventanas estables. RSS acquisition-worker 348119040/475639808/504508416 bytes; cgroup 470806528/1194463232/1610575872 bytes; CPU entre probes 7.411996/7.084830/13.356226 s. Máximo compartido de spill 0/395608064/884178944 bytes y temporales 418121845/1551199733/3003577491 bytes, sin sumar el mismo filesystem por servicio. Límites del motor 256 MiB, lote 5000 filas/8 MiB; no son cotas de RSS/cgroup. En perfil 1024, memory.events.max acquisition-worker +19206; OOM/oom_kill 0.
Recursos integrales CSV adicionales	PROBADO 191ff280, objetivos 100/500/1024: CPU backend entre probes 251.894378/286.168562/381.188579 s. Pico conjunto del sampler integral 1901072384/4216512512/5970259968 bytes; máximo entre todos los samplers independientes 1911558144/4216512512/5981884416 bytes (1823.004/4021.180/5704.770 MiB), bajo guard 6144 MiB. Spill integral 0/529760256/1023967232 bytes; temporales 539134094/1826738901/3139728234 bytes. Disco libre mínimo entre samplers 1329615585280/1328872882176/1328863682560 bytes. memory.events.max integral 500: adquisición +3259; integral 1024: adquisición +40334, API +10182, Delivery +83, DEFAULT +22373; OOM/oom_kill 0. Picos secuenciales no atómicos, con caché; hay presión real y no se certifica concurrencia adicional. CPU de inbox null por una sola muestra; spill de fases de cadena null/no retenido, aunque sí medido en integral/perfil.
Procedencia del ensayo adicional	PASS sobre 191ff2805f755f7b2d090ef3aeaa642fc2ae476a, runner acquisition_timing_cycle --with-chain, evidencia saneada docs/development/evidence/0.7.0/acquisition-timing-certification.json. Reloj observado HTTP→Intake APPROVED→Delivery COMMITTED→hash SQL→inbox, incluidas verificaciones del certificador; no es suma de fases ni caché fría. Polling de 0.5 s publica intervalos y recursos por rol/fase; versiones originales intactas y cero Jobs/adquisiciones/Runs/jobleases/eventleases al concluir, principal UNCHANGED. Otro FAIL 504 al publicar configuración se conserva: se comprobó su publicación sin repetir POST, stack tardó idle y causa NOT_PROVEN. Tras descriptor/muestra Delivery Arrow y recrear sólo API, los tres recorridos se repitieron desde UUID/reloj nuevos, sin aumentar cotas/timeouts.

Los benchmarks de 0.4..0.6 conservados en development/evidence son antecedentes. No se reutilizan tiempos, memoria ni conteos como certificación de 0.7.0.

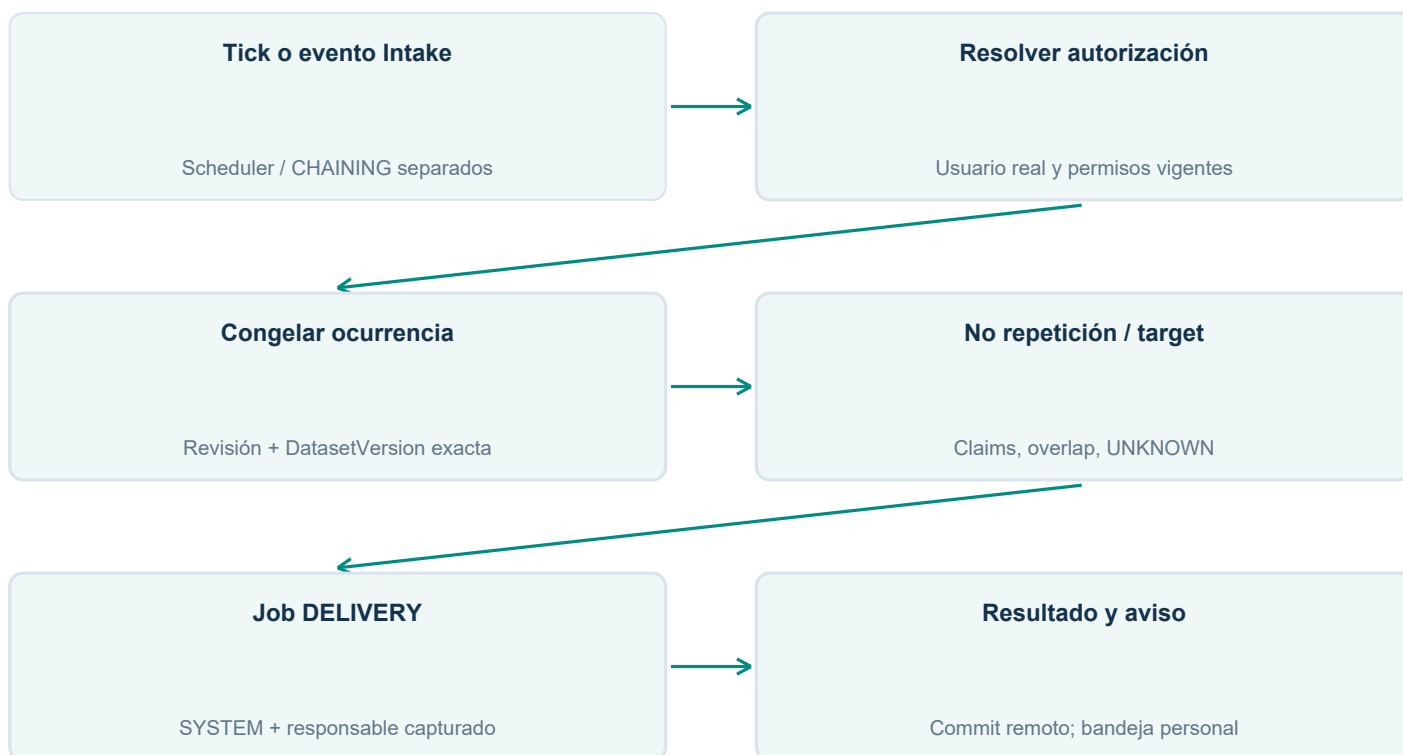
21. Automatización Delivery, ocurrencias y outbox

Configuración versionada y entrada

Data Delivery→Automatización permite ONCE, intervalo, diario/semanal con timezone, inicio, pausa/reanudación, próxima ocurrencia e historia. No hay cron libre ni DAG. Cada edición crea DeliveryAutomationVersion con configuración publicada, responsable_user_id y settings. La configuración manual histórica sigue fijando DatasetVersion; una automatización resuelve otra entrada en una ocurrencia nueva y congela esa identidad junto con revisión, destino, mapping y estrategia.

FIXED_VERSION entrega una versión fijada; LATEST_REGISTERED toma la última elegible ya registrada del dataset, sin refrescar conexiones. INTAKE_OUTPUT usa la salida exacta del Intake concreto que origina el evento. SUCCESS técnico es necesario pero se exige APPROVED por defecto; warnings requieren permiso explícito de política. REJECTED no despacha. La salida debe estar publicada, completa y verificable; se bloquea input vacío salvo autorización explícita, especialmente para OVERWRITE.

La certificación API/PostgreSQL creó y ejecutó Intakes reales para cuatro casos. APPROVED_WITH_WARNINGS sin opt-in terminó SKIPPED/INTAKE_DECISION_NOT_ACCEPTED y cero escrituras; con opt-in entregó dos filas idénticas a sus aceptados. REJECTED terminó SKIPPED y cero escrituras. Un CSV sólo con cabecera produjo Intake APPROVED con salida publicada vacía: EMPTY_INPUT_BLOCKED y cero escrituras. Repetir cada evento no añadió ocurrencias ni escrituras. Son pruebas reales de API/worker/SQL; la cobertura de navegador se declara por separado.



Tres identidades de idempotencia

trigger_key identifica el slot/evento y UNIQUE(automation_id,trigger_key) permite una ocurrencia lógica. La solicitud manual tiene request_key/request_hash para rechazar reutilización con distinto cuerpo. DeliveryInputClaim identifica automation+DatasetVersion y se crea en la misma transacción que la ocurrencia ENQUEUED y su Run/Job, antes de ejecutar la escritura. Permanece aunque la Run termine FAILED, FAILED_PRECONDITION o CANCELLED; una nueva ocurrencia predeterminada no vuelve a procesar esa versión. Las ocurrencias SKIPPED/BLOCKED sin Run no crean un claim. Repetir requiere repeat=true en el despacho manual o habilitar settings.repeat_versions para futuras ocurrencias; la decisión queda visible en execution_plan. Estas opciones no eliminan permisos, estrategia, exclusión del target ni el bloqueo UNKNOWN. Ninguna garantía es deduplicación de negocio de APPEND. CREATE_AND_LOAD no cambia a APPEND/OVERWRITE si la tabla ya existe: se conserva el bloqueo de estrategia.

Ticks, atraso, DST y solapamiento

Scheduler hace sólo despacho sobre metadata. COALESCE_LATEST evita avalanchas de slots vencidos al reiniciar y registra coalesced_intervals/motivo; SKIP_WHILE_ACTIVE evita solapamientos de la misma automatización. Pausa conserva revisiones/historia; reanudación resuelve siguiente slot según timezone/política. Los tests incluyen horarios y DST. DAILY/WEEKLY resuelven la hora en la zona IANA declarada: una hora inexistente por adelanto del reloj se omite; una hora ambigua por retroceso usa únicamente la primera ocurrencia, fold=0, nunca ambas. INTERVAL calcula slots por segundos transcurridos desde starts_at en UTC; no convierte un intervalo en días locales. No hay cron libre ni replay de eventos anteriores al punto de inicio de una automatización nueva.

El responsable debe ser User real de la organización y mantener permisos/cuenta activa. Se comprueba al despachar y antes de STARTED. Se registra SYSTEM en actor de trigger y responsable/captured username en plan/Run para autorización y auditoría remota. Sentinel legacy sin Actor USER verificable queda pausado hasta asignación explícita, sin convertir su etiqueta Responsable en identidad.

Target reconocido y UNKNOWN

DeliveryTargetGuard serializa manual y automática sobre organización+fingerprint de endpoint/destino reconocido y schema/table normalizados. Advisory locks y CAS protegen carrera PostgreSQL. El fingerprint no descubre alias universales, sinónimos, proxies o nombres que llegan al mismo almacenamiento físico; esa limitación se documenta y no se pretende exclusión de todos los escritores externos.

Un UNKNOWN bloquea nuevas entregas al target hasta revisión operacional concluyente y TargetDecision explícita/autorizada. El review no cambia Run/Attempt ni crea entrega. resume-target registra review_id, actor y nota; varios UNKNOWN legacy requieren decidir cada uno. PENDING_REPAIR no es autorización para reenviar. El bloqueo sigue aplicando a una ocurrencia posterior que intentaría actuar como reintento encubierto. No se reenvía una escritura cuya confirmación se desconoce.

Outbox atómica y consumidores independientes

OutboxEvent se registra en la misma transacción de la transición relevante. La publicación de salida Intake y su evento son atómicos. El listener observa transiciones actuales de entidades dirty y eventos explícitos de fallos iniciales; no reinterpreta db.new restaurados ni escanea runs históricos. dedupe_key es persistente por organización y aggregate/transición significativa.

EventConsumption tiene identidad evento+consumer y claim CAS con owner/lease. NOTIFICATIONS y CHAINING tienen progreso/retry separados. El lease default es 300 segundos, configurable entre 60 y 3.600; el heartbeat de salud del proceso es independiente y no sustituye el fence del consumo.

Estado de consumo	Transición y autoridad
PENDING	Elegible cuando available_at ha llegado; el CAS cambia a RUNNING, fija owner/lease e incrementa attempts.
RUNNING	Un lease vencido permite nuevo claim con otra autoridad e intento. El trabajo bloquea la fila y sólo confirma si su owner y lease siguen vigentes.
DONE	La acción del consumidor y esta transición se confirman en la misma transacción. El mismo consumidor no vuelve a despachar el evento.
DEAD	Cinco intentos agotados; no hay retry automático adicional. Conserva error_code sanitizado para revisión operacional.

Un fallo conocido devuelve PENDING con available_at posterior en min(300, 2^attempts) segundos, salvo el quinto intento, que termina DEAD. La comprobación final de lease precede al commit; si venció, se revierte la acción y se registra el retry sin modificar

el proceso que originó el evento. Fallar un aviso no altera SUCCESS/decisión de la Run. Consumir un evento repetido no genera otro aviso ni otro Delivery lógico por sus UNIQUE/claims transaccionales. Un consumidor interrumpido conserva el pending; un owner antiguo no puede confirmar la acción del owner vigente. No se introduce Kafka, RabbitMQ o Redis.

22. Calidad, certificación y matriz de requisitos

Cada resultado debe indicar comando, entorno, revisión, status y evidencia. La suite host no habilita los opt-ins JVM; esa omisión no es PASS. Las suites Docker dedicadas ejecutan los escenarios JVM obligatorios sin SKIP. Las pruebas E2E principales atraviesan navegador, API, PostgreSQL, workers, artefactos, Spark y SQL real; mocks de componente no las sustituyen.

Resultados ejecutados 0.7.0 y gates pendientes

Verificación	Resultado ejecutado
Baseline	PROBADO: 6fac26b, 0.6.1, Alembic 0012 y 31 tablas. Baseline inicial: 1197 backend y 174 frontend; esos resultados son antecedentes.
Preflight mensajes	PROBADO: e37d529 separado, mensajes/códigos y decisiones preservados; pruebas focales de backend/UI.
Suite host completa	PROBADO final: 1527 PASS, 17 SKIP y 0 fallos/errores. Dos invocaciones disjuntas: backend 1076 PASS/17 SKIP/226.00 s sobre 191ff280, fuentes backend idénticas en a5f12dd; scripts completos 451 PASS/0 SKIP/7.43 s sobre el guard/toolkit final a5f12dd. Suma de tiempos 233.43 s, no reloj integral. Incluye Decimal, recuperación multipart, metadata/vista previa Delivery Arrow concurrente y guard de tablas/columnas/FK físicas. Doce casos SQLite reales y transports copiados/stdin con SHA rechazan omisiones y tamper; PostgreSQL aislado real: 42 tablas/509 columnas/75 FK PASS. Dieciséis opt-ins JVM se ejecutan en suites Linux dedicadas; un symlink requiere permisos ausentes en Windows y se ejecuta en CI Linux. Ninguna omisión es PASS. Ruff del alcance CI y mypy de 61 módulos con cuerpos sin anotación PASS. Contratos idempotentes: 117 rutas, 42 permisos, 42 tablas/509 campos/95 índices/164 restricciones; 11 nuevas, 3 modificadas y 28 preservadas. Doce migraciones históricas idénticas al baseline auténtico. CI final vuelve a ejecutar ambos conjuntos juntos en el HEAD documental completo.
Frontend	PROBADO final: 198 PASS/25 archivos, 10.95 s; lint y types PASS sobre a5f12dd. Build de producción previo PASS, fuentes de producto sin cambios posteriores. Incluye defaults null del snapshot, rechazo de cambios reales de precisión/auditoría/DatasetVersion, cinco etiquetas de formato e historial ordenado. Nueva regresión verifica avisos CHAINED omitidos y COMMITTED coexistentes con sus destinos distintos. El selector E2E exige Confirmado/Encadenado y href /runs dentro de la fila confirmada; conserva todos los asserts/timeouts. El CI59 falló en el selector genérico; 13 otros jobs SUCCESS. El rerun real se exige sobre el HEAD final.
Delivery focal	PROBADO: 99 pytest PASS; spool acotado, binding/tamper, claves globales, preflight persistente y compatibilidad.
Automatización SQL/API	PROBADO: PostgreSQL real, 2→4→6 filas, salida exacta Intake, no-repeat, exclusión concurrente de target, recuperación del consumidor y bandeja personal. Cuatro decisiones finales sobre Intakes reales: warnings default bloqueado/0 escrituras; warnings opt-in exacto/2 filas; REJECTED bloqueado/0; salida APPROVED vacía bloqueada EMPTY_INPUT_BLOCKED/0. Duplicar eventos no repite las cuatro entregas/decisiones. Main intacto y cero trabajos/leases al concluir.
Navegador automatización	PROBADO: certificación final 2/2 Playwright PASS, 0 SKIP/flaky/unexpected, privacidad PASS, 206.098 s. El recorrido normal adquiere 1 M, publica contrato y configuración, preflight completo, CHAINED, Spark, SQL y bandeja; fuente/aceptados/SQL tienen hash completo idéntico.
Spark LOCAL	PROBADO final: 35 PASS/0 SKIP (16 escenarios JVM + 19 contratos/guards), suite 57.828 s; ciclo 216.452 s. Intake/Recon/Sentinel sobre 1 M y 999900 aceptados: todos los valores y posiciones iguales al oráculo y a Standalone. Pico cgroup 2151247872 bytes bajo cota 3 GiB, driver exit 0 y cero OOM; cancelación y publicación fenced incluidas.
Spark STANDALONE	PROBADO final: 35 PASS/0 SKIP (16 escenarios JVM + 19 contratos/guards), suite 65.767 s; ciclo 263.744 s. Driver client y dos executors reales; los cuatro fingerprints completos coinciden con oráculo y Local. Executors pico cgroup 792555520/796758016 bytes, cota 1.5 GiB cada uno, exit 0 y cero OOM. Misma huella de fuentes en ambos modos. En esta suite, 15 escenarios JVM utilizan el cluster y la prueba de publicación Run/Job fija local[2]; los tres módulos de millón de filas sí utilizan driver y dos executors Standalone. No se atribuye esa publicación Local al cluster.
Migraciones PostgreSQL	PROBADO: roundtrip 0008→0015, conservación 0012→0015 de 31 tablas y 8 relaciones históricas, paridad ORM y pausa de identidad no verificable; 5.24 s.
Volumen integrado	PROBADO: 100, 500 y 1024 MiB, cada uno con 1 M de registros en CSV/JSONL/Parquet; perfil completo y hash canónico idéntico de fuente, aceptados y SQL. Incluye cadena Spark→SQL→bandeja, extracción/refresh/cancel PostgreSQL real, recuperación de lease y rechazo de staging corrupto. El FAIL 504 inicial y el reintento PASS quedan registrados; causa interna NOT_PROVEN.

Verificación	Resultado ejecutado
CSV integral y etapas	PROBADO final sobre 191ff280: tres recorridos nuevos 100/500/1024 MiB, HTTP→Intake APPROVED→Delivery COMMITTED→hash SQL completo→inbox, en 191.686482/237.246066/319.704406 s de reloj real, incluidas verificaciones. No se suman fases ni se dividen tiempos del ensayo anterior. PROFILING separado por polling de 0.5 s: intervalos 12.498934-13.516264/11.999178-13.015148/16.004406-17.032454 s, no tiempos exactos; 4/3/5 ciclos de recursos dentro de ventanas estables. Perfil/esquema completos iguales al original y versiones anteriores intactas; source/accepted/SQL 1 M y hashes iguales, inbox 3. Jobs/adquisiciones/Runs/jobleases/eventleases 0 y principal UNCHANGED. Evidencia saneada: acquisition-timing-certification.json.
Recursos CSV adicionales	PROBADO 191ff280: pico conjunto cgroup máximo entre samplers independientes 1911558144/4216512512/5981884416 bytes, bajo guard 6 GiB; no instantánea atómica ni suma de máximos por rol. RSS acquisition-worker en PROFILING 348119040/475639808/504508416 bytes; CPU 7.411996/7.084830/13.356226 s entre probes. Spill perfil 0/395608064/884178944 bytes. Integral 1024: CPU backend 381.188579 s, spill 1023967232 bytes, temporales 3139728234 bytes, disco libre mínimo 1328863682560 bytes. memory.events.max perfil 1024 adquisición +19206; integral 500 adquisición +3259; integral 1024 adquisición/API/Delivery/DEFAULT +40334/+10182/+83/+22373. OOM/oom_kill 0; presión real sin garantía de concurrencia adicional. Inbox CPU null por una sonda; spill separado de fases de cadena no retenido/null. Recursos completos por rol/fase y límites 256 MiB/5000 filas/8 MiB en el JSON saneado.
Antecedente publicación CSV adicional	CONSERVADO: primer intento FAIL 504 después de preflight completo; configuración efectivamente publicada, sin repetir POST. Stack tardó inactivo y causa interna NOT_PROVEN. Tras descriptor/muestra Arrow en API Delivery se recreó sólo API aislada y se repitieron los tres CSV con UUID/reloj nuevos sobre 191ff280; todos PASS. Sin aumentar timeouts/cotas ni rescatar tiempos del intento fallido. Antecedente y GET posteriores 200 registrados en acquisition-timing-certification.json.
Recuperación	PROBADO: nativa 42 tablas/0015, 5643 artefactos, 4 secretos fuente y 6 Delivery; huella completa original/restaurada idéntica y multipartes PASS. Backup 11.28 GB, escaneo privado 5666 archivos y SQL descomprimido PASS. Los dos FAIL de backfill se conservaron; fix sin editar filas y fase posterior 150.541 s (no tiempo total del backup). Auténtica 0.6.1 6fac26b→0.7.0 PASS: 31→42 tablas, 11 nuevas vacías, 29 artefactos, 1+1 secretos y notificación histórica preservada, cero replay; origen destruido antes restore, 114.609 s ciclo completo. Automáticos detenidos durante comparación, main intacto y limpieza propia PASS.
SQL/E2E regresión	PROBADO: Delivery final 312 comprobaciones PostgreSQL 16/18 y SQL Server/cuatro estrategias, navegador 1/1, 0 SKIP/fallos, 7.202 s. Connections final 97 comprobaciones + smoke84; navegador30 PASS/13 exclusiones opt-in de otras suites/0 fallos/flaky, 194.160 s. FreeTDS real: 30000 filas y 61440000 bytes payload, fetchmany≤8, primer yield a264 filas, RSS delta5283840 bytes, cancel tras520 leídos y cierre cursor/conexión, 14.525 s. Core/identity y ambos smokes CI48a SUCCESS; final se reejecuta sobre HEAD documental completo.
Promoción final GitHub	PENDIENTE: todos los workflows/jobs del SHA final con código, documentación y PDF. CI191 completó las cuatro etapas de recuperación (operacional 0.7.0 y auténticas 0.5.1/0.6.0/0.6.1): PASS; 133 s paso operacional GitHub de resolución 1 s y 149.785/150.416/124.619 s medidos por los runners históricos. ci-191-recovery.json conserva los resultados y el FAIL anterior ba183 por identidad SQL del fixture 0.6.0, corregido sin ignorar huella ni replay. Estos verdes intermedios no certifican el HEAD documental final. CI59 terminó 13 SUCCESS y un FAIL del navegador 100 MiB en href de notificación sin scope; API/formatos y cuatro decisiones reales PASS. Investigación segura ci-intermediate-59-investigation.json; fix de selector en a5f12dd, pendiente de ejecución real del HEAD final.
Upgrade instalación	PENDIENTE hasta aprobar todos los gates. Proyecto principal intacto; plan/backup preparado sin ejecución.
Certificación release	NO CERRADA mientras existan obligaciones pendientes.

Verificación	Resultado ejecutado
Perfil concurrente	PROBADO: 10 solicitudes con paralelismo 2 y 30 con paralelismo 10: 40/40 HTTP 200, perfil 1 M y muestra 20 exactos; máximo 4.686 s. JSONL 1 GiB: HTTP 200 en 3.035 s, timeout 30 s intacto. Footer/hashes/esquema de todas las partes verificados; Arrow presenta la muestra acotada sin COUNT ni DuckDB. El proceso anterior mostró espera de importcache/importlib; causa completa NOT_PROVEN.

Matriz de requisitos, implementación y evidencia

Requisito	Implementación verificable	Evidencia requerida
A Preflight	Mensajes centralizados, códigos/decisiones preservados, UI checks y auditoría.	test_preflight_messages; Delivery real; preflight-messages.json.
B Adquisición	Stage acotado, AcquisitionRun/Job XOR, formatos/SQL, freeze/refresh, perfil, cancel y fence.	test_async_acquisition*, readers, volume_cycle, acquisition_timing_cycle con perfil/etapas, connections_cycle y browser.
C Spark	Intake/Recon/Sentinel, exact Decimal/Unicode/temporal, refs/grupos globales, local/Standalone y recursos.	test_spark_engine opt-in, spark_cycle, service_e2e, driver/executor evidence.
D Delivery	PreparedRows binding/hash, claves globales, ambas SQL/cuatro estrategias, audit/drift/rollback/UNKNOWN/repair.	test_delivery_streams/service/data_sinks, delivery_cycle y benchmark.
E Automatización	Horarios/timezone/revisiones, identity, no-repeat, aceptación salida exacta, target guard y unknown decisions.	test_automation_events/scheduler y chain_decisions; automation_cycle API/PostgreSQL: warnings default/opt-in, REJECTED, salida APPROVED vacía y eventos duplicados; Playwright real.
F Notificaciones	Inbox personal, filters/read/count, permisos actuales, dedupe y enlaces.	automation/inbox tests; browser, recovery persistente, probes secretos.
G Recuperación	Jobs leases, publicación fenced, parts/staging, outbox pendiente/retry y scheduler independiente.	fault injection determinista de suites, concurrencia PostgreSQL, worker/consumer recovery.
H Compatibilidad	Nuevas migraciones, state 6 exacto, secrets separados, restauración multipart, auténtico 0.6.1→0.7.0.	check_postgres_migrations, native/legacy recovery results y hashes.
I E2E integrado	Large acquire→Intake PySpark→aceptados exactos→Delivery SQL→bandeja del responsable.	volume_cycle/volume.spec y acquisition_timing_cycle --with-chain; negativa/revocación; target count/hash y reloj integral observado.
Documentación/PDF	Capítulos vigentes, tablas/diagramas, generator/input/extraction, copias oficiales.	Hash/páginas/texto seleccionable + inspección visual completa final.
Promoción	HEAD contiene código+migraciones+UI+tests+PDF, todos workflows/jobs aplicables aprobados.	URLs de runs GitHub, SHA exacto y conclusiones de jobs final externos.
Instalación	Backup consistente, mismos volúmenes/PostgreSQL 16, runtime/UI 0.7.0, todos los servicios y huella.	Inventario/verificación del backup y state 5→6 sin fixtures en real.

Reglas de aislamiento y evidencia

Los proyectos temporales usan UUID y puertos localhost disponibles distintos de la instalación principal (el perfil de volumen reserva 32000..32999), DB/secretos/staging propios, imágenes de prueba y límites CPU/RAM/PIDs. Los guards comprueban contexto/rutas/labels/SQL desechable y ausencia de recursos preexistentes. La limpieza elimina sólo recursos de esa ejecución. No usa recursos o credenciales de la instalación principal ni docker prune/down -v sobre ella.

Se conservan fallos/reintentos en historia saneada. Los screenshots/traces/vídeos de rutas con credenciales se deshabilitan; los reportes públicos contienen sólo agregados/IDs/códigos/hasbes. UNKNOWN con supresión determinista del acuse tras commit real se identifica como fault injection; no acredita una pérdida física de red no ejecutada. CI verde de otro SHA no es certificación final.

23. Decisiones vigentes y control documental

ADRs y responsabilidades

Decisión	Registro
Reglas/evidencia/puertos/conectores /Delivery/RBAC/SSO y auditoría histórica	ADR 0001..0019 se conservan y continúan aplicando donde no fueron evolucionadas explícitamente.
Adquisición asíncrona y conjuntos Parquet	ADR 0020.
Inventario state 6, proyección legacy y secretos/backup	ADR 0021.
PySpark real, kernels portables y recursos	ADR 0022.
Preparación/preflight de Delivery de volumen	ADR 0023.
Automatización, outbox y bandeja personal	ADR 0024.

Fuente, candidato, revisión y publicación

Revisión de implementación y runners conocida: a5f12ddc2850dea4053ad77121835e42a37a72cf, rama feat/local-prototype, baseline auténtica 6fac26b3648cb4a4b50c094ef12c1e103bc97ddd. Los dos drivers Spark finales leyeron la misma huella de fuentes 3bdfef0166af8dab6e50ee916c83b02fda9d0c3a2b6908717766df443d2acc5d1; los informes seguros registran hashes de inputs/resultados, entornos y alcance de cada ejecución. El cambio posterior 191ff280 sólo evita SQL nativo en metadata/vistas previas de Delivery API; la iteración poblacional y los engines Spark permanecen iguales. a5f12dd añade únicamente verificación física/transportes de recuperación y pruebas de selector de bandeja; no modifica los engines ni las fuentes de producto backend/frontend. Los ciclos Spark host conservan su procedencia anterior; los dos modos se reejecutan en CI sobre el HEAD final. Esta revisión antecede al commit documental final; sus gates GitHub y upgrade se registran en el informe externo después de aprobarse.

Esta fuente editable, build_specification.py, backend/openapi.json, model_contract_0.7.0.json, permission_contract_0.7.0.json, parameters_0.7.0.json, volume_results_0.7.0.json y validation_results_0.7.0.json son entradas explícitas. Los resúmenes del ensayo CSV adicional proceden de la evidencia saneada acquisition-timing-certification.json (./development/evidence/0.7.0/acquisition-timing-certification.json), SHA-256 f17fc85c2a2303cd4232013125f26339218f3adefb8387c9696b8ac257ec23a9, medida sobre 191ff280; conserva los inputs, intervalos, recursos y antecedentes. Modelos/permisos/OpenAPI se regeneran con scripts/export_contracts.py desde el runtime. El generador produce candidato, valida estructura/texto/bookmarks, y sólo publica tras revisar visualmente todas las páginas. Se regenera extracted.txt y sincroniza la copia oficial ProductOne/Documentación, verificando SHA idéntico. Archivo conserva ediciones originales; no se destruye historia para actualizar la especificación.

Las secciones 24, 31 y 35 son antecedentes de releases anteriores. Las secciones vigentes describen 0.7.0; una decisión histórica de no tener bandeja/PySpark no se interpreta como capacidad actual. Las tablas de validación de aquellos antecedentes conservan sus propios números, sin contarlos como tests 0.7.0.

El PDF documenta la revisión de implementación/evidencia conocida antes del commit documental final. El informe externo identifica el HEAD final y todos los jobs de CI de ese SHA. No se genera otro commit sólo para incluir dentro del PDF el hash que lo contiene. Si cambia cualquier archivo versionado, la promoción y gates correspondientes se verifican de nuevo.

Registro de revisión

Fecha	Edición	Cambio
27-09-2026	v1.1, implementación 0.6.1	Temporal efímera, retiro de SMTP, SSO opcional, 31 tablas/0012.
03-10-2026	v1.1, implementación 0.7.0	Asincronía, multipartes, PySpark, Delivery de volumen, automatización, outbox, bandeja, 42 tablas/0015.

24. Antecedente: cambios funcionales y técnicos 0.5.1

1. Reparación local de evidencia COMMITTED

Antes: el worker preservaba COMMITTED y marcaba PENDING_REPAIR, pero el operador no tenía una acción segura para completar receipt/manifest. Riesgo: confundir evidencia ausente con entrega fallida y volver a escribir.

Solución funcional: "Reparar evidencia" conserva el commit. Solución técnica: validación local estricta, bloqueo de Run, artifacts deterministas, publicación sin reemplazo y reutilización de enlaces. Archivos: delivery_service.py, delivery_api.py, delivery_schemas.py, DeliveryOperations.tsx y tests operacionales. Modelo: no añade tabla para reparar. API: POST repair-evidence con runs:execute.

Compatibilidad: conserva bytes 0.5.0 y no altera intentos ni migraciones anteriores. Pruebas: negar DataSink/SecretStore, repetición/concurrencia, parcial/ausente, tampering, organización, permisos y restore. Se cubre también la ventana normal COMMITTED anterior a receipt/manifest: UI y runners esperan publicación con límite, sólo por GET y sin ocultar PENDING_REPAIR. Límite: no puede inventar evidencia independiente ausente ni corregir bytes corruptos sobrescribiendo su hash.

2. Revisión operacional de UNKNOWN

Antes: UNKNOWN expresaba correctamente la incertidumbre, pero carecía de un historial estructurado de verificación externa. Riesgo: sobrescribir el intento original o esconder la conclusión en un comentario no consultable.

Solución funcional: observaciones append-only con tres outcomes, nota y revisor. Solución técnica: DeliveryOperationalReview y 0009_delivery_reviews, FKs/CHECK, scope y fecha validada; el worker deja intacto un attempt ya UNKNOWN. Archivos: models.py, migration 0009, delivery_service/api/schemas.py, worker.py y UI.

API: GET/POST reviews. UI: formulario e historial con advertencia de no replay. Compatibilidad: ninguna fila previa cambia. Pruebas: actor/fecha/organización, outcomes, historial, intento inmutable, sin Job nuevo, upgrade/downgrade y recovery. Límite: una afirmación humana no es confirmación automática ni aprobación dual.

3. Semántica uniforme de métricas

Antes: algunas descripciones trataban rows_written como filas físicas y podían dar a null apariencia de cero. Riesgo: inferir una población remota incorrecta, especialmente con UPSERT, triggers o filas preexistentes.

Solución funcional: mostrar filas enviadas y N/D explícito. Solución técnica: delivery_metrics.py centraliza metric_semantics versión 1; servicio, DTO, receipt, manifest y UI conservan null. Run nuevo persiste tiempos separados de preflight y escritura. Modelo: sin nuevas columnas ni reinterpretación histórica.

Pruebas: valores conocidos/desconocidos/cero, recibos históricos sin metadata aditiva y formato UI. Archivos: data_sinks.py, delivery_metrics.py, delivery_service.py y feature delivery. Límite: bytes preparados no son tráfico de red y filas enviadas no equivalen a inventario físico post-trigger.

4. Desglose UPSERT PostgreSQL por capacidades

Antes: el conteo diferenciado no podía garantizarse en todas las versiones. Riesgo: usar un SELECT previo sujeto a carreras o un detalle interno como xmax.

Solución: PostgreSQL 18 RETURNING OLD/NEW, preservando ON CONFLICT, locks y constraint nombrada; PostgreSQL 16 conserva null cuando no sabe. No modifica el servidor metadata ni exige actualización de destinos. Archivos: data_sinks.py, delivery_metrics_probe.py, delivery_cycle.py y overlay de destinos de pruebas.

API/UI reciben enteros o null existentes; no hay migración. Pruebas reales: inserción, actualización, mezcla, clave compuesta, vacío, rollback, drift, supresión por trigger y registro OLD totalmente null. Límite: acciones reportadas no incluyen un censo de efectos secundarios del receptor.

5. Regresión temporal exhaustiva

Antes: existía la política conservadora, con cobertura parcial de precisiones y tránsito entre módulos. Riesgo: que un cambio en driver/lector pierda fracción, invente zona o convierta un texto sólo porque parece una fecha.

Solución: matrices de precisión PostgreSQL y SQL Server, casos null/offset y valores exactos, refresh e historia, Intake/Sentinel/Recon y entrega. Archivos: test_dataset_sources.py, test_data_sinks.py, connections_cycle.py y fixtures. No cambia modelo, API ni política temporal como parte de la regresión.

Compatibilidad: snapshots anteriores permanecen inmutables. La UI conserva su tipo lógico y no ofrece coerción prohibida en mapping. Pruebas reales y unitarias se registran separadas. Límite: TIMESTAMP llega hasta microsegundos; el séptimo dígito o ausencia de zona se conserva como STRING. La adquisición SQL Server estilo 127 puede normalizar a UTC; no se promete preservar la forma numérica original del offset anterior a materializar el snapshot.

6. Linaje canónico sin migración histórica

Antes: documentación confundía DELIVERED_TO con el tipo de entidad DELIVERY_DESTINATION_VERSION. Riesgo: lectores/documentación dibujando un grafo distinto del persistente.

Solución: tabla y diagrama exactos de source_type, relation y target_type; se mantienen DELIVERY_INPUT, DELIVERED_TO, DELIVERY_RECEIPT, EVIDENCE_OF y RUN_OUTPUT. Archivos: especificación, ADR0015, API_CONTRACT, services.py y pruebas de linaje. Modelo/API: ningún rename de datos ni backfill destructivo.

La reparación reconstruye los mismos enlaces faltantes y no inventa una relación nueva. El drill detectó que el backfill legacy de arranque añadía RUN_INPUT a Runs Delivery; ahora los excluye, sin borrar vínculos históricos preexistentes. Pruebas contrastan triples, ausencia de duplicados y preservación exacta tras reinicio/restore. Límite: este grafo describe evidencia interna; no representa linaje a nivel fila en el sistema receptor.

7. Benchmark exclusivo de Delivery

Antes: el benchmark general cubría adquisición/calidad, no aislaba estrategias de salida. Riesgo: usar su throughput como si midiera Delivery.

Solución: runner dedicado, fixture variado determinista, ocho combinaciones motor/estrategia, smoke y representativo acotado, watchdog y recursos observados. Archivos: delivery_benchmark_cycle.py, overlay delivery-benchmark, tests, workflow CI, informe y evidencia JSON 0.5.1. API: utiliza contratos públicos; modelo/UI de producto no cambian por el benchmark.

Pruebas: guard de proyecto, límites sin aumento, generación determinista, mediciones reales y limpieza. Compatibilidad: no usa main ni datos del usuario. Límite: NOT_RUN_RESOURCE_LIMIT se conserva; no se extrapola a producción ni se afirma una precisión temporal/espacial mayor que la del muestreo.

8. División del frontend en módulos de carga diferida

Antes: rutas pesadas compartían un bundle inicial de 611,41 kB. Riesgo: coste de arranque y advertencia de tamaño, aunque la pantalla solicitada fuera pequeña.

Solución: React.lazy/Suspense, imports dinámicos y frontera de error; separación del detalle Delivery del builder. Archivos: App.tsx, RouteContent.tsx, Runs.tsx, DeliveryRunDetail.tsx y pruebas. Modelo/API: sin cambio de contratos por splitting. Compatibilidad: rutas, sesión, permisos, callbacks y navegación directa conservados.

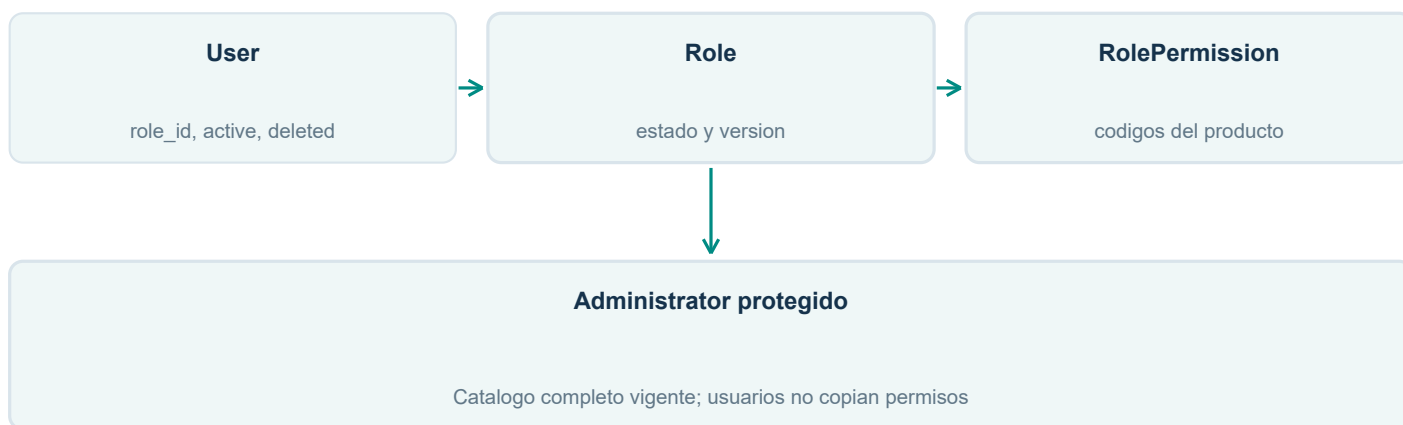
Pruebas: Vitest incluye una promesa lazy rechazada para el fallo de carga; Playwright intercepta la API para rutas/deep links y se ejecuta por separado con backend real. No se atribuye al navegador una inyección de fallo de red del chunk. Build medido

antes/después: bundle inicial nuevo de 364,97 kB; la suma de todos los chunks se informa por separado. Límite: la primera apertura de una feature puede requerir su descarga; una caché o red defectuosa debe mostrar error recuperable, no una pantalla vacía.

25. RBAC dinámico y administración real de roles

En 0.5.1 cada usuario contenía una etiqueta de rol y `permissions.py` resolvía un diccionario estático. Cambiar accesos exigía un release; el cliente podía retener permisos hasta cerrar sesión y una ruta desconocida recibía `runs: read`. En 0.6.0 la relación persistida es `User.role_id → Role → RolePermission`. La etiqueta histórica `User.role` se conserva por compatibilidad de almacenamiento, pero no autoriza peticiones. La API devuelve el nombre vigente desde `Role`.

El catálogo de códigos pertenece al producto, versionado junto con la matriz de rutas. No hay API para inventar códigos. Un rol configura un subconjunto de ese catálogo. Incluye Datasets, Conexiones, Intake, ReconOps, Sentinel, Delivery, Destinos, Excepciones, Reglas, exports, artifacts, Auditoría, Usuarios, Roles, Notificaciones, Sistema y las vistas transversales de Runs. Los permisos de consulta, configuración, ejecución, sobrescritura, ALTER, revisión y reparación se distinguen. Las rutas compartidas de Runs deben validar además el módulo del recurso consultado: un permiso transversal nunca debe abrir datos de un módulo que el rol no puede consultar.



El backend valida dependencias transitivas. Por ejemplo, sobrescribir exige ejecutar y consultar Delivery; alterar exige ejecutar; administrar destinos exige consultarlos. La UI puede completar esas dependencias, pero una petición manipulada que las omita recibe error. `users:manage` y `roles:manage` son exclusivos de Administrator: no pueden asignarse a un rol custom, aunque los solicite un administrador. Resolver permisos siempre vuelve a intersectar con el catálogo y a excluir capacidades no delegables de los roles ordinarios.

Administrator se reconoce por `system_key=ADMINISTRATOR`, no por el texto visible. Es permanente: no se elimina, desactiva, renombra ni pierde permisos. Obtiene el catálogo completo en cada resolución; no conserva una copia que envejezca con las versiones. Las mutaciones de usuarios se serializan por organización para evitar que dos administradores se desactiven simultáneamente dejando cero administradores activos. La baja o degradación del último administrador se rechaza, igual que una eliminación propia que comprometa la administración.

La migración crea Administrator, Data Owner / Lead, Data Analyst, Operations y Auditor por organización. Data Owner histórico pasa a referenciar el rol equivalente. Se conservan los accesos funcionales anteriores mediante permisos granulares; no se reescriben actores, auditorías ni configuraciones históricas. Las semillas son sólo valores iniciales: nunca se reaplican sobre un rol que ya ha sido administrado. Los roles desconocidos heredados fallan cerrado.



Un usuario desactivado sigue bloqueando la baja de su rol; un usuario eliminado lógicamente deja de bloquearla. El nombre normalizado continúa reservado después de eliminar el rol. La versión esperada evita sobrescribir una edición concurrente. El número de usuarios que muestra la UI cuenta asociaciones no eliminadas, independientemente del estado activo.

La sesión identifica al usuario; no almacena permisos autoritativos. Cada petición valida usuario activo/no eliminado, rol vigente y organización. Cambiar permisos afecta la siguiente petición sin recrear usuarios. /me entrega `role_version` y permisos vigentes; el frontend refresca periódicamente, al recuperar foco y ante un 403. Puede existir un intervalo breve de presentación desactualizada, pero la API ya aplica la revocación. Cambiar el rol de un usuario revoca todas sus sesiones y exige autenticarse de nuevo.



Las APIs de roles permiten listar/buscar, consultar catálogo, crear, editar y eliminar lógicamente. Configuración presenta permisos agrupados y explica la protección de Administrator. Los eventos `ROLE_CREATED`, `ROLE_UPDATED`, `ROLE_PERMISSIONS_CHANGED`, `ROLE_DISABLED` y `ROLE_DELETED` fijan actor, organización, identidad y cambio; no guardan credenciales. La matriz explícita de endpoints se mantiene junto al contrato y se contrasta con las rutas instaladas por FastAPI.

26. Usuarios, credenciales y primer acceso

Alta e identidad estable

El formulario conserva nombres, apellidos, username, correo electrónico, rol y estado. Email sigue siendo metadata de identidad y puede participar en SSO; no se usa para entregar credenciales. Para usuarios nuevos, name una nombres y apellidos. Los campos heredados nulos no se inventan. Username admite 3 a 80 caracteres ASCII (letras, números, punto, guion y guion bajo) y se normaliza para unicidad global; email también. La baja lógica conserva identificadores, FKs e historia. Cambiar username no reescribe auditoría anterior ni el snapshot del actor de una Run encolada.

Emisión efímera y entrega externa

POST /users requiere users:manage, sesión normal y CSRF. Genera mediante CSPRNG una temporal de 32 caracteres, persiste únicamente su hash Argon2 y fija must_change_password=true y temporary_password_expires_at=ahora+24h. No acepta una contraseña elegida por el administrador. La respuesta 201 separa explícitamente UserResponse del DTO de emisión UserCredentialIssueResponse:

```
{
  "user": {"id": "...", "username": "usuario", "must_change_password": true},
  "temporary_credentials": {
    "username": "usuario",
    "temporary_password": "<valor efimero, ejemplo no utilizable>",
    "expires_at": "<UTC ahora + 24 horas>",
    "must_change_password": true
  }
}
```

La respuesta lleva Cache-Control: no-store y Pragma: no-cache. El DTO normal nunca contiene el secreto: GET /users, GET /users/{id}, /me y auditoría no lo devuelven. No existe consulta para recuperar una temporal. Si la respuesta se pierde, el usuario puede haber quedado creado; se consulta su estado y se regenera con la versión vigente. No se repite el alta a ciegas ni se supone que una entrega por red se completó.



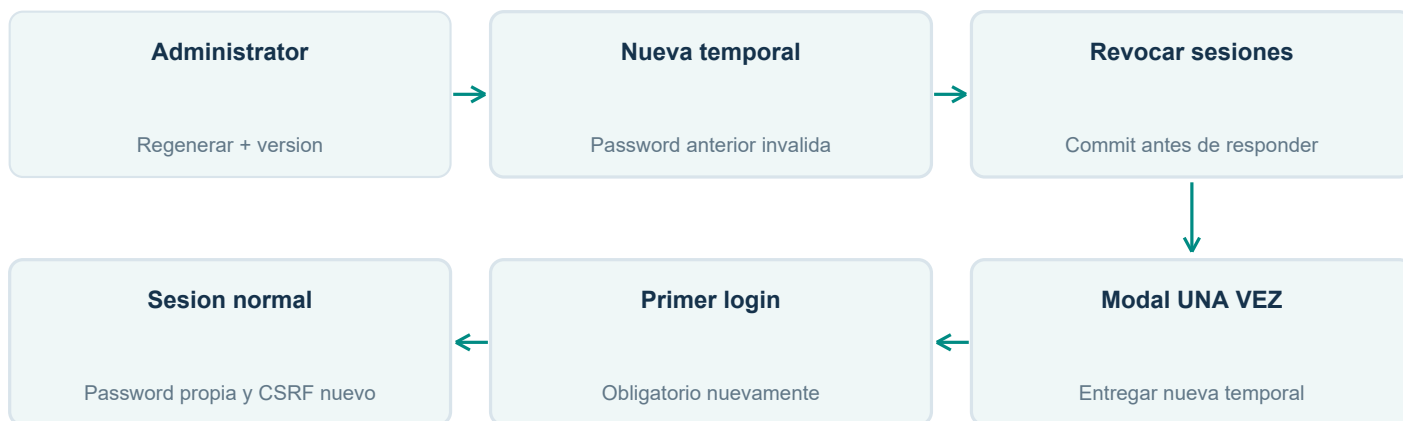
Modal obligatorio y ciclo de vida del plaintext

Al completar el alta aparece «Usuario creado» con nombre, username, contraseña temporal, vigencia y rol. El texto indica: «Guarda estas credenciales ahora. La contraseña temporal sólo se muestra una vez y Trackvance no podrá recuperarla posteriormente». Permite copiar username, contraseña o ambas credenciales, y mostrar/ocultar la temporal. Cerrar elimina el objeto de estado; desmontar la pantalla también lo elimina y descarta respuestas tardías. Volver a Usuarios o navegar atrás no puede restaurarlo.

La petición se realiza directamente, fuera del mutation cache de React Query. No se escribe el secreto en localStorage, sessionStorage, cookies, URL, query string ni estado persistido. La memoria del navegador no ofrece borrado criptográfico verificable; el contrato elimina referencias controladas por la aplicación y evita retención persistente. Copiar es una acción explícita del administrador: el portapapeles y el canal externo elegido quedan fuera del almacenamiento de Trackvance. El producto no promete borrar portapapeles, historiales del sistema ni mensajes enviados por ese canal.

Regeneración y concurrencia

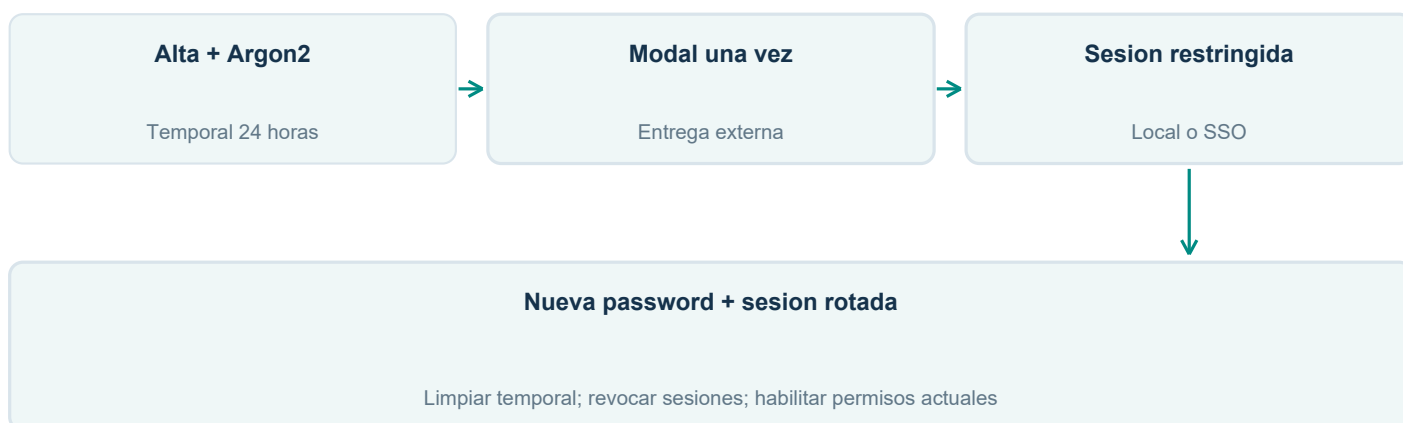
POST /users/{id}/regenerate-credentials recibe la versión esperada. Una operación exitosa genera una temporal nueva, reemplaza el hash, renueva 24h, restablece el cambio obligatorio y revoca sesiones en la misma operación persistente antes de devolver la respuesta 200. La UI presenta el mismo modal con título «Credenciales regeneradas». La temporal anterior y la contraseña previa quedan inválidas. Un conflicto de versión devuelve 409 y no emite un secreto nuevo. El alias /resend-credentials y el alias histórico /reset-password se conservan deprecated con esta misma semántica; ninguno envía email.



Login, expiración y primer acceso

Login acepta username o email y aplica un error genérico. La temporal vencida no permite login local; el administrador debe regenerarla. Una sesión must_change_password sólo puede consultar /me, cerrar sesión o invocar /auth/first-login/change-password. Se bloquean los módulos y sus endpoints indirectos. La nueva contraseña tiene mínimo 12 caracteres, debe confirmarse y no puede coincidir con la temporal vigente. Al cambiarla se fija must_change_password=false, temporary_password_expires_at=null y password_changed_at, se invalidan sesiones anteriores y se emiten nueva sesión y nuevo CSRF.

Errores inesperados de hashing o persistencia durante emisión y primer cambio se convierten en CREDENTIAL_ISSUE_FAILED o PASSWORD_CHANGE_FAILED con texto fijo y rollback cuando corresponde. La excepción original no llega al logger general; las regresiones inyectan texto secreto para comprobar que no aparece en respuesta ni log.



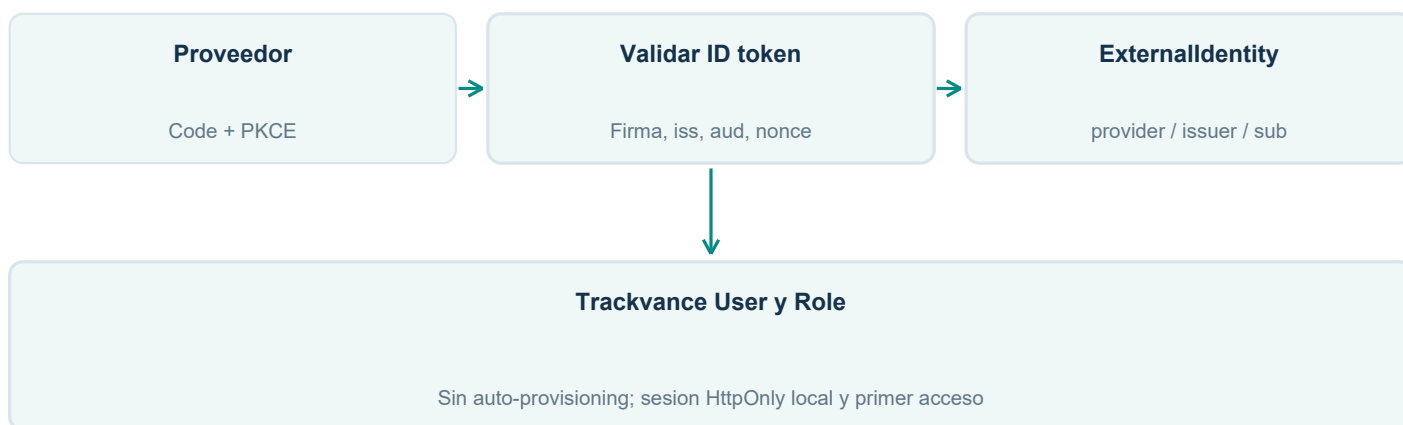
SSO no omite este paso: una cuenta preprovisionada que entra primero con Microsoft o Google y mantiene la obligación recibe una sesión restringida hasta definir contraseña local. La temporal obtenida por el administrador no se mantiene como alternativa activa después del cambio. Local sigue habilitado aunque ambos proveedores estén deshabilitados.

Estado, permisos y auditoría

Usuarios muestra «Primer acceso pendiente», «Contraseña definida» o «Contraseña temporal vencida»; no muestra estado SMTP. Acciones: Acceso, Editar, Regenerar credenciales y Eliminar. Desactivar revoca sesiones; eliminar fija deleted, active=false y deleted_at. El último Administrator y el rol del sistema conservan sus protecciones. USER_CREATED, USER_UPDATED, USER_ROLE_CHANGED, USER_DISABLED, USER_DELETED, USER_CREDENTIALS_REGENERATED y USER_PASSWORD_CHANGED no contienen plaintext ni password_hash. No se producen NOTIFICATION_SENT/FAILED por altas o regeneraciones 0.6.1; los eventos históricos se conservan.

27. SSO Microsoft y Google

SSO está implementado pero deshabilitado por defecto. Configuración → Autenticación muestra login local habilitado y Microsoft/Google deshabilitados en la instalación estándar. Su activación se decide durante la implantación según el cliente; no es requisito para usar ni certificar una instalación local. No se prueban proveedores reales en esta edición: NOT_RUN_EXTERNAL_CREDENTIALS. Las guías del capítulo 32 son opcionales y no justifican introducir secretos externos en CI.



SSO no crea cuentas. El administrador debe preprovisionar User y elegir su rol. El primer enlace exige un identificador cuya autoridad haya validado el proveedor, coincidente con esa cuenta existente. Después se resuelve únicamente por provider + issuer + subject. Un cambio de email externo no reasigna la identidad ni puede transferirla a otra cuenta. ExternalIdentity tiene una clave única global y vínculo estable a User; no contiene access tokens, refresh tokens, ID tokens ni client secrets.

El flujo es server-side Authorization Code con PKCE S256, state aleatorio, nonce y binding al navegador. Un intento efímero tiene expiración y consumo único. El callback consume el intento, intercambia el code y valida firma, algoritmo, issuer, audience, expiración y nonce mediante bibliotecas JOSE/OAuth mantenidas. El secreto de cliente sólo sale hacia el token endpoint esperado. La URL final es local y controlada, sin un return_to arbitrario. La sesión resultante usa la cookie HttpOnly normal de Trackvance.

Microsoft usa discovery de la autoridad common y la App Registration debe aceptar directorios organizacionales y cuentas personales mediante AzureADandPersonalMicrosoftAccount. La validación enlaza tenant, issuer y clave de firma; common no significa aceptar cualquier issuer. Outlook/Hotmail/Live y cuentas work/school están dentro del flujo. Un email/preferred_username por sí solo no prueba propiedad suficiente para enlazar desde un tenant arbitrario: el backend aplica la política de autoridad documentada en la guía de SSO. La redirect URI se registra como Web y sin query parameters.

Google admite Gmail y Workspace. El primer enlace exige email_verified=true y correo Gmail, o Workspace con hd adecuado. Una cuenta Google basada en un correo externo sin esa autoridad se rechaza para enlace automático incluso si Google marcó el email como verificado tiempo atrás. Después del enlace se usa issuer/sub, no el email. No se solicita Gmail API, Microsoft Graph, calendarios ni scopes de escritura: únicamente openid, profile y email.

SSO como primer acceso no elimina la obligación de contraseña local. Si User todavía conserva una temporal, el login externo válido crea una sesión restringida y obliga a definir una contraseña local nueva. Sólo completar ese paso invalida definitivamente la temporal. Así todas las cuentas mantienen un método local en 0.6.1 sin dejar contraseñas de incorporación activas indefinidamente.

Usuarios desactivados/eliminados no pueden acceder por ningún proveedor. Roles y permisos continúan resolviéndose en Trackvance; claims de grupos, directorios o roles externos nunca elevan privilegios. Un Administrator puede desvincular la identidad externa desde Métodos de acceso sin borrar User. Los eventos SSO_LINKED, SSO_UNLINKED, SSO_LOGIN_SUCCESS y SSO_LOGIN_FAILED se sanitizan. Se excluyen query strings del access log para impedir almacenar códigos OAuth.

Los proveedores deshabilitados o incompletos no muestran botón y no bloquean el arranque ni readiness. /auth/providers sólo expone estado público. Discovery, JWKS y token exchange se hacen durante autenticación, no como requisito de salud de workers. El modo de proveedor falso exige un indicador explícito de test y vive en un overlay desechable; no está habilitado en Compose base.

28. Notificaciones internas y conservación de historia

Bandeja vigente por usuario

La navegación principal incorpora Notificaciones y contador de no leídas. TanStack Query actualiza periódicamente sin WebSockets. La página filtra por leídas/no leídas, módulo y origen MANUAL/SCHEDULED/CHAINED, pagina y enlaza al detalle. Cada notificación muestra fecha, descripción funcional, estado técnico y decisión de negocio, sin datos de filas ni contenido sensible.

Adquisición informa publicación/fallo/cancelación. Intake SUCCESS+REJECTED explica que terminó y rechazó el dataset; Recon/Sentinel conserva decisión global y alertas. Delivery COMMITTED confirma la transacción; PENDING_REPAIR identifica evidencia local pendiente; UNKNOWN exige revisión; FAILED_PRECONDITION describe bloqueo anterior al intento. DELIVERY_PREFLIGHT enlaza a /delivery/validation/{id} y diferencia SUCCESS técnico de PASS/FAIL. Se crean resúmenes por transición, nunca por fila.

Destinatarios y autorización

Manual corresponde al iniciador; programada/encadenada al responsable_user_id real de automatización/schedule. SYSTEM no recibe una inbox y no hay difusión automática a administradores. UNIQUE(event_id,recipient_user_id) evita duplicar avisos por retry. read_at se persiste; reiniciar conserva lista/contador/lectura. Evento y consumo se pueden recuperar de pending sin cambiar resultado original.

La API exige organización, usuario destinatario y permiso actual del módulo enlazado además de notifications:read. Aplica la misma condición al contar, listar y marcar leído/read-all. Administrator no obtiene acceso a avisos ajenos. Los detalles vinculados revalidan permisos/propiedad; un enlace no concede acceso que el recurso haya perdido por revocación.

Historia y credenciales

notification_deliveries mantiene registros de 0.6.0 sin reinterpretarlos como InternalNotification. Sus endpoints deprecated conservan HISTORICAL_ONLY. No existe SMTP, Mailpit, envío de credenciales ni canal externo en 0.7.0. Alta/regeneración mantiene presentación efímera única y primer cambio obligatorio. Los avisos nunca conservan password, cookie, CSRF, token OIDC, DSN o excepción cruda.

29. Persistencia y recuperación de identidad/automatización

Los usuarios/roles/grants/OIDC/notification_deliveries históricos conservan identidad. Nuevas tablas de bandeja y automatización no alteran roles personalizados ni el historial de intentos Delivery. state 6 incluye todo el inventario; unknown tables no se omiten. Secrets cifrados y claves se conservan separados en restore.

Sentinel sólo atribuye responsable si su Actor USER legacy es verificable dentro de la organización. Otros schedules se pausan con bandera de enabled anterior. Asignar una cuenta real es una operación explícita; no reescribe actores de ejecuciones antiguas. Esto permite preservar huella legacy y asegurar despacho futuro autorizado. La etiqueta libre Responsable no determina acceso.

La recuperación nativa debe conservar inbox leído/no leído, outbox pendiente, consumos/leases, revisiones/ocurrencias/input claims/target decisions, adquisición y conjunto multipartes. Restaurar no activa destinos reales ni reproduce eventos históricos. Los dispatchers se inician sólo después de verificar el estado y aislar/configurar destinos. UNKNOWN se conserva bloqueado; reparar evidencia no contacta el servidor SQL.

Las credenciales efímeras siguen fuera de metadata/evidencia; backup conserva sólo hashes Argon2 y secretos de conectores cifrados. Los probes revisan dump descomprimido, tar, logs, artifacts y evidencia de navegador con secretos conocidos sintéticos, y publican únicamente counts/PASS/FAIL.

30. Auditoría técnica de Data Delivery

Cambio funcional respecto a 0.5.1

Data Delivery ya publicaba DatasetVersion inmutables a PostgreSQL y SQL Server, con destinos versionados, mapping técnico, cuatro estrategias y evidencia de cada intento. La tabla receptora no permitía atribuir una fila a su última ingesta Trackvance. 0.6.0 agrega una opción conjunta de auditoría de publicación y una obligación persistente por target. Los datos de negocio no se transforman ni se modifican en origen. No se implementan SHIST, tablas históricas paralelas, SCD, fechaDesde/fechaHasta ni detección histórica de cambios.

El riesgo de una simple opción en Configuration sería que otra configuración publicara sobre la misma tabla sin auditoría o que una rotación de credenciales hiciera desaparecer el requisito. La solución introduce DeliveryTargetPolicy, independiente de revisiones del destino y de credenciales. Otro riesgo era atribuir retroactivamente la fecha de habilitación a registros previos: las columnas agregadas a tablas existentes son nullable, sin default.

Uso del builder

1. Elegir DatasetVersion y destino, con la revisión correspondiente.
2. Seleccionar schema/tabla existente o solicitar tabla nueva.
3. Activar **Incluir campos de auditoría de Trackvance**.
4. Revisar el mapping de negocio. fechaIngesta y usuario quedan reservadas y no aceptan valores del mapping. No se solicita username al operador.
5. Ejecutar preflight. Sobre tabla existente debe haber metadata compatible y, si falta alguna columna, permiso ALTER remoto.
6. Publicar. Desde ese momento el target queda obligado a usar ambas columnas, incluso si la primera Run no se ejecuta o falla.
7. Ejecutar una Run con la DatasetVersion publicada. El worker vuelve a comprobar el target y ejecuta DDL/DML juntos cuando corresponde.

Si la política ya existe, el builder muestra ambas columnas activadas y bloquea su deshabilitación con el texto: «Esta tabla utiliza auditoría de ingesta de Trackvance. Todas las entregas posteriores deben registrar fechaIngesta y usuario». La API también rechaza drafts manipulados; ocultar un checkbox no es autorización.

Qué significan las columnas

Campo	Valor	Momento de captura
fechaIngesta	Un timestamp UTC por DeliveryAttempt, hasta seis dígitos fraccionales (microsegundos)	Preparación local del intento, antes de STARTED
usuario	Username interno del User que encoló la Run	Creación de la Run, snapshot inmutable

Una cuenta que entra por Microsoft o Google conserva exactamente el username asignado en Trackvance. No se usan display name, email, subject externo ni usuario de conexión SQL. Si después se renombra el User, la Run conserva su snapshot. Los timestamps corresponden al intento de ingesta, no al instante de commit SQL ni a la fecha de creación original del registro de negocio.

Estrategia	Efecto de auditoría
CREATE_AND_LOAD	Cada fila insertada recibe timestamp y username del intento
APPEND	Sólo las filas nuevas reciben valores; filas previas quedan intactas
OVERWRITE	DELETE e INSERT transaccionales; todas las filas resultantes reciben valores
UPSERT INSERT	La nueva fila recibe ambos valores
UPSERT UPDATE	La fila coincidente actualiza también ambos valores

Por tanto, estas columnas representan la **última ingesta Trackvance** que tocó la fila. Un actor externo todavía puede escribir en una tabla administrada por su organización; Trackvance no instala triggers globales ni intercepta cambios externos. La garantía de auditoría se aplica a sus propias entregas.

Esquemas y compatibilidad de tipos

Motor	Columna temporal creada	Columna de actor creada
PostgreSQL	"fechaIngesta" TIMESTAMPTZ(6)	"usuario" VARCHAR(128)
SQL Server	[fechaIngesta] DATETIMEOFFSET(6)	[usuario] NVARCHAR(128)

Para CREATE_TABLE las dos son NOT NULL. Para una tabla existente, el ALTER declara explícitamente NULL y omite DEFAULT. No se ejecuta un UPDATE de backfill. Una fila previa permanece NULL/NULL salvo que la propia estrategia la actualice o sustituya: por ejemplo, UPSERT UPDATE registra correctamente la nueva ingesta.

Se adoptan columnas existentes compatibles. El campo temporal debe preservar zona/offset y al menos seis dígitos de precisión fraccional (microsegundos). El campo usuario debe ser Unicode/string variable con capacidad mínima de 128, o sin límite. Identidades y campos generated no se consideran escribibles. Si falta una sola columna, se crea esa columna; si una existente es incompatible, no se cambia su tipo, no se renombra y no se elimina/recrea la tabla.

PostgreSQL exige identidad exacta del nombre citado. Una columna fechaIngesta creada sin comillas no se adopta silenciosamente como fechaIngesta. SQL Server compara nombres de auditoría de forma insensible a mayúsculas, conserva el nombre real descubierto para DML y rechaza coincidencias ambiguas. El mapping de negocio se conserva con los contratos estrictos de tipos de 0.5.1.

Modelo persistente y migración

0012_delivery_target_audit es aditiva y sucede a 0011_notification_delivery. No modifica migraciones históricas. Agrega delivery_target_policies y delivery_attempts.system_audit JSON. Los intentos históricos reciben {}; no se inventan usernames ni timestamps retrospectivos.

La política contiene identidad estable, organización, destino inicial, fingerprint único por organización, motor, host, puerto, base, schema, tabla, audit_columns_required=true, habilitador user ID/username y fechas de habilitación/materialización/creación. La restricción CHECK impide false y no hay endpoint de deshabilitación o borrado. El User se conserva con baja lógica, de forma que el habilitador y las Runs siguen teniendo identidad histórica.

La huella se deriva de JSON canónico de organización/motor/host/puerto/base/ schema/tabla. No incluye password, username SQL, secret_reference ni TLS. Normaliza host DNS e IP; conserva el case de identificadores PostgreSQL. Para SQL Server aplica casefold de base/schema/tabla conservadoramente: alias de mayúsculas no evaden la política, pero dos tablas distintas sólo por mayúsculas en un servidor case-sensitive comparten obligación. Alias DNS diferentes no se resuelven como un único servidor físico automáticamente. Ese límite del locator configurado requiere gobierno de conexiones, no acceso a credenciales.



Fronteras transaccionales y permisos

Publicación serializa por fingerprint en PostgreSQL mediante advisory lock de transacción, incluyendo la ausencia de fila, y persiste Configuration y policy juntas. El unique constraint añade defensa. El preflight no cambia el target. El worker persiste STARTED antes de la operación remota de escritura, como en 0.5.1; el preflight previo sólo consulta metadata y permisos del destino.

El adaptador prepara y valida valores localmente. Para una tabla existente toma lock remoto y vuelve a consultar columnas antes de ALTER/DML. PostgreSQL utiliza ACCESS EXCLUSIVE en entregas auditadas; SQL Server usa el lock transaccional TABLOCKX/HOLDLOCK existente. Un cambio incompatible entre preflight y escritura se rechaza. Agregar campos y escribir filas comparten commit/rollback. No se abre una transacción DDL independiente previa, ni existe transacción distribuida con la metadata de Trackvance.

Acción	Permiso Trackvance adicional	Privilegio SQL
Publicar/encolar CREATE_TABLE	delivery:alter_target	CREATE TABLE y schema según selección
Activar auditoría sin materialización confirmada	delivery:alter_target	ALTER si falta alguna columna
OVERWRITE	delivery:overwrite	Privilegios de la estrategia y visibilidad de metadata
Entrega auditada ya materializada	Permisos ordinarios de configuración/ejecución	Privilegios de estrategia; campos deben existir

La dependencia RBAC no convierte delivery:execute en permiso de ALTER. Los permisos se comprueban en backend; ser dueño de una conexión o poder leer su metadata tampoco equivale a poder administrar roles. El administrador del target debe otorgar privilegios SQL apropiados. PostgreSQL ALTER requiere ownership efectivo/superusuario; SQL Server consulta HAS_PERMS_BY_NAME para ALTER.

Drift, fallos y estado UNKNOWN

Estado observado	Respuesta
Policy ausente, auditoría false	Comportamiento compatible con 0.5.1
Policy required, draft false	AUDIT_COLUMNS_REQUIRED
Falta columna, nunca se confirmó materialización	Puede agregarse dentro de la nueva entrega deliberada
Falta columna tras materialización confirmada	AUDIT_COLUMNS_DRIFT; no recreación automática
Tipo externo incompatible	AUDIT_COLUMNS_INCOMPATIBLE; intervención explícita
Mapping intenta escribir campo técnico	AUDIT_MAPPING_COLLISION
ALTER remoto insuficiente	FAILED_PRECONDITION con check AUDIT_ALTER_PERMISSION
Commit remoto no confirmado	UNKNOWN; policy requerida, materialización no afirmada

UNKNOWN sigue siendo evidencia de incertidumbre, no prueba de rollback ni de commit. El intento no se reintenta automáticamente al reiniciar. Su snapshot conserva timestamp y username, pero `columns_created` permanece null porque no se conoce el resultado confirmado. Una nueva Run deliberada inspecciona el target; si adopta campos compatibles y confirma su propio commit, marca policy materialized sin reescribir el intento UNKNOWN anterior.



Contratos y evidencia

El nuevo GET `/api/v1/delivery/destinations/{id}/target-policy` recibe `schema_name/table_name` y `destination_version_id` opcional. Permite consultar policy antes de crear una tabla; no abre conexión SQL ni necesita secretos. Devuelve `audit_columns_required`, `policy_id`, `materialized_at` y `target_fingerprint`. Se aplica scope de organización y `delivery:read`.

DeliveryDraft agrega `audit_columns_enabled`; preflight devuelve `system_audit` con `enabled`, `policy_id`, nombres reales, campos faltantes y `materialized_at`. Intentos, receipt y sección delivery del manifest incluyen un snapshot consistente:

```

{
  "enabled": true,
  "fecha_ingesta": "2026-09-26T12:34:56.123456+00:00",
  "username": "operador.interno",
  "policy_id": "uuid-policy",
  "columns": {"fecha_ingesta": "fechaIngesta", "usuario": "usuario"},
  "columns_created": true
}
  
```

COMMITTED y materialized se persisten antes de publicar archivos locales. Un fallo de ArtifactStore no provoca replay; reparación local conserva los valores persistidos y verifica su consistencia con Run/Attempt. Configuraciones históricas sin el nuevo flag mantienen sus hashes canónicos y no se reescriben manifests para agregar campos. El linaje añade AUDITED_TARGET entre Run y policy.

La auditoría registra DELIVERY_TARGET_AUDIT_ENABLED, DELIVERY_TARGET_AUDIT_COLUMNS_CREATED y DELIVERY_TARGET_AUDIT_DRIFT. La metadata permitida incluye IDs y fingerprint, nunca credenciales SQL/SSO, bodies de notificaciones, tokens ni datos de negocio completos.

Backup/restore, pruebas y límites

Backups incluyen política y snapshots de intentos en la misma base de metadata, junto con roles, usuarios y artifacts. .env continúa fuera del archivo; configuración y secretos de cliente SSO siguen configurándose externamente. Un restore debe conservar el requisito incluso si el destino configurado rota su password. El downgrade controlado de metadata no borra columnas en servidores externos.

Las pruebas de unidad/API comprueban fingerprint, publicación atómica, rechazo de deshabilitación, tipos, permisos, snapshots, evidencia, UNKNOWN y rollback. El runner scripts/tests/delivery_cycle.py usa proyectos/volúmenes exclusivos, PostgreSQL y SQL Server reales y OIDC firmado desechable en overlays, sin Mailpit. Certifica CREATE con/sin auditoría, APPEND, OVERWRITE, UPSERT INSERT/UPDATE, históricos NULL, adopción parcial/completa, ALTER denegado, mapping reservado, drift, receipt/manifest, usuarios local/Microsoft/Google y restart.

La prueba UNKNOWN hace commit real y pierde deliberadamente el acknowledgement del adaptador; su nombre y evidencia declaran la simulación. No certifica un fallo de red físico ni a Google/Microsoft reales. El resultado ejecutado de cada matriz histórica queda en docs/development/evidence/0.6.1/; la evidencia vigente está en 0.7.0; este documento describe casos, no convierte escenarios preparados en PASS. Los providers reales requieren credenciales externas y su estado se informa por separado.

La auditoría de target incorporada en 0.6.0 no agregó scheduling Delivery, nuevos DataSink, SHIST, masking, retención avanzada ni gestores empresariales de secretos. La automatización Delivery y las nuevas cotas de volumen se implementan en 0.7.0 y se describen en los capítulos vigentes. Se conserva la semántica de rows_written: mide filas fuente enviadas en un commit confirmado, no un inventario remoto post-trigger. La inspección y locks protegen nuestras transacciones; los cambios fuera de Trackvance pertenecen al gobierno del target.

31. Antecedente: cambios funcionales y técnicos 0.6.0

Este capítulo conserva la decisión y alcance de 0.6.0. El envío SMTP aquí descrito fue retirado deliberadamente en 0.6.1; no representa una capacidad vigente ni una configuración habilitable. La validación histórica está en evidence/0.6.0.

Este capítulo une una situación anterior, riesgo, decisión y solución con los contratos desarrollados en los capítulos anteriores. El estado de pruebas se toma del informe ejecutado, nunca de los requisitos o de una certificación histórica.

A. Roles y catálogo de permisos

Aspecto	Cambio y evidencia de diseño
1. Situación 0.5.1	Un nombre en User resolvía permisos estáticos; rutas desconocidas caían en runs:read.
2. Problema	No había administración de roles ni propagación fiable a sesiones existentes.
3. Riesgo	Exceso de privilegios por fallback, sesión con permisos obsoletos o escalación por rol custom.
4. Decisión funcional	Administrar roles, conservar catálogo de producto y proteger Administrator.
5. Solución técnica	Matriz explícita, dependencias, resolución request-by-request y /me refrescado.
6. Persistencia	Role, RolePermission y User.role_id; system_key identifica Administrator.
7. Migración	0010 asigna equivalencias y username sin inventar nombres ni reescribir actores.
8. API	CRUD lógico /roles, catálogo /roles/permissions y DTO role_version.
9. UI	Roles agrupados por función; dependencias, usuarios asociados y estado protegido visibles.
10. Seguridad	users:manage/roles:manage no delegables; último admin serializado por organización.
11. Compatibilidad	Cinco roles base y Data Owner histórico conservan accesos funcionales.
12. E2E	Rol custom, modificación en sesión activa, revocación al reasignar y API manipulada.
13. Resultados	Tabla de validación vigente y JSON 0.6.0; no heredar conteos de 0.5.1.
14. Limitaciones	Permisos por recurso individual, aprobación de cuatro ojos y gobierno ampliado pendientes.
15. Pendientes	Dominios/grupos administrables y mapeos externos requieren otra decisión de producto.

B. Usuarios, credenciales y SSO

Aspecto	Cambio y evidencia de diseño
1. Situación 0.5.1	Login email/password y contraseñas introducidas por administrador, sin federación.
2. Problema	Incorporación manual, sin expiración temporal ni identidad externa estable.
3. Riesgo	Passwords expuestas, cuentas externas ambiguas, replay y confundir autenticación con autorización.
4. Decisión funcional	Username/email local, temporal 24h, primer acceso y proveedores preprovisionados.
5. Solución técnica	Argon2, sesión restringida, rotación; Code Flow/PKCE y JOSE mantenido.
6. Persistencia	User aditivo, ExternalIdentity provider/issuer/subject y state efímero consumible.
7. Migración	Username determinista, colisiones controladas, campos personales legacy nullable.
8. API	Login compatible, first-login, regeneración, providers/start/callback y desvinculación.
9. UI	Usuario o correo, botones habilitados por configuración, cambio obligatorio y métodos de acceso.
10. Seguridad	State/nonce/aud/iss/firma/exp, cookie HttpOnly, CSRF, no auto-provisioning ni roles externos.
11. Compatibilidad	Email sigue autenticando, todos mantienen contraseña local y actores históricos.
12. E2E	Mailpit y navegador con Microsoft personal/organizacional y Google Gmail/Workspace simulados.
13. Resultados	Casos reales externos sin credenciales: NOT_RUN_EXTERNAL_CREDENTIALS. Mock tiene resultado separado.
14. Limitaciones	Políticas de consentimiento/MFA externas y autoridad del primer identificador requieren configuración.
15. Pendientes	Gobierno de múltiples organizaciones y mapeo de grupos no se implementan.

C. Notificaciones

Aspecto	Cambio y evidencia de diseño
1. Situación 0.5.1	Puerto preparado sin email operativo.
2. Problema	No existía entrega reutilizable para incorporación segura.
3. Riesgo	Crear motores paralelos o persistir cuerpos con contraseñas.
4. Decisión funcional	Un solo servicio/puerto; primera notificación USER_TEMPORARY_CREDENTIALS.
5. Solución técnica	Mensaje en memoria, adaptador SMTP TLS, errores codificados.
6. Persistencia	NotificationDeliveryRecord conserva destinatario, intento, estado y fechas, nunca body.
7. Migración	0011 crea tabla independiente sin alterar usuarios históricos.
8. API	Estado de integración y listado de entregas; alta/regeneración devuelve resultado saneado.
9. UI	Notificaciones y estado de credenciales visibles, sin vista de password.
10. Seguridad	Secretos sólo en entorno API; no logs de SMTP exception ni cuerpos persistidos.
11. Compatibilidad	SMTP deshabilitado no bloquea arranque; alta conserva User con FAILED.
12. E2E	Recepción real Mailpit, uso temporal, regeneración y escaneo DB/log de plaintext.
13. Resultados	Evidencia ejecutada en identity-sso-e2e; proveedor externo SMTP depende de configuración.
14. Limitaciones	SENT es aceptación SMTP; interrupción puede dejar PENDING y exige regenerar.
15. Pendientes	ROLE/USERS/DOMAIN/GROUP, alertas de ejecuciones y SMTP OAuth2.

D. Auditoría de publicación

Aspecto	Cambio y evidencia de diseño
1. Situación 0.5.1	Delivery transaccional con evidencia por intento, sin campos técnicos en cada fila.
2. Problema	El target no atribuía la última ingesta a usuario/instante Trackvance.
3. Riesgo	Auditoría falseable por mapping, falsa fecha en filas anteriores o deshabilitación desde otra configuración.
4. Decisión funcional	Activación conjunta e irreversible de fechaIngesta/usuario por target.
5. Solución técnica	Fingerprint estable, preflight, ALTER+DML remoto único y revisión bajo lock.
6. Persistencia	DeliveryTargetPolicy y DeliveryAttempt.system_audit; username snapshot de Run.
7. Migración	0012 aditiva; intentos históricos con objeto vacío, sin metadata inventada.
8. API	audit_columns_enabled, target-policy y metadata en preflight/attempt/receipt/manifest.
9. UI	Checkbox conjunto y bloqueado al existir policy, explicación de columnas reservadas.
10. Seguridad	delivery:alter_target, overwrite separado y privilegios SQL efectivos.
11. Compatibilidad	Audit false sin policy conserva semántica y hashes históricos. DatasetVersion no cambia.
12. E2E	PG/SQL Server reales; estrategias, adopción/drift, identidad local/SSO, UNKNOWN y restore.
13. Resultados	Runner real con fallo de acknowledgement inyectado; no se declara una pérdida física de red certificada.
14. Limitaciones	No intercepta escritores externos, alias DNS no se unifican y SQL Server casefold es conservador.
15. Pendientes	Scheduling Delivery, SHIST/SCD, targets nuevos y gobierno ampliado fuera de alcance.

Resultados del antecedente

Los resultados 0.6.0 permanecen en `validation_results_0.6.0.json` y `evidence/0.6.0`. La validación de este antecedente corresponde a 0.6.0; capítulo 35 conserva 0.6.1. El capítulo 22 contiene la evidencia propia de 0.7.0.

32. Configuración y prueba real SSO

Estas instrucciones permiten ejecutar pruebas externas opt-in. El mock de CI certifica el cliente OIDC de Trackvance; no certifica cuentas ni servicios reales. Mientras no se hayan configurado credenciales y ejecutado cada caso, el resultado es `NOT_RUN_EXTERNAL_CREDENTIALS`. Nunca escribas secretos en una captura, ticket, commit, comando compartido o archivo de evidencia.

Preparación común

1. Confirma el origen público real de la instalación. En el equipo revisado es `http://localhost:3100`; otro entorno debe usar su propia URL. Establece `TRACKVANCE_PUBLIC_URL` y `TRACKVANCE_WEB_ORIGIN` coherentes en el `.env` privado. En HTTPS las cookies deben viajar por HTTPS. No uses URLs con credenciales, query strings ni fragmentos como URL pública.
2. En Configuración → Usuarios crea primero la cuenta con el correo exacto de la identidad que vas a probar y un rol de Trackvance. No existe auto-provisioning. La temporal se muestra una sola vez al administrador, sin envío de correo. El primer acceso seguirá exigiendo definir una contraseña local.
3. Registra una aplicación Web con callback exacto para cada proveedor. El secreto permanece sólo en la API, no en React ni en los workers. No habilites `implicit grant` ni scopes de correo/calendario/Graph.
4. Configura las variables descritas abajo y recrea la API con Compose. Consulta Configuración → Autenticación o `/api/v1/auth/providers`: sólo debe aparecer un proveedor que esté habilitado y completo. Readiness no prueba Internet.
5. Utiliza una ventana privada para aislar las cuentas del proveedor. No mezcles `localhost` y `127.0.0.1` durante un mismo flujo: state está ligado al navegador.

A. Cuenta personal Microsoft: Outlook, Hotmail, Live

1. Abre el [centro de administración de Microsoft Entra](#), entra en App registrations y registra una aplicación para Trackvance.
2. En Supported account types selecciona **Accounts in any organizational directory and personal Microsoft accounts**. La audiencia correspondiente es `AzureADandPersonalMicrosoftAccount`. Una aplicación sólo de un tenant o sólo organizacional no cubre esta prueba.
3. Añade plataforma **Web**. Para la instalación de ejemplo registra `http://localhost:3100/api/v1/auth/sso/microsoft/callback`. No añadas parámetros de consulta a la URI personal. No registres el callback como SPA.
4. Copia Application (client) ID. En Certificates & secrets crea un client secret con vigencia administrada y copia su **valor**, no su ID, al `.env` privado.
5. Configura `TRACKVANCE_SSO_MICROSOFT_ENABLED=true`, `TRACKVANCE_SSO_MICROSOFT_CLIENT_ID` y `TRACKVANCE_SSO_MICROSOFT_CLIENT_SECRET`. Trackvance usa la autoridad common para Code Flow + PKCE; no pide Microsoft Graph.
6. Preprovisiona el correo personal que el proveedor devuelve. Pulsa **Continuar con Microsoft** e inicia sesión con esa cuenta. El primer enlace personal requiere el tenant consumer de Microsoft y un dominio personal reconocido (Outlook/Hotmail/Live/MSN); otras identidades ambiguas se rechazan.

7. Si es primer acceso, define una contraseña local nueva. Confirma el mismo

username/rol interno en /me, luego consulta Métodos de acceso en Usuarios: Microsoft vinculado y último acceso. La fecha de enlace está disponible como `external_identities[].linked_at` en el DTO del usuario.

La configuración de registro y audiencia sigue la [guía oficial de registro de aplicaciones](#).

B. Microsoft Entra work/school

Usa la misma aplicación con audiencia organizacional + personal y callback Web. Preprovisiona el correo de la cuenta organizacional. El primer vínculo exige prueba de dominio verificado (`xms_edov`) o que el tenant UUID figure explícitamente en `TRACKVANCE_SSO_MICROSOFT_TRUSTED_TENANTS`, separado por comas. Declara únicamente tenants cuyo gobierno de identidades controlas o confías: permitir un tenant significa confiar en sus afirmaciones de identidad para este primer enlace. No uses comodines ni añadas un tenant desconocido para hacer pasar un test.

Pulsa Microsoft, selecciona la cuenta corporativa y completa los requisitos del tenant (consentimiento, MFA o políticas de acceso). Trackvance no administra esas políticas ni toma roles/grupos de ellas. Confirma User, username y permisos internos. El issuer validado corresponde al tenant concreto, aunque discovery inicial use common; la firma debe provenir de una clave permitida para ese issuer. Consulta las [reglas de validación de claims de Microsoft](#).

C. Gmail personal

1. En [Google Cloud Console](#) selecciona un

proyecto de integración propio. Configura Google Auth Platform/consent screen: nombre, contacto de soporte y audiencia apropiada. Si el proyecto está en modo de pruebas externo, añade la cuenta Gmail a los test users.

2. Crea un OAuth client de tipo **Web application**. Añade como Authorized redirect

URI `http://localhost:3100/api/v1/auth/sso/google/callback`, sustituyendo origen por el real. Esta integración backend no necesita Gmail API.

3. Guarda client ID y client secret en el `.env` privado mediante

```
TRACKVANCE_SSO_GOOGLE_CLIENT_ID y TRACKVANCE_SSO_GOOGLE_CLIENT_SECRET; establece  
TRACKVANCE_SSO_GOOGLE_ENABLED=true.
```

4. Preprovisiona el Gmail exacto en Trackvance. Pulsa **Continuar con Google**,

selecciona esa cuenta y concede sólo `openid/email/profile`. Define contraseña local si la sesión es de primer acceso. Confirma Google vinculado y rol local.

El cliente Web y la redirect URI se documentan en el [flujo OAuth para aplicaciones Web](#). La [referencia OIDC](#) describe issuer/sub y los claims de email.

D. Google Workspace

Reutiliza un cliente Web cuya audiencia/consentimiento permita ese dominio. Una aplicación de audiencia Internal puede limitarse a su propia organización; para probar también Gmail debe elegirse una configuración que admita ambas cuentas. El administrador Workspace puede necesitar aprobar el cliente.

Preprovisiona el email corporativo y autentica desde Google. El enlace inicial exige `email_verified=true` y `hd` compatible con el dominio del email. Una Google Account creada con un correo externo, sin Gmail ni `hd`, no se vincula automáticamente: `email_verified` por sí solo no garantiza que Google siga siendo autoridad sobre ese correo. La razón está en la [guía de autenticación backend de Google](#).

Verificación y desvinculación

Después de cada prueba consulta el usuario y los eventos `SSO_LINKED` y `SSO_LOGIN_SUCCESS`. Compara el `user_id` interno antes/después; no debe crearse un User adicional. Un segundo login debe usar la misma ExternalIdentity. Cambiar permisos del Role afecta la próxima petición; cambiar el Role revoca sesión. Desactivar o eliminar lógicamente User impide tanto login local como SSO.

Un Administrator abre Configuración → Usuarios → Métodos de acceso y desvincula Microsoft o Google. Esto elimina únicamente el vínculo de autenticación, conserva User e historia y revoca las sesiones afectadas según el contrato. Para volver a enlazar se aplica de nuevo la validación inicial; no se heredan grupos externos.

Guarda por caso una evidencia saneada: proveedor, tipo de cuenta, hora, commit, resultado, user_id de fixture, permisos comprobados y error genérico si falló. No guardes callback completo, código, tokens, cookies, ID token ni capturas de secretos. En `real-provider-results.json` distingue PASS, FAIL y NOT_RUN con motivo.

Errores frecuentes

Síntoma	Comprobación
No aparece botón	Enabled=true, client ID y secret presentes, URL pública válida; recrear API.
redirect_uri_mismatch	Esquema, host, puerto y path exactos; cliente Web, no SPA.
Cuenta personal Microsoft rechazada	Audiencia AzureADandPersonalMicrosoftAccount, autoridad common y correo personal permitido.
Cuenta organizacional sin vínculo	Correo preprovisionado y autoridad del dominio; xms_edov o tenant UUID explícitamente confiable.
Google external email rechazado	Falta autoridad Gmail/Workspace; no saltar la validación.
State o nonce inválido	Flujo vencido/reutilizado, cookies ausentes o mezcla de hosts; iniciar un flujo nuevo.
Usuario no habilitado	Cuenta inexistente o desactivada/eliminada; no se auto-crea. Un rol inactivo no concede permisos de negocio.
Sólo aparece cambio de contraseña	Es primer acceso obligatorio, incluso por SSO.
Error de red o firma	Discovery/JWKS accesibles, reloj del host correcto, secret vigente; no desactivar validación.

La prueba real depende de credenciales y consentimiento externos. La suite `identity-sso-e2e` usa un mock desechable y no requiere esas credenciales.

33. Operación de credenciales y retiro de SMTP

Administrator crea/regenera credenciales mediante sesión/CSRF y permisos protegidos. Se muestra temporal una sola vez en modal; cerrar elimina plaintext del estado de la UI. Sólo Argon2 permanece en DB, con expiración y cambio obligatorio. El administrador entrega externamente por el canal de su organización; Trackvance no envía password por correo ni incluye temporal en una notificación interna.

SMTP/Mailpit y adaptadores de emisión fueron retirados en 0.6.1 y no se reactivan en 0.7.0. `notification_deliveries` conserva sólo historia. La bandeja nueva informa procesos del producto mediante outbox/inbox personal; ese cambio no revierte la decisión de credenciales efímeras. Microsoft/Google SSO siguen opcionales y deshabilitados por defecto; nunca se deduce una identidad sólo por un correo no verificado o se inventa propietario de una programación.

Secrets fuente/destino se almacenan cifrados en SecretStore independientes, con sus claves fuera de Git y sin exponerlos en JSON/API/logs. Los backups privados incluyen claves necesarias separadas del env. Restaurar pruebas no reutiliza credenciales reales. Las claves no se montan en los executors Spark ni procesos ligeros, y el worker de calidad sólo recibe snapshots inmutables.

34. Matriz completa de autorización

La tabla se genera desde permissions.py del runtime 0.7.0. La autoridad backend incluye clausura transitiva, organización, propietario, CSRF, primer acceso, recurso activo y controles de estrategia. Administrator continúa protegido; no tiene acceso automático a inbox/validaciones personales ajenas.

Código	Dependencias / delegable
artifacts:download	; Sí
audit:read	; Sí
connections:manage	connections:read; Sí
connections:read	; Sí
connections:use	connections:read; Sí
datasets:read	; Sí
datasets:write	datasets:read; Sí
delivery:alter_target	delivery:execute, delivery:read; Sí
delivery:configure	datasets:read, delivery:read, destinations:read, destinations:use; Sí
delivery:execute	datasets:read, delivery:read; Sí
delivery:overwrite	delivery:execute, delivery:read; Sí
delivery:read	; Sí
delivery:repair_evidence	delivery:execute, delivery:read; Sí
delivery:review_unknown	delivery:read; Sí
delivery:schedule	delivery:configure, delivery:execute, delivery:read; Sí
destinations:manage	destinations:read; Sí
destinations:read	; Sí
destinations:use	destinations:read; Sí
exceptions:close	exceptions:read, exceptions:write; Sí
exceptions:read	; Sí
exceptions:write	exceptions:read; Sí
exports:download	; Sí
intake:configure	datasets:read, intake:read; Sí
intake:execute	datasets:read, intake:read; Sí
intake:read	; Sí
notifications:manage	notifications:read; Sí
notifications:read	; Sí
recon:configure	datasets:read, recon:read; Sí
recon:execute	datasets:read, recon:read; Sí
recon:read	; Sí
roles:manage	roles:read; No
roles:read	; Sí

Código	Dependencias / delegable
rules:read	; Sí
runs:execute	runs:read; Sí
runs:read	; Sí
sentinel:configure	datasets:read, sentinel:read; Sí
sentinel:execute	datasets:read, sentinel:read; Sí
sentinel:read	; Sí
sentinel:schedule	sentinel:execute, sentinel:read; Sí
system:read	; Sí
users:manage	users:read; No
users:read	; Sí

Método	Ruta	Permiso
DELETE	/connections/{id}	connections:manage
DELETE	/delivery/destinations/{id}	destinations:manage
DELETE	/roles/{id}	roles:manage
DELETE	/users/{id}	users:manage
DELETE	/users/{id}/external-identities/{identity_id}	users:manage
GET	/acquisitions	datasets:read
GET	/acquisitions/{id}	datasets:read
GET	/artifacts/{id}/download	artifacts:download
GET	/audit-events	audit:read
GET	/connections	connections:read
GET	/connections/{id}	connections:read
GET	/connections/{id}/objects	connections:use
GET	/connections/{id}/preview	connections:use
GET	/connections/{id}/schemas	connections:use
GET	/dashboard	runs:read
GET	/dataset-versions/{id}/profile	datasets:read
GET	/datasets	datasets:read
GET	/datasets/uploads/{id}/inspect	datasets:write
GET	/datasets/{id}	datasets:read
GET	/datasets/{id}/schema	datasets:read
GET	/delivery/automations	delivery:read
GET	/delivery/automations/{id}	delivery:read
GET	/delivery/automations/{id}/occurrences	delivery:read
GET	/delivery/configurations	delivery:read
GET	/delivery/destinations	destinations:read

Método	Ruta	Permiso
GET	/delivery/destinations/{id}	destinations:read
GET	/delivery/destinations/{id}/schemas	destinations:use
GET	/delivery/destinations/{id}/table-metadata	destinations:use
GET	/delivery/destinations/{id}/tables	destinations:use
GET	/delivery/destinations/{id}/target-policy	delivery:read
GET	/delivery/runs	delivery:read
GET	/delivery/runs/{id}/attempts	delivery:read
GET	/delivery/runs/{id}/receipt	artifacts:download
GET	/delivery/runs/{id}/reviews	delivery:read
GET	/delivery/validations	delivery:read
GET	/delivery/validations/{id}	delivery:read
GET	/exceptions	exceptions:read
GET	/exceptions/assignees	exceptions:read
GET	/exceptions/{id}	exceptions:read
GET	/exceptions/{id}/attachments/{attachment_id}/download	artifacts:download
GET	/findings	runs:read
GET	/intake/contracts	intake:read
GET	/intake/runs/{id}/errors	intake:read
GET	/monitors	sentinel:read
GET	/monitors/{id}/alerts	sentinel:read
GET	/monitors/{id}/metrics	sentinel:read
GET	/monitors/{id}/occurrences	sentinel:read
GET	/monitors/{id}/schedule	sentinel:read
GET	/monitors/{id}/series	sentinel:read
GET	/notifications/deliveries	notifications:read
GET	/notifications/inbox	notifications:read
GET	/notifications/status	notifications:read
GET	/notifications/unread-count	notifications:read
GET	/recon/controls	recon:read
GET	/recon/runs/{id}/results	recon:read
GET	/roles	roles:read
GET	/roles/permissions	roles:read
GET	/roles/{id}	roles:read
GET	/rules	rules:read
GET	/runs	runs:read
GET	/runs/{id}	runs:read
GET	/runs/{id}/diagnostics	runs:read

Método	Ruta	Permiso
GET	/runs/{id}/evidence	artifacts:download
GET	/runs/{id}/execution-plan	runs:read
GET	/runs/{id}/export.csv	exports:download
GET	/runs/{id}/export.xlsx	exports:download
GET	/runs/{id}/results	runs:read
GET	/system/engines	system:read
GET	/users	users:read
GET	/users/roles	users:read
GET	/users/{id}	users:read
PATCH	/connections/{id}	connections:manage
PATCH	/delivery/destinations/{id}	destinations:manage
PATCH	/exceptions/{id}	exceptions:write
PATCH	/roles/{id}	roles:manage
PATCH	/users/{id}	users:manage
POST	/acquisitions/{id}/cancel	datasets:write
POST	/connections	connections:manage
POST	/connections/test	connections:manage
POST	/connections/{id}/acquisitions	connections:use
POST	/connections/{id}/datasets	connections:use
POST	/connections/{id}/test	connections:use
POST	/datasets	datasets:write
POST	/datasets/uploads/inspect	datasets:write
POST	/datasets/uploads/stage	datasets:write
POST	/datasets/{id}/acquisitions	datasets:write
POST	/datasets/{id}/acquisitions/refresh	connections:use
POST	/datasets/{id}/refresh-source	connections:use
POST	/datasets/{id}/versions/upload	datasets:write
POST	/delivery/automations	delivery:schedule
POST	/delivery/automations/{id}/dispatch	delivery:execute
POST	/delivery/automations/{id}/versions	delivery:schedule
POST	/delivery/configurations	delivery:configure
POST	/delivery/configurations/{id}/versions	delivery:configure
POST	/delivery/destinations	destinations:manage
POST	/delivery/destinations/test	destinations:manage
POST	/delivery/destinations/{id}/test	destinations:use
POST	/delivery/preflight	delivery:configure
POST	/delivery/preview	delivery:configure

Método	Ruta	Permiso
POST	/delivery/runs	delivery:execute
POST	/delivery/runs/{id}/repair-evidence	delivery:repair_evidence
POST	/delivery/runs/{id}/resume-target	delivery:review_unknown
POST	/delivery/runs/{id}/reviews	delivery:review_unknown
POST	/delivery/validations	delivery:configure
POST	/delivery/validations/{id}/cancel	delivery:configure
POST	/exceptions/{id}/attachments	exceptions:write
POST	/exceptions/{id}/comments	exceptions:write
POST	/exceptions/{id}/validate	exceptions:write
POST	/execution-plans/preview	runs:execute
POST	/findings/{id}/exceptions	exceptions:write
POST	/intake/contracts	intake:configure
POST	/intake/contracts/{id}/versions	intake:configure
POST	/intake/runs	intake:execute
POST	/monitors	sentinel:configure
POST	/monitors/{id}/runs	sentinel:execute
POST	/monitors/{id}/schedule	sentinel:schedule
POST	/monitors/{id}/versions	sentinel:configure
POST	/notifications/inbox/read-all	notifications:read
POST	/notifications/inbox/{id}/read	notifications:read
POST	/recon/controls	recon:configure
POST	/recon/controls/{id}/versions	recon:configure
POST	/recon/runs	recon:execute
POST	/roles	roles:manage
POST	/runs/{id}/cancel	runs:execute
POST	/users	users:manage
POST	/users/{id}/regenerate-credentials	users:manage
POST	/users/{id}/resend-credentials	users:manage
POST	/users/{id}/reset-password	users:manage

La fuente SQL exige connections:use y datasets:write según la operación. Un usuario desactivado/permisos revocados no puede despachar ni pasar STARTED. La configuración efectiva guarda responsable User real y revisión del destino; no se autoriza por una etiqueta o por el actor SYSTEM del disparador. La documentación de rutas no sustituye la política aplicada a la entidad y su recurso vinculado; se prueban 401/403/404/crossorg/crossuser.

35. Antecedente: cambios funcionales y técnicos 0.6.1

Este capítulo conserva el estado de la release 0.6.1. Sus exclusiones y resultados son históricos; no describen el estado vigente 0.7.0.

1. Situación 0.6.0

0.6.0 generaba la temporal y la pasaba a NotificationService para intentar entregarla por SMTP. Persistía Argon2 e historia de entrega, con estados SENT/FAILED/PENDING. El administrador no veía la temporal. RBAC, SSO y Delivery se incorporaron en la misma release y continúan implementados. El antecedente y su PDF permanecen archivados.

2. Problema funcional

La incorporación de un usuario dependía operativamente de configurar correo, de su conectividad y de políticas del proveedor. Esa complejidad no era necesaria para la administración local. Un fallo podía dejar al usuario creado sin una temporal entregada.

3. Decisión de producto

Revertir el envío automático y mostrar la temporal una sola vez al administrador autorizado. Trackvance genera, vence y valida la credencial; el administrador decide cómo comunicarla. Notificaciones de negocio vuelven al backlog, sin canal predeterminado.

4. Solución funcional

Alta → temporal de 24h → modal → entrega externa → login → cambio obligatorio → sesión normal. Regenerar → nueva temporal → revocación de sesiones y contraseña anterior → modal → primer acceso nuevamente. Email del usuario se conserva como identidad.

5. Solución técnica

CSPRNG de 32 caracteres, hash Argon2 y campos existentes de User. El endpoint emite un DTO efímero separado; no-store/no-cache evita almacenamiento HTTP ordinario. La UI usa una llamada directa y estado local del modal, fuera de mutation cache. Cierre, desmontaje y respuesta tardía eliminan referencias controladas por la aplicación.

6. Cambios API

POST /users devuelve 201 UserCredentialIssueResponse; POST /users/{id}/regenerate-credentials devuelve 200 con el mismo envelope y exige versión. Alias legacy deprecated no envían correo. GET y UserResponse no contienen temporal ni credential_delivery. Lectura legacy de notificaciones permanece deprecated y declara HISTORICAL_ONLY. La matriz incorpora el nuevo endpoint sin delegar users:manage a roles personalizados.

7. Cambios UI

Modal de alta/regeneración con nombre, username, temporal, expiración y rol; copiar username/password/ambas, mostrar/ocultar y cerrar. La tabla presenta pendiente, definida o vencida. Se elimina Notificaciones de Configuración; Usuarios locales, Roles y permisos y Autenticación permanecen. Los botones SSO sólo aparecen con proveedores habilitados.

8. Persistencia

No hay 0013: Alembic sigue 0012_delivery_target_audit y el modelo conserva 31 tablas. 0001..0012 permanecen byte por byte. notification_deliveries y auditoría histórica no se borran ni reescriben. Alta/regeneración no insertan entregas nuevas. Data Delivery conserva políticas, columnas técnicas, receipts, manifests y linaje de 0.6.0.

9. Seguridad y límites de memoria

La temporal sólo se genera en memoria, cruza la respuesta autorizada y vive en el modal. No es un atributo recuperable de User ni se guarda en auditoría, logs, artifacts, backups o navegador persistente. Las pruebas buscan coincidencias de las temporales reales sin imprimirlas. Los informes E2E evitan captura automática de secretos incluso ante fallo. No se garantiza borrado forense de RAM ni del portapapeles externo. Perder la respuesta obliga a regenerar; un GET nunca resuelve esa pérdida.

10. Compatibilidad

El envelope de alta cambia deliberadamente respecto a 0.6.0 y los clientes deben leer user y temporary_credentials. Los alias preservan la URL, no la semántica de correo. SSO conserva OIDC, PKCE, state, nonce, JOSE y ExternalIdentity, incluido primer acceso. El login local permanece disponible. Restore nativo061/060 y heredado051 valida la compatibilidad persistente, sin atribuir ese resultado a todos los clientes HTTP previos.

11. E2E y privacidad de evidencia

Recorridos compose, identidad con cuatro perfiles OIDC mock, conexiones, Delivery y demo limpia verifican integración. El navegador captura temporales exclusivamente en RAM y valida cierre, primer acceso, regeneración y revocación. Se escanean PostgreSQL, logs, artifacts, backups, storage de navegador, URL y DOM. No se requiere Mailpit y no se ejecutan proveedores Microsoft/Google reales: NOT_RUN_EXTERNAL_CREDENTIALS.

12. Upgrade Docker real

trackvance-certification se actualiza desde 0.6.0 a 0.6.1 con backup previo verificado, sin borrar volúmenes ni modificar datos de negocio para probar. Se comprueban ausencia de trabajo activo, igualdad state 5, artifacts, secretos, reinicio, doctor, salud de cinco servicios, Alembic y versiones/footer. La creación desde UI se ensaya en entornos desechables; la instalación principal se revisa mediante navegación de lectura.

13. CI y trazabilidad documental

Se exige éxito de los nueve jobs en el commit de producto y nuevamente en el HEAD final tras documentación/PDF. La evidencia registra workflow ID, SHA y jobs. Este PDF registra el commit de implementación usado para certificar; el informe final externo al repo registra el HEAD documental y su CI para evitar una referencia circular de commit.

El primer cierre documental afb8f8d (workflow 36328842735) obtuvo ocho SUCCESS y un fallo al preparar PostgreSQL externo, antes de iniciar Trackvance. Se reprodujo una condición prematura del healthcheck por socket durante initdb; el runner ahora espera TCP del servidor definitivo. La causa exacta del fallo original no es recuperable porque se suprimió stderr. El diagnóstico nuevo conserva sólo exit code y categoría cerrada, nunca SQL ni salida cruda. La reproducción no se confunde con prueba de causalidad retrospectiva; el cierre exige repetir los nueve jobs en el HEAD corregido.

14. Limitaciones

No se prueba SSO real ni se requieren secretos externos. No hay envío de correo, notificaciones operativas, auto-provisioning o recuperación de temporales cerradas. Los benchmarks smoke validan los casos medidos, no capacidad empresarial general. La UI no controla el canal externo ni el historial de portapapeles del administrador.

15. Pendientes posteriores

Diseñar eventos funcionales y sus destinatarios/canales cuando exista una necesidad real. Decidir activación SSO por cliente y entonces planear su validación externa. Mantener en backlog gobierno, masking, retención y Secret Manager; no se implementan nuevos conectores, destinos, scheduling Delivery ni plataformas distribuidas aquí.

Resultados ejecutados 0.6.1

Resultados detallados conservados en validation_results_0.6.1.json; no certifican 0.7.0.